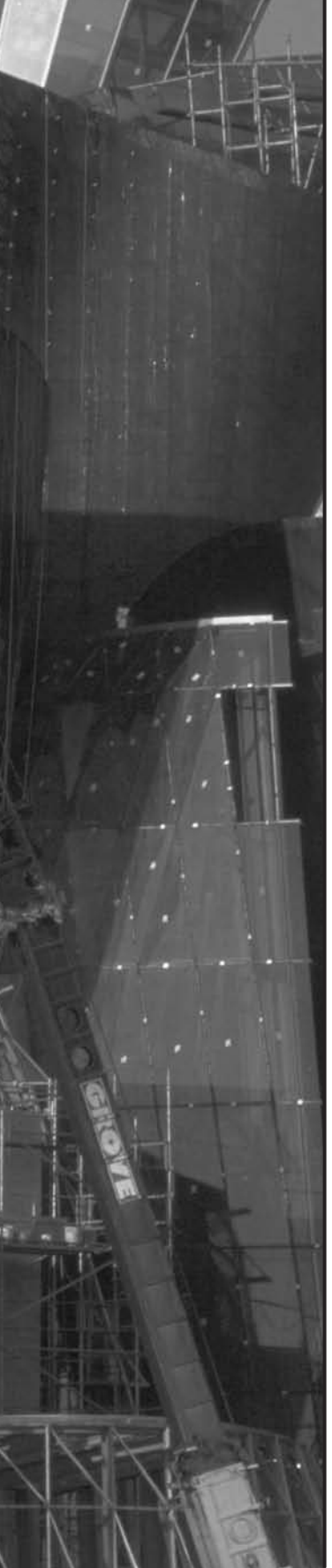


8

Pruebas de software

Objetivos

El objetivo de este capítulo es introducirlo a las pruebas del software y los procesos necesarios para tales pruebas. Al estudiar este capítulo:

- comprenderá las etapas de las pruebas, desde las pruebas durante el desarrollo hasta la prueba de aceptación por los clientes del sistema;
- se introducirá en las técnicas que ayudan a elegir casos de prueba que se ponen en funcionamiento para descubrir los defectos del programa;
- entenderá el desarrollo de la primera prueba, donde se diseñan pruebas antes de escribir el código, las cuales operan automáticamente;
- conocerá las diferencias importantes entre pruebas de componente, de sistema y de liberación, y estará al tanto de los procesos y las técnicas de prueba del usuario.

Contenido

8.1 Pruebas de desarrollo
8.2 Desarrollo dirigido por pruebas
8.3 Pruebas de versión
8.4 Pruebas de usuario

Las pruebas intentan demostrar que un programa hace lo que se intenta que haga, así como descubrir defectos en el programa antes de usarlo. Al probar el software, se ejecuta un programa con datos artificiales. Hay que verificar los resultados de la prueba que se opera para buscar errores, anomalías o información de atributos no funcionales del programa.

El proceso de prueba tiene dos metas distintas:

1. Demostrar al desarrollador y al cliente que el software cumple con los requerimientos. Para el software personalizado, esto significa que en el documento de requerimientos debe haber, por lo menos, una prueba por cada requerimiento. Para los productos de software genérico, esto quiere decir que tiene que haber pruebas para todas las características del sistema, junto con combinaciones de dichas características que se incorporarán en la liberación del producto.
2. Encontrar situaciones donde el comportamiento del software sea incorrecto, indeseable o no esté de acuerdo con su especificación. Tales situaciones son consecuencia de defectos del software. La prueba de defectos tiene la finalidad de erradicar el comportamiento indeseable del sistema, como caídas del sistema, interacciones indeseadas con otros sistemas, cálculos incorrectos y corrupción de datos.

La primera meta conduce a la prueba de validación; en ella, se espera que el sistema se desempeñe de manera correcta mediante un conjunto dado de casos de prueba, que refleje el uso previsto del sistema. La segunda meta se orienta a pruebas de defectos, donde los casos de prueba se diseñan para presentar los defectos. Los casos de prueba en las pruebas de defecto pueden ser deliberadamente confusos y no necesitan expresar cómo se usa normalmente el sistema. Desde luego, no hay frontera definida entre estos dos enfoques de pruebas. Durante las pruebas de validación, usted descubrirá defectos en el sistema; en tanto que durante las pruebas de defecto algunas de las pruebas demostrarán que el programa cumple con sus requerimientos.

El diagrama de la figura 8.1 ayuda a explicar las diferencias entre las pruebas de validación y de defecto. Piense en el sistema que va a probar como si fuera una caja negra. El sistema acepta entradas desde algún conjunto de entradas I y genera salidas en un conjunto de salidas O. Algunas de las salidas serán erróneas. Son las salidas en el conjunto O_e que el sistema genera en respuesta a las entradas en el conjunto I_e. La prioridad en las pruebas de defecto es encontrar dichas entradas en el conjunto I_e porque ellas revelan problemas con el sistema. Las pruebas de validación involucran pruebas con entradas correctas que están fuera de I_e y estimulan al sistema para generar las salidas correctas previstas.

Las pruebas no pueden demostrar que el software esté exento de defectos o que se comportará como se especifica en cualquier circunstancia. Siempre es posible que una prueba que usted pase por alto descubra más problemas con el sistema. Como afirma de forma elocuente Edsger Dijkstra, uno de los primeros contribuyentes al desarrollo de la ingeniería de software (Dijkstra *et al*., 1972):

Las pruebas pueden mostrar sólo la presencia de errores, mas no su ausencia.

Las pruebas se consideran parte de un proceso más amplio de verificación y validación (V&V) del software. Aunque ambas no son lo mismo, se confunden con frecuencia. Barry

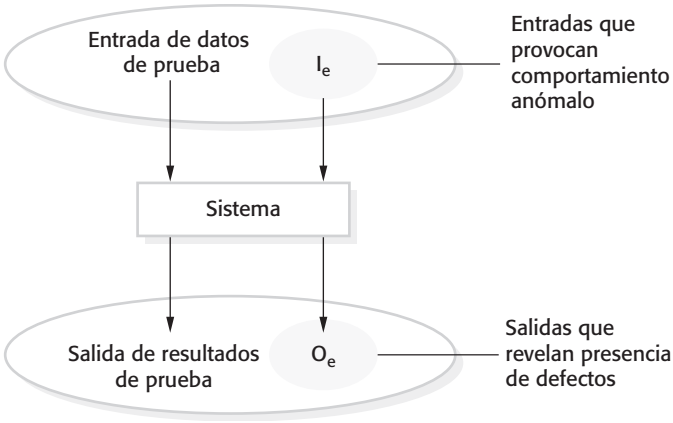


Figura 8.1 Modelo de entrada y salida de una prueba de programa

Boehm, pionero de la ingeniería de software, expresó de manera breve la diferencia entre las dos (Boehm, 1979):

- "Validación: ¿construimos el producto correcto?".
- "Verificación: ¿construimos bien el producto?".

Los procesos de verificación y validación buscan comprobar que el software por desarrollar cumpla con sus especificaciones, y brinde la funcionalidad deseada por las personas que pagan por el software. Dichos procesos de comprobación comienzan tan pronto como están disponibles los requerimientos y continúan a través de todas las etapas del proceso de desarrollo.

La finalidad de la verificación es comprobar que el software cumpla con su funcionalidad y con los requerimientos no funcionales establecidos. Sin embargo, la validación es un proceso más general. La meta de la validación es garantizar que el software cumpla con las expectativas del cliente. Va más allá del simple hecho de comprobar la conformidad con la especificación, para demostrar que el software hace lo que el cliente espera que haga. La validación es esencial pues, como se estudió en el capítulo 4, las especificaciones de requerimientos no siempre reflejan los deseos o las necesidades reales de los clientes y usuarios del sistema.

El objetivo final de los procesos de verificación y validación es establecer confianza de que el sistema de software es "adecuado". Esto significa que el sistema tiene que ser lo bastante eficaz para su uso esperado. El nivel de confianza adquirido depende tanto del propósito del sistema y las expectativas de los usuarios del sistema, como del entorno del mercado actual para el sistema:

1. *Propósito del software* Cuanto más crítico sea el software, más importante debe ser su confiabilidad. Por ejemplo, el nivel de confianza requerido para que se use el software en el control de un sistema crítico de seguridad es mucho mayor que el requerido para un prototipo desarrollado para demostrar nuevas ideas del producto.
2. *Expectativas del usuario* Debido a su experiencia con software no confiable y plagado de errores, muchos usuarios tienen pocas expectativas de la calidad del software, por lo que no se sorprenden cuando éste falla. Al instalar un sistema, los usuarios

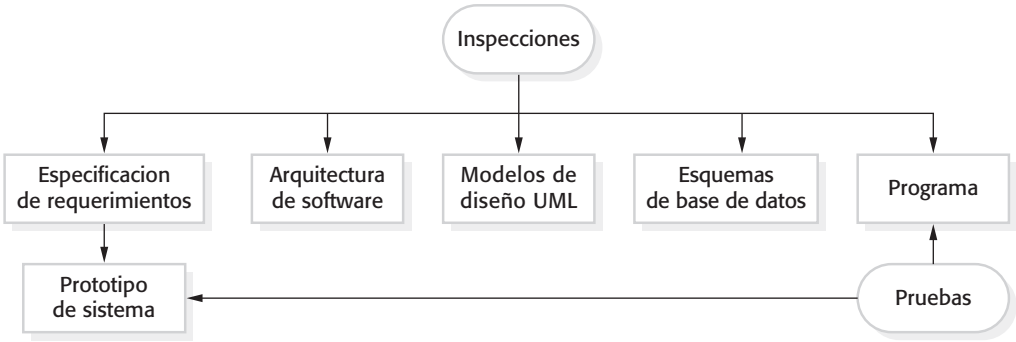


Figura 8.2 Inspecciones y pruebas

podrían soportar fallas, porque los beneficios del uso exceden los costos de la recuperación de fallas. Ante tales situaciones, no es necesario dedicar mucho tiempo en la puesta a prueba del software. Sin embargo, conforme el software se completa, los usuarios esperan que se torne más confiable, de modo que pueden requerirse pruebas exhaustivas en versiones posteriores.

3. *Entorno de mercado* Cuando un sistema se comercializa, los vendedores del sistema deben considerar los productos competitivos, el precio que los clientes están dispuestos a pagar por un sistema y la fecha requerida para entregar dicho sistema. En un ambiente competitivo, una compañía de software puede decidir lanzar al mercado un programa antes de estar plenamente probado y depurado, pues quiere que sus productos sean los primeros en ubicarse. Si un producto de software es muy económico, los usuarios tal vez toleren un nivel menor de fiabilidad.

Al igual que las pruebas de software, el proceso de verificación y validación implicaría inspecciones y revisiones de software. Estas últimas analizan y comprueban los requerimientos del sistema, los modelos de diseño, el código fuente del programa, e incluso las pruebas propuestas para el sistema. Éstas son las llamadas técnicas V&V "estáticas" donde no es necesario ejecutar el software para verificarlo. La figura 8.2 indica que las inspecciones y las pruebas del software soportan V&V en diferentes etapas del proceso del software. Las flechas señalan las etapas del proceso en que pueden usarse las técnicas.

Las inspecciones se enfocan principalmente en el código fuente de un sistema, aun cuando cualquier representación legible del software, como sus requerimientos o modelo de diseño, logre inspeccionarse. Cuando un sistema se inspecciona, se utiliza el conocimiento del sistema, su dominio de aplicación y el lenguaje de programación o modelado para descubrir errores.

Hay tres ventajas en la inspección del software sobre las pruebas:

1. Durante las pruebas, los errores pueden enmascarar (ocultar) otras fallas. Cuando un error lleva a salidas inesperadas, nunca se podrá asegurar si las anomalías de salida posteriores se deben a un nuevo error o son efectos colaterales del error original. Puesto que la inspección es un proceso estático, no hay que preocuparse por las interacciones entre errores. En consecuencia, una sola sesión de inspección descubriría muchos errores en un sistema.



Planeación de pruebas

La planeación de pruebas se interesa por la fecha y los recursos de todas las actividades durante el proceso de pruebas. Incluye la definición del proceso de pruebas, al tomar en cuenta tanto al personal como el tiempo disponible. Por lo general, se creará un plan de prueba que define lo que debe probarse, la fecha establecida de pruebas y cómo se registrarán éstas. Para sistemas críticos, el plan de prueba también puede incluir detalles de las pruebas que se van a correr en el software.

http://www.SoftwareEngineering-9.com/Web/Testing/Planning.html

2. Las versiones incompletas de un sistema se pueden inspeccionar sin costos adicionales. Si un programa está incompleto, entonces es necesario desarrollar equipos de prueba especializados para poner a prueba las partes disponibles. Evidentemente, esto genera costos para el desarrollo del sistema.

3. Además de buscar defectos de programa, una inspección puede considerar también atributos más amplios de calidad de un programa, como el cumplimiento con estándares, la portabilidad y la mantenibilidad. Pueden buscarse ineficiencias, algoritmos inadecuados y estilos de programación imitados que hagan al sistema difícil de mantener y actualizar.

Las inspecciones de programa son una idea antigua y la mayoría de estudios y experimentos indican que las inspecciones son más efectivas para el descubrimiento de defectos, que para las pruebas del programa. Fagan (1986) reportó que más del 60% de los errores en un programa se detectan mediante inspecciones informales de programa. En el proceso de Cleanroom (cuarto limpio) (Prowell *et al*., 1999) se afirma que más del 90% de los defectos pueden detectarse en inspecciones del programa.

Sin embargo, las inspecciones no sustituyen las pruebas del software, ya que no son eficaces para descubrir defectos que surjan por interacciones inesperadas entre diferentes partes de un programa, problemas de temporización o dificultades con el rendimiento del sistema. Más aún, en compañías o grupos de desarrollo pequeños, suele ser especialmente difícil y costoso reunir a un equipo de inspección separado, ya que todos los miembros potenciales del equipo también podrían ser desarrolladores de software. En el capítulo 24 (“Gestión de la calidad”) se estudian las revisiones e inspecciones con más detenimiento. En el capítulo 15 se explica el análisis estático automatizado, en el cual el texto fuente de un programa se analiza automáticamente para descubrir anomalías. Este capítulo se enfoca en las pruebas y los procesos de pruebas.

La figura 8.3 presenta un modelo abstracto del proceso de prueba “tradicional”, como se utiliza en el desarrollo dirigido por un plan. Los casos de prueba son especificaciones de las entradas a la prueba y la salida esperada del sistema (los resultados de la prueba), además de información sobre lo que se pone a prueba. Los datos de prueba son las entradas que se diseñaron para probar un sistema. En ocasiones pueden generarse automáticamente datos de prueba; no obstante, es imposible la generación automática de casos de prueba, pues debe estar implicada gente que entienda lo que se supone que tiene que hacer el sistema para especificar los resultados de prueba previstos. Sin embargo, es posible automatizar la ejecución de pruebas. Los resultados previstos se comparan automáticamente con los resultados establecidos, de manera que no haya necesidad de que un individuo busque errores y anomalías al correr las pruebas.

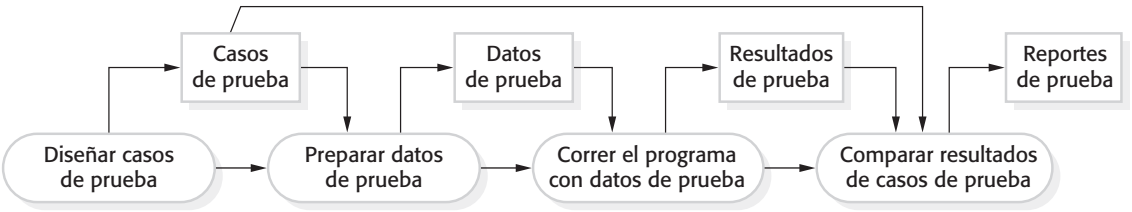


Figura 8.3 Modelo del proceso de pruebas de software

Por lo general, un sistema de software comercial debe pasar por tres etapas de pruebas:

1. Pruebas de desarrollo, donde el sistema se pone a prueba durante el proceso para descubrir errores (*bugs*) y defectos. Es probable que en el desarrollo de prueba intervengan diseñadores y programadores del sistema.
2. Versiones de prueba, donde un equipo de prueba por separado experimenta una versión completa del sistema, antes de presentarlo a los usuarios. La meta de la prueba de versión es comprobar que el sistema cumpla con los requerimientos de los participantes del sistema.
3. Pruebas de usuario, donde los usuarios reales o potenciales de un sistema prueban el sistema en su propio entorno. Para productos de software, el "usuario" puede ser un grupo interno de marketing, que decide si el software se comercializa, se lanza y se vende. Las pruebas de aceptación se efectúan cuando el cliente prueba de manera formal un sistema para decidir si debe aceptarse del proveedor del sistema, o si se requiere más desarrollo.

En la práctica, el proceso de prueba por lo general requiere una combinación de pruebas manuales y automatizadas. En las primeras pruebas manuales, un examinador opera el programa con algunos datos de prueba y compara los resultados con sus expectativas. Anota y reporta las discrepancias con los desarrolladores del programa. En las pruebas automatizadas, éstas se codifican en un programa que opera cada vez que se prueba el sistema en desarrollo. Comúnmente esto es más rápido que las pruebas manuales, sobre todo cuando incluye pruebas de regresión, es decir, aquellas que implican volver a correr pruebas anteriores para comprobar que los cambios al programa no introdujeron nuevos bugs.

El uso de pruebas automatizadas aumentó de manera considerable durante los últimos años. Sin embargo, las pruebas nunca pueden ser automatizadas por completo, ya que esta clase de pruebas sólo comprueban que un programa haga lo que supone que tiene que hacer. Es prácticamente imposible usar pruebas automatizadas para probar sistemas que dependan de cómo se ven las cosas (por ejemplo, una interfaz gráfica de usuario) o probar que un programa no presenta efectos colaterales indeseados.

8.1 Pruebas de desarrollo

Las pruebas de desarrollo incluyen todas las actividades de prueba que realiza el equipo que elabora el sistema. El examinador del software suele ser el programador que diseñó dicho software, aunque éste no es siempre el caso. Algunos procesos de desarrollo usan parejas de programador/examinador (Cusamano y Selby, 1998) donde cada programador



Depuración

Depuración (*debugging*) es el proceso para corregir los errores y problemas descubiertos por las pruebas. Al usar información de las pruebas del programa, los depuradores, para localizar y reparar el error del programa, emplean tanto su conocimiento del lenguaje de programación como el resultado que se espera de la prueba. Este proceso recibe con frecuencia apoyo de herramientas de depuración interactivas que brindan información adicional sobre la ejecución del programa.

http://www.SoftwareEngineering-9.com/Web/Testing/Debugging.html

tiene un examinador asociado que desarrolla pruebas y auxilia con el proceso de pruebas. Para sistemas críticos, puede usarse un proceso más formal, con un grupo de prueba independiente dentro del equipo de desarrollo. Son responsables del desarrollo de pruebas y del mantenimiento de registros detallados de los resultados de las pruebas.

Durante el desarrollo, las pruebas se realizan en tres niveles de granulación:

1. Pruebas de unidad, donde se ponen a prueba unidades de programa o clases de objetos individuales. Las pruebas de unidad deben enfocarse en comprobar la funcionalidad de objetos o métodos.
2. Pruebas del componente, donde muchas unidades individuales se integran para crear componentes compuestos. La prueba de componentes debe enfocarse en probar interfaces del componente.
3. Pruebas del sistema, donde algunos o todos los componentes en un sistema se integran y el sistema se prueba como un todo. Las pruebas del sistema deben enfocarse en poner a prueba las interacciones de los componentes.

Las pruebas de desarrollo son, ante todo, un proceso de prueba de defecto, en las cuales la meta consiste en descubrir bugs en el software. Por lo tanto, a menudo están entrelazadas con la depuración: el proceso de localizar problemas con el código y cambiar el programa para corregirlos.

8.1.1 Pruebas de unidad

Las pruebas de unidad son el proceso de probar componentes del programa, como métodos o clases de objetos. Las funciones o los métodos individuales son el tipo más simple de componente. Las pruebas deben llamarse para dichas rutinas con diferentes parámetros de entrada. Usted puede usar los enfoques para el diseño de casos de prueba que se estudian en la sección 8.1.2, con la finalidad de elaborar las pruebas de función o de método.

Cuando pone a prueba las clases de objetos, tiene que diseñar las pruebas para brindar cobertura a todas las características del objeto. Esto significa que debe:

- probar todas las operaciones asociadas con el objeto;
- establecer y verificar el valor de todos los atributos relacionados con el objeto;
- poner el objeto en todos los estados posibles. Esto quiere decir que tiene que simular todos los eventos que causen un cambio de estado.

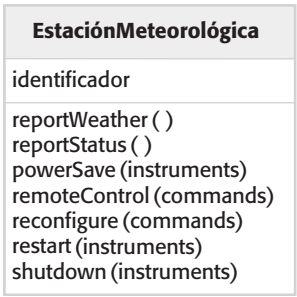


Figura 8.4 Interfaz de objeto de estación meteorológica

Considere, por ejemplo, el objeto de estación meteorológica del modelo analizado en el capítulo 7. La interfaz de este objeto se muestra en la figura 8.4. Tiene un solo atributo, que es su identificador (*identifier*). Ésta es una constante que se establece cuando se instala la estación meteorológica. Por consiguiente, sólo se necesita una prueba que demuestre si se configuró de manera adecuada. Usted necesita definir casos de prueba para todos los métodos asociados con el objeto, como reportWeather, reportStatus, etcétera. Aunque lo ideal es poner a prueba los métodos en aislamiento, en algunos casos son precisas ciertas secuencias de prueba. Por ejemplo, para someter a prueba el método que desactiva los instrumentos de la estación meteorológica (*shutdown*), se necesita ejecutar el método *restart* (reinicio).

La generalización o herencia provoca que sea más complicada la prueba de las clases de objetos. Usted no debe poner únicamente a prueba una operación en la clase donde se definió, ni suponer que funcionará como se esperaba en las subclases que heredan la operación. La operación que se hereda puede hacer conjeturas sobre otras operaciones y atributos. Es posible que no sean válidas en algunas subclases que hereden la operación. Por consiguiente, tiene que poner a prueba la operación heredada en todos los contextos en que se utilice.

Para probar los estados de la estación meteorológica, se usa un modelo de estado, tal como el que se muestra en la figura 7.8 del capítulo anterior. Al usar este modelo, identificará secuencias de transiciones de estado que deban probarse y definirá secuencias de eventos para forzar dichas transiciones. En principio, hay que probar cualquier secuencia posible de transición de estado, aunque en la práctica ello resulte muy costoso. Los ejemplos de secuencias de estado que deben probarse en la estación meteorológica incluyen:

Shutdown → Running → Shutdown
Configuring → Running → Testing → Transmitting → Running
Running → Collecting → Running → Summarizing → Transmitting → Running

Siempre que sea posible, se deben automatizar las pruebas de unidad. En estas pruebas de unidad automatizadas, podría usarse un marco de automatización de pruebas (como JUnit) para escribir y correr sus pruebas de programa. Los marcos de pruebas de unidad ofrecen clases de pruebas genéricas que se extienden para crear casos de prueba específicos. En tal caso, usted podrá correr todas las pruebas que implementó y reportar, con frecuencia mediante alguna GUI, el éxito o el fracaso de las pruebas. Es común que toda una serie de pruebas completa opere en algunos segundos, de modo que es posible ejecutar todas las pruebas cada vez que efectúe un cambio al programa.

Un conjunto automatizado de pruebas tiene tres partes:

1. Una parte de configuración, en la cual se inicializa el sistema con el caso de prueba, esto es, las entradas y salidas esperadas.

2. Una parte de llamada (*call*), en la cual se llama al objeto o al método que se va a probar.
3. Una parte de declaración, en la cual se compara el resultado de la llamada con el resultado esperado. Si la información se evalúa como verdadera, la prueba tuvo éxito; pero si resulta falsa, entonces fracasó.

En ocasiones, el objeto que se prueba tiene dependencias de otros objetos que tal vez no se escribieron o que, si se utilizan, frenan el proceso de pruebas. Si su objeto llama a una base de datos, por ejemplo, esto requeriría un proceso de configuración lento antes de usarse. En tales casos, usted puede decidir usar objetos *mock* (simulados). Éstos son objetos con la misma interfaz como los usados por objetos externos que simulan su funcionalidad. Por ende, un objeto mock que aparenta una base de datos suele tener sólo algunos ítems de datos que se organizan en un arreglo. Por lo tanto, puede entrar rápidamente a ellos, sin las sobrecargas de llamar a una base de datos y acceder a discos. De igual modo, los objetos mock pueden usarse para simular una operación anormal o eventos extraños. Por ejemplo, si se pretende que el sistema tome acción en ciertas horas del día, su objeto mock simplemente regresará estas horas, independientemente de la hora real en el reloj.

8.1.2 Elección de casos de pruebas de unidad

Las pruebas son costosas y consumen tiempo, así que es importante elegir casos efectivos de pruebas de unidad. La efectividad significa, en este caso, dos cuestiones:

1. Los casos de prueba tienen que mostrar que, cuando se usan como se esperaba, el componente que se somete a prueba hace lo que se supone que debe hacer.
2. Si hay defectos en el componente, éstos deberían revelarse mediante los casos de prueba.

En consecuencia, hay que escribir dos tipos de casos de prueba. El primero debe reflejar una operación normal de un programa y mostrar que el componente funciona. Por ejemplo, si usted va a probar un componente que crea e inicia el registro de un nuevo paciente, entonces, su caso de prueba debe mostrar que el registro existe en una base de datos, y que sus campos se configuraron como se especificó. El otro tipo de caso de prueba tiene que basarse en probar la experiencia de donde surgen problemas comunes. Debe usar entradas anormales para comprobar que se procesan de manera adecuada sin colapsar el componente.

Aquí se discuten dos estrategias posibles que serían efectivas para ayudarle a elegir casos de prueba. Se trata de:

1. Prueba de partición, donde se identifican grupos de entradas con características comunes y se procesan de la misma forma. Debe elegir las pruebas dentro de cada uno de dichos grupos.
2. Pruebas basadas en lineamientos, donde se usan lineamientos para elegir los casos de prueba. Dichos lineamientos reflejan la experiencia previa de los tipos de errores que suelen cometer los programadores al desarrollar componentes.

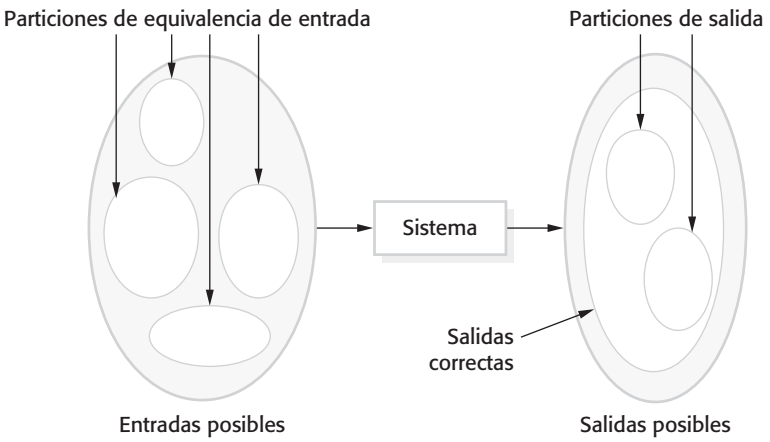


Figura 8.5 Partición de equivalencia

Los datos de entrada y los resultados de salida de un programa regularmente caen en un número de clases diferentes con características comunes. Los ejemplos de estas clases son números positivos, números negativos y selecciones de menú. Por lo general, los programas se comportan en una forma comparable a todos los miembros de una clase. Esto es, si usted prueba un programa que hace un cálculo y requiere dos números positivos, entonces esperaría que el programa se comportara de igual modo en todos los números positivos.

Debido a este comportamiento equivalente, dichas clases se llaman en ocasiones particiones de equivalencia o dominios (Bezier, 1990). Para el diseño de casos de prueba, un enfoque sistemático se basa en identificar todas las particiones de entrada y salida para un sistema o unos componentes. Los casos de prueba se elaboran de forma que las entradas o salidas se encuentren dentro de dichas particiones. La prueba de partición sirve para diseñar casos de prueba tanto para sistemas como para componentes.

En la figura 8.5, la elipse sombreada más grande ubicada en el lado izquierdo representa el conjunto de todas las entradas posibles al programa que se someterá a prueba. Las elipses más pequeñas sin sombrear constituyen particiones de equivalencia. Un programa que se ponga a prueba debe procesar de la misma forma todos los miembros de las particiones de equivalencia de entrada. Las particiones de equivalencia de salida son particiones dentro de las cuales todas las salidas tienen algo en común. En ocasiones hay un mapeo 1:1 entre particiones de equivalencia de entrada y salida. Sin embargo, éste no siempre es el caso; quizás usted necesite definir una partición de equivalencia de entrada independiente, donde la única característica común de las entradas sea que generan salidas dentro de la misma partición de salida. El área sombreada en la elipse izquierda representa excepciones que pueden ocurrir (es decir, respuestas a entradas inválidas).

Una vez identificado el conjunto de particiones, los casos de prueba se eligen de cada una de dichas particiones. Una buena regla empírica para la selección de casos de prueba es seleccionar casos de prueba en las fronteras de las particiones, además de casos cerca del punto medio de la partición. La razón es que diseñadores y programadores tienden a considerar valores típicos de entradas cuando se desarrolla un sistema. Éstos se prueban al elegir el punto medio de la partición. Los valores frontera son usualmente atípicos (por ejemplo, cero puede comportarse de manera diferente a otros números no negativos), de modo que a veces los desarrolladores los pasan por alto. Con frecuencia ocurren fallas del programa cuando se procesan estos valores atípicos.

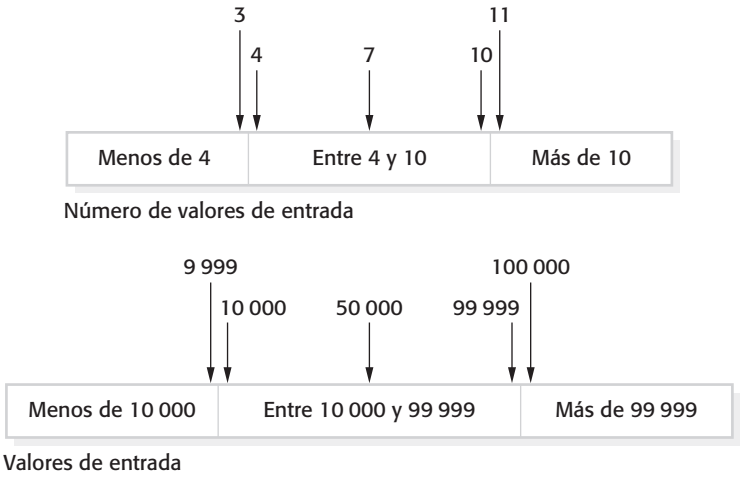


Figura 8.6 Particiones de equivalencia

Las particiones se identifican mediante la especificación del programa o la documentación del usuario y a partir de la experiencia, de donde se predicen las clases de valor de entrada que es probable que detecten errores. Por ejemplo, digamos que la especificación de un programa establece que el programa acepta de 4 a 8 entradas que son cinco dígitos enteros mayores que 10 000. Usted usa esta información para identificar las particiones de entrada y los posibles valores de entrada de prueba. Esto se muestra en la figura 8.6.

Cuando se usa la especificación de un sistema para reconocer particiones de equivalencia, se llama "pruebas de caja negra". Aquí no es necesario algún conocimiento de cómo funciona el sistema. Sin embargo, puede ser útil complementar las pruebas de caja negra con "pruebas de caja blanca", en las cuales se busca el código del programa para encontrar otras posibles pruebas. Por ejemplo, su código puede incluir excepciones para manejar las entradas incorrectas. Este conocimiento se utiliza para identificar "particiones de excepción": diferentes rangos donde deba aplicarse el mismo manejo de excepción.

La partición de equivalencia es un enfoque efectivo para las pruebas, porque ayuda a explicar los errores que cometen con frecuencia los programadores al procesar entradas en los bordes de las particiones. Usted también puede usar lineamientos de prueba para ayudarse a elegir casos de prueba. Los lineamientos encapsulan conocimiento sobre qué tipos de casos de prueba son efectivos para la detección de errores. Por ejemplo, cuando se prueban programas con secuencias, arreglos o listas, los lineamientos que pueden ayudar a revelar defectos incluyen:

1. Probar software con secuencias que tengan sólo un valor único. Los programadores naturalmente consideran a las secuencias como compuestas por muchos valores y, en ocasiones, incrustan esta suposición en sus programas. En consecuencia, si se presenta una secuencia de un valor único, es posible que un programa no funcione de manera adecuada.
2. Usar diferentes secuencias de diversos tamaños en distintas pruebas. Esto disminuye las oportunidades de que un programa con defectos genere accidentalmente una salida correcta, debido a algunas características accidentales de la entrada.
3. Derivar pruebas de modo que se acceda a los elementos primero, medio y último de la secuencia. Este enfoque revela problemas en las fronteras de la partición.



Pruebas de trayectoria

Las pruebas de trayectoria son una estrategia de prueba que se dirige principalmente a ejercitar cada trayectoria de ejecución independiente a través de un componente o programa. Si se ejecuta cualquier trayectoria independiente, entonces deben ejecutarse todos los enunciados en el componente al menos una vez. Todos los enunciados condicionales se prueban para los casos verdadero y falso. En un proceso de desarrollo orientado a objetos, la prueba de trayectoria puede usarse cuando se prueban los métodos asociados con los objetos.

http://www.SoftwareEngineering-9.com/Web/Testing/PathTest.html

El libro de Whittaker (2002) incluye muchos ejemplos de lineamientos que se pueden utilizar en el diseño de casos de prueba. Algunos de los lineamientos más generales que sugiere son:

- Elegir entradas que fuercen al sistema a generar todos los mensajes de error;
- Diseñar entradas que produzcan que los buffers de entrada se desborden;
- Repetir varias veces la misma entrada o serie de entradas;
- Forzar la generación de salidas inválidas;
- Forzar resultados de cálculo demasiado largos o demasiado pequeños.

Conforme adquiera experiencia con las pruebas, usted podrá desarrollar sus propios lineamientos sobre cómo elegir casos de prueba efectivos. En la siguiente sección de este capítulo se incluyen más ejemplos de lineamientos de prueba.

8.1.3 Pruebas de componentes

En general, los componentes de software son componentes compuestos constituidos por varios objetos en interacción. Por ejemplo, en el sistema de la estación meteorológica, el componente de reconfiguración incluye objetos que tratan con cada aspecto de la reconfiguración. El acceso a la funcionalidad de dichos objetos es a través de la interfaz de componente definida. Por consiguiente, la prueba de componentes compuestos tiene que enfocarse en mostrar que la interfaz de componente se comporta según su especificación. Usted puede suponer que dentro del componente se completaron las pruebas de unidad sobre el objeto individual.

La figura 8.7 ilustra la idea de la prueba de interfaz de componente. Suponga que los componentes A, B y C se integraron para crear un componente o subsistema más grande. Los casos de prueba no se aplican a los componentes individuales, sino más bien a la interfaz del componente compuesto, creado al combinar tales componentes. Los errores de interfaz en el componente compuesto quizá no se detecten al poner a prueba los objetos individuales, porque dichos errores resultan de interacciones entre los objetos en el componente.

Existen diferentes tipos de interfaz entre componentes de programa y, en consecuencia, distintos tipos de error de interfaz que llegan a ocurrir:

1. *Interfaces de parámetro* Son interfaces en que los datos, o en ocasiones referencias de función, pasan de un componente a otro. Los métodos en un objeto tienen una interfaz de parámetro.

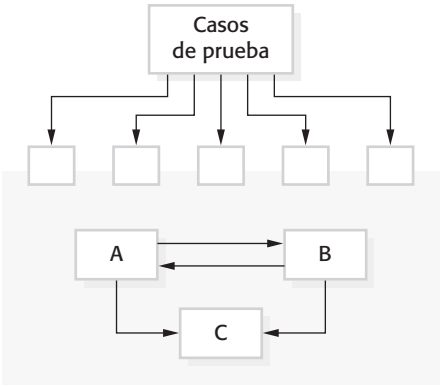


Figura 8.7 Prueba de interfaz

2. *Interfaces de memoria compartida* Son interfaces en que un bloque de memoria se reparte entre componentes. Los datos se colocan en la memoria de un subsistema y otros subsistemas los recuperan de ahí. Este tipo de interfaz se usa con frecuencia en sistemas embebidos, donde los sensores crean datos que se recuperan y son procesados por otros componentes del sistema.
3. *Interfaces de procedimiento* Son interfaces en que un componente encapsula un conjunto de procedimientos que pueden ser llamados por otros componentes. Los objetos y otros componentes reutilizables tienen esta forma de interfaz.
4. *Interfaces que pasan mensajes* Se trata de interfaces donde, al enviar un mensaje, un componente solicita un servicio de otro componente. El mensaje de retorno incluye los resultados para ejecutar el servicio. Algunos sistemas orientados a objetos tienen esta forma de interfaz, así como los sistemas cliente-servidor.

Los errores de interfaz son una de las formas más comunes de falla en los sistemas complejos (Lutz, 1993). Dichos errores caen en tres clases:

- *Uso incorrecto de interfaz* Un componente que llama a otro componente y comete algún error en el uso de su interfaz. Este tipo de error es común con interfaces de parámetro, donde los parámetros pueden ser del tipo equivocado, o bien, pasar en el orden o el número equivocados de parámetros.
- *Mala interpretación de interfaz* Un componente que malinterpreta la especificación de la interfaz del componente llamado y hace suposiciones sobre su comportamiento. El componente llamado no se comporta como se esperaba, lo cual entonces genera un comportamiento imprevisto en el componente que llama. Por ejemplo, un método de búsqueda binaria puede llamarse con un parámetro que es un arreglo desordenado. Entonces fallaría la búsqueda.
- *Errores de temporización* Ocurren en sistemas de tiempo real que usan una memoria compartida o una interfaz que pasa mensajes. El productor de datos y el consumidor de datos operan a diferentes niveles de rapidez. A menos que se tenga cuidado particular en el diseño de interfaz, el consumidor puede acceder a información obsoleta,

porque el productor de la información no actualizó la información de la interfaz compartida.

Las pruebas por defectos de interfaz son difíciles porque algunas fallas de interfaz sólo pueden manifestarse ante condiciones inusuales. Por ejemplo, se dice que un objeto implementa una cola como una estructura de datos de longitud fija. Un objeto que llama puede suponer que la cola se implementó como una estructura de datos infinita y no verificaría el desbordamiento de la cola, cuando se ingresa un ítem. Esta condición sólo se logra detectar durante las pruebas, al diseñar casos de prueba que fuercen el desbordamiento de la cola, y causen que el desbordamiento corrompa el comportamiento del objeto en cierta forma detectable.

Un problema posterior podría surgir derivado de interacciones entre fallas en diferentes módulos u objetos. Las fallas en un objeto sólo se detectan cuando algún otro objeto se comporta de una forma inesperada. Por ejemplo, un objeto llama a otro objeto para recibir algún servicio y supone que es correcta la respuesta; si el servicio de llamada es deficiente en algún modo, el valor devuelto puede ser válido pero equivocado. Esto no se detecta de inmediato, sino sólo se vuelve evidente cuando algún cálculo posterior sale mal.

Algunos lineamientos generales para las pruebas de interfaz son:

1. Examinar el código que se va a probar y listar explícitamente cada llamado a un componente externo. Diseñe un conjunto de pruebas donde los valores de los parámetros hacia los componentes externos estén en los extremos finales de sus rangos. Dichos valores extremos tienen más probabilidad de revelar inconsistencias de interfaz.
2. Donde los punteros pasen a través de una interfaz, pruebe siempre la interfaz con parámetros de puntero nulo.
3. Donde un componente se llame a través de una interfaz de procedimiento, diseñe pruebas que deliberadamente hagan que falle el componente. Diferir las suposiciones de falla es una de las interpretaciones de especificación equivocadas más comunes.
4. Use pruebas de esfuerzo en los sistemas que pasan mensajes. Esto significa que debe diseñar pruebas que generen muchos más mensajes de los que probablemente ocurran en la práctica. Ésta es una forma efectiva de revelar problemas de temporización.
5. Donde algunos componentes interactúen a través de memoria compartida, diseñe pruebas que varíen el orden en que se activan estos componentes. Tales pruebas pueden revelar suposiciones implícitas hechas por el programador, sobre el orden en que se producen y consumen los datos compartidos.

En ocasiones, las inspecciones y revisiones suelen ser más efectivas en costo que las pruebas para descubrir errores de interfaz. Las inspecciones pueden concentrarse en interfaces de componente e interrogantes sobre el comportamiento supuesto de la interfaz planteada durante el proceso de inspección. Un lenguaje robusto como Java permite que muchos errores de interfaz sean descubiertos por el compilador. Los analizadores estáticos (capítulo 15) son capaces de detectar un amplio rango de errores de interfaz.



Integración y pruebas incrementales

Las pruebas de sistema implican integrar diferentes componentes y, después, probar el sistema integrado que se creó. Siempre hay que usar un enfoque incremental para la integración y las pruebas (es decir, se debe incluir un componente, probar el sistema, integrar otro componente, probar de nuevo y así sucesivamente). Esto significa que, si ocurren problemas, quizá se deban a interacciones con el componente que se integró más recientemente.

La integración y las pruebas incrementales son fundamentales para los métodos ágiles como XP, donde las pruebas de regresión (véase sección 8.2) se efectúan cada vez que se integra un nuevo incremento.

http://www.SoftwareEngineering-9.com/Web/Testing/Integration.html

8.1.4 Pruebas del sistema

Las pruebas del sistema durante el desarrollo incluyen la integración de componentes para crear una versión del sistema y, luego, poner a prueba el sistema integrado. Las pruebas de sistema demuestran que los componentes son compatibles, que interactúan correctamente y que transfieren los datos correctos en el momento adecuado a través de sus interfaces. Evidentemente, se traslapan con las pruebas de componentes, pero existen dos importantes diferencias:

1. Durante las pruebas de sistema, los componentes reutilizables desarrollados por separado y los sistemas comerciales pueden integrarse con componentes desarrollados recientemente. Entonces se prueba el sistema completo.
2. Los componentes desarrollados por diferentes miembros del equipo o de grupos pueden integrarse en esta etapa. La prueba de sistema es un proceso colectivo más que individual. En algunas compañías, las pruebas del sistema implican un equipo de prueba independiente, sin la inclusión de diseñadores ni de programadores.

Cuando se integran componentes para crear un sistema, se obtiene un comportamiento emergente. Esto significa que algunos elementos de funcionalidad del sistema sólo se hacen evidentes cuando se reúnen los componentes. Éste podría ser un comportamiento emergente planeado que debe probarse. Por ejemplo, usted puede integrar un componente de autenticación con un componente que actualice información. De esta manera, tiene una característica de sistema que restringe la información actualizada de usuarios autorizados. Sin embargo, algunas veces, el comportamiento emergente no está planeado ni se desea. Hay que desarrollar pruebas que demuestren que el sistema sólo hace lo que se supone que debe hacer.

Por lo tanto, las pruebas del sistema deben enfocarse en poner a prueba las interacciones entre los componentes y los objetos que constituyen el sistema. También se prueban componentes o sistemas reutilizables para acreditar que al integrarse nuevos componentes funcionen como se esperaba. Esta prueba de interacción debe descubrir aquellos bugs de componente que sólo se revelan cuando lo usan otros componentes en el sistema. Las pruebas de interacción también ayudan a encontrar interpretaciones erróneas, cometidas por desarrolladores de componentes, acerca de otros componentes en el sistema.

Por su enfoque en las interacciones, las pruebas basadas en casos son un enfoque efectivo para la prueba del sistema. Normalmente, cada caso de uso es implementado por

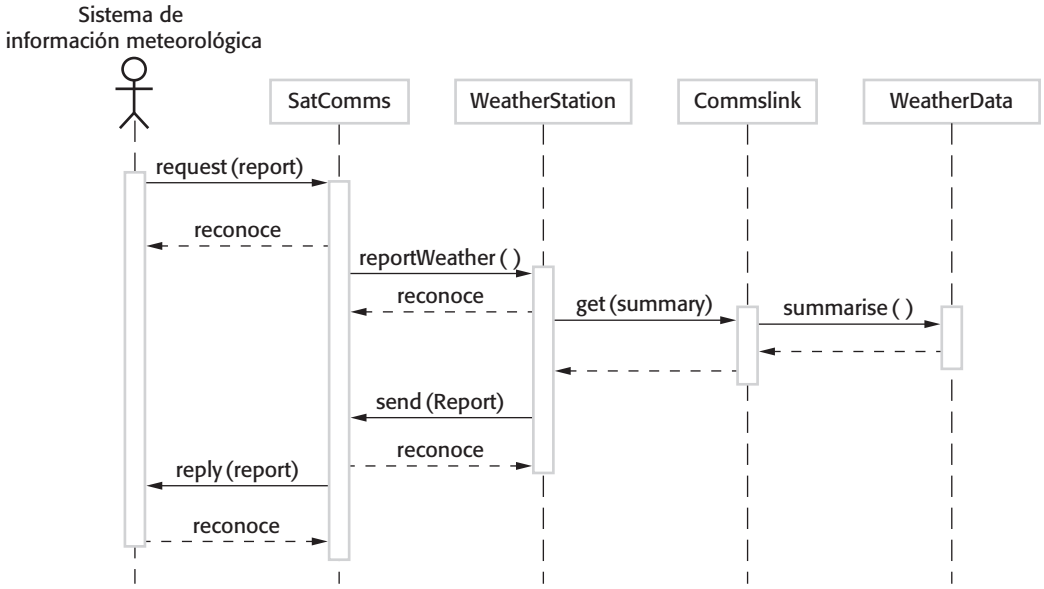


Figura 8.8 Gráfico de secuencia de recolección de datos meteorológicos

varios componentes u objetos en el sistema. Probar los casos de uso obliga a que ocurran estas interacciones. Si usted desarrolló un diagrama de secuencia para modelar la implementación de casos de uso, verá los objetos o componentes implicados en la interacción.

Para ilustrar lo anterior, se usa un ejemplo del sistema de estación meteorológica a campo abierto, donde se pide a la estación meteorológica reportar un resumen de datos a una computadora remota. El caso de uso para esto se describe en la figura 7.3 (capítulo anterior). La figura 8.8 (copia de la figura 7.7) muestra la secuencia de operaciones en la estación meteorológica, al responder a una petición de recolección de datos para el sistema de mapeo. Este diagrama sirve para identificar operaciones que se probarán y para ayudar a diseñar los casos de prueba para efectuar las pruebas. Por consiguiente, emitir una petición para un reporte dará como resultado la ejecución de la siguiente cadena de métodos:

SatComms:request → WeatherStation:reportWeather → Commslink:Get(summary) → WeatherData:summarize

El diagrama de secuencia ayuda a diseñar los casos de prueba específicos necesarios, pues muestra cuáles entradas se requieren y cuáles salidas se crean:

1. Una entrada de una petición para un reporte tiene que contar con reconocimiento asociado. En última instancia, a partir de la petición debe regresarse un reporte. Durante las pruebas, se debe crear un resumen de datos que sirva para comprobar que el reporte se organiza correctamente.
2. Una petición de entrada para un reporte a WeatherStation da como resultado la generación de un reporte resumido. Usted puede probar esto en aislamiento, creando datos brutos correspondientes al resumen que preparó para la prueba de SatComms, y demostrar que el objeto WeatherStation produce este resumen. Tales datos brutos se usan también para probar el objeto WeatherData.

Desde luego, en la figura 8.8 se simplificó el diagrama de secuencia para que no muestre excepciones. Asimismo, una prueba completa de caso/escenario de uso considera esto y garantiza que los objetos manejen adecuadamente las excepciones.

Para la mayoría de sistemas es difícil saber cuántas pruebas de sistemas son esenciales y cuándo hay que dejar de hacer pruebas. Las pruebas exhaustivas, donde se pone a prueba cada secuencia posible de ejecución del programa, son imposibles. Por lo tanto, las pruebas deben basarse en un subconjunto de probables casos de prueba. De manera ideal, para elegir este subconjunto, las compañías de software cuentan con políticas, las cuales pueden basarse en políticas de prueba generales, como una política de que todos los enunciados del programa se ejecuten al menos una vez. Como alternativa, pueden basarse en la experiencia de uso de sistema y, a la vez, enfocarse en probar las características del sistema operativo. Por ejemplo:

1. Tienen que probarse todas las funciones del sistema que se ingresen a través de un menú.
2. Debe experimentarse la combinación de funciones (por ejemplo, formateo de texto) que se ingrese por medio del mismo menú.
3. Donde se proporcione entrada del usuario, hay que probar todas las funciones, ya sea con entrada correcta o incorrecta.

Por experiencia con los principales productos de software, como procesadores de texto u hojas de cálculo, es claro que lineamientos similares se usan por lo general durante la prueba del producto. Usualmente funcionan cuando las características del software se usan en aislamiento. Los problemas se presentan, dice Whittaker (2002), cuando las combinaciones de características de uso menos común no se prueban en conjunto. Él da el ejemplo de cómo, en un procesador de texto de uso común, el uso de notas al pie de página con una plantilla en columnas múltiples causa la distribución incorrecta del texto.

Las pruebas automatizadas del sistema suelen ser más difíciles que las pruebas automatizadas de unidad o componente. Las pruebas automatizadas de unidad se apoyan en la predicción de salidas y, luego, en la codificación de dichas predicciones en un programa. En tal caso, se compara el pronóstico con el resultado. Sin embargo, el punto de aplicar un sistema puede ser generar salidas que sean grandes o no logren predecirse con facilidad. Se tiene que examinar una salida y demostrar su credibilidad sin crearla necesariamente por adelantado.

8.2 Desarrollo dirigido por pruebas

El desarrollo dirigido por pruebas (TDD, por las siglas de *Test-Driven Development*) es un enfoque al diseño de programas donde se entrelazan el desarrollo de pruebas y el de código (Beck, 2002; Jeffries y Melnik, 2007). En esencia, el código se desarrolla incrementalmente, junto con una prueba para ese incremento. No se avanza hacia el siguiente incremento sino hasta que el código diseñado pasa la prueba. El desarrollo dirigido por pruebas se introdujo como parte de los métodos ágiles como la programación extrema. No obstante, se puede usar también en los procesos de desarrollo basados en un plan.

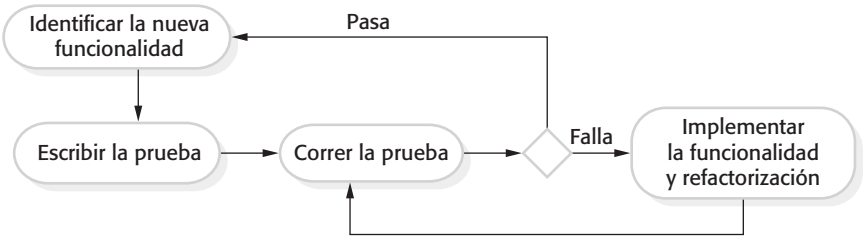


Figura 8.9 Desarrollo dirigido por pruebas

En la figura 8.9 se ilustra el proceso TDD fundamental. Los pasos en el proceso son los siguientes:

1. Se comienza por identificar el incremento de funcionalidad requerido. Éste usualmente debe ser pequeño y aplicable en pocas líneas del código.
2. Se escribe una prueba para esta funcionalidad y se implementa como una prueba automatizada. Esto significa que la prueba puede ejecutarse y reportarse, sin importar si aprueba o falla.
3. Luego se corre la prueba, junto con todas las otras pruebas que se implementaron. Inicialmente, no se aplica la funcionalidad, de modo que la nueva prueba fallará. Esto es deliberado, pues muestra que la prueba añade algo al conjunto de pruebas.
4. Luego se implementa la funcionalidad y se opera nuevamente la prueba. Esto puede incluir la refactorización del código existente, para perfeccionarlo y adicionar nuevo código a lo ya existente.
5. Una vez puestas en funcionamiento con éxito todas las pruebas, se avanza a la implementación de la siguiente funcionalidad.

Un entorno automatizado de pruebas, como el entorno JUnit que soporta pruebas del programa Java (Massol y Husted, 2003), es esencial para TDD. Conforme el código se desarrolla en incrementos muy pequeños, uno tiene la posibilidad de correr cada prueba, cada vez que se adiciona funcionalidad o se refactoriza el programa. Por consiguiente, las pruebas se incrustan en un programa independiente que corre las pruebas y apela al sistema que se prueba. Al usar este enfoque, en unos cuantos segundos se efectúan cientos de pruebas independientes.

Un argumento consistente con el desarrollo dirigido por pruebas es que ayuda a los programadores a aclarar sus ideas acerca de lo que realmente debe hacer un segmento de código. Para escribir una prueba, es preciso entender lo que se quiere, pues esta comprensión facilita la escritura del código requerido. Desde luego, si el conocimiento o la comprensión son incompletos, entonces no ayudará el desarrollo dirigido por pruebas. Por ejemplo, si su cálculo implica división, debería comprobar que no divide los números entre cero. En caso de que olvide escribir una prueba para esto, en el programa nunca se incluirá el código a comprobar.

Además de la mejor comprensión del problema, otros beneficios del desarrollo dirigido por pruebas son:

1. *Cobertura de código* En principio, cualquier segmento de código que escriba debe tener al menos una prueba asociada. Por lo tanto, puede estar seguro de que cual-

quier código en el sistema se ejecuta realmente. El código se prueba a medida que se escribe, de modo que los defectos se descubren con oportunidad en el proceso de desarrollo.

2. *Pruebas de regresión* Un conjunto de pruebas se desarrolla incrementalmente conforme se desarrolla un programa. Siempre es posible correr pruebas de regresión para demostrar que los cambios al programa no introdujeron nuevos bugs.

3. *Depuración simplificada* Cuando falla una prueba, debe ser evidente dónde yace el problema. Es preciso comprobar y modificar el código recién escrito. No se requieren herramientas de depuración para localizar el problema. Los reportes del uso del desarrollo dirigido por pruebas indican que difícilmente alguna vez se necesitará usar un depurador automatizado en el desarrollo dirigido por pruebas (Martin, 2007).

4. *Documentación del sistema* Las pruebas en sí actúan como una forma de documentación que describen lo que debe hacer el código. Leer las pruebas suele facilitar la comprensión del código.

Uno de los beneficios más importantes del desarrollo dirigido por pruebas es que reduce los costos de las pruebas de regresión. Estas últimas implican correr los conjuntos de pruebas ejecutadas exitosamente después de realizar cambios a un sistema. La prueba de regresión verifica que dichos cambios no hayan introducido nuevos bugs en el sistema, y que el nuevo código interactúa como se esperaba con el código existente. Las pruebas de regresión son muy costosas y, por lo general, poco prácticas cuando un sistema se prueba manualmente, pues son muy elevados los costos en tiempo y esfuerzo. Ante tales situaciones, usted debe ensayar y elegir las pruebas más relevantes para volver a correrlas, y es fácil perder pruebas importantes.

Sin embargo, las pruebas automatizadas, que son fundamentales para el desarrollo de primera prueba, reducen drásticamente los costos de las pruebas de regresión. Las pruebas existentes pueden volverse a correr de manera más rápida y menos costosa. Después de realizar cambios a un sistema en el desarrollo de la primera prueba, todas las pruebas existentes deben correr con éxito antes de añadir cualquier funcionalidad accesoria. Como programador, usted podría estar seguro de que la nueva funcionalidad que agregue no causará ni revelará problemas con el código existente.

El desarrollo dirigido por pruebas se usa más en el diseño de software nuevo, donde la funcionalidad se implementa en código nuevo o usa librerías estándar perfectamente probadas. Si se reutilizan grandes componentes en código o sistemas heredados, entonces se necesita escribir pruebas para dichos sistemas como un todo. El desarrollo dirigido por pruebas también puede ser ineficaz con sistemas multihilo. Los diferentes hilos pueden entrelazarse en diferentes momentos y en diversas corridas de pruebas y, por lo tanto, producirán resultados variados.

Si se usa el desarrollo dirigido por pruebas, se necesitará de un proceso de prueba del sistema para validar el sistema; esto es, comprobar que cumple con los requerimientos de todos los participantes del sistema. Las pruebas de sistema también demuestran rendimiento, confiabilidad y evidencian que el sistema no haga aquello que no debe hacer, como producir salidas indeseadas, etcétera. Andrea (2007) sugiere cómo pueden extenderse las herramientas de prueba para integrar algunos aspectos de las pruebas de sistema con TDD.

El desarrollo dirigido por pruebas resulta ser un enfoque exitoso para proyectos de dimensión pequeña y mediana. Por lo general, los programadores que adoptan dicho enfoque están contentos con él y descubren que es una forma más productiva de desarrollar

software (Jeffries y Melnik, 2007). En algunos ensayos, se demostró que conduce a mejorar la calidad del código; en otros, los resultados no son concluyentes. Sin embargo, no hay evidencia de que el TDD conduzca a un código con menor calidad.

8.3 Pruebas de versión

Las pruebas de versión son el proceso de poner a prueba una versión particular de un sistema que se pretende usar fuera del equipo de desarrollo. Por lo general, la versión del sistema es para clientes y usuarios. No obstante, en un proyecto complejo, la versión podría ser para otros equipos que desarrollan sistemas relacionados. Para productos de software, la versión sería para el gerente de producto, quien después la prepara para su venta.

Existen dos distinciones importantes entre las pruebas de versión y las pruebas del sistema durante el proceso de desarrollo:

1. Un equipo independiente que no intervino en el desarrollo del sistema debe ser el responsable de las pruebas de versión.
2. Las pruebas del sistema por parte del equipo de desarrollo deben enfocarse en el descubrimiento de bugs en el sistema (pruebas de defecto). El objetivo de las pruebas de versión es comprobar que el sistema cumpla con los requerimientos y sea suficientemente bueno para uso externo (pruebas de validación).

La principal meta del proceso de pruebas de versión es convencer al proveedor del sistema de que éste es suficientemente apto para su uso. Si es así, puede liberarse como un producto o entregarse al cliente. Por lo tanto, las pruebas de versión deben mostrar que el sistema entrega su funcionalidad, rendimiento y confiabilidad especificados, y que no falla durante el uso normal. Deben considerarse todos los requerimientos del sistema, no sólo los de los usuarios finales del sistema.

Las pruebas de versión, por lo regular, son un proceso de prueba de caja negra, donde las pruebas se derivan a partir de la especificación del sistema. El sistema se trata como una caja negra cuyo comportamiento sólo puede determinarse por el estudio de entradas y salidas relacionadas. Otro nombre para esto es "prueba funcional", llamada así porque al examinador sólo le preocupa la funcionalidad y no la aplicación del software.

8.3.1 Pruebas basadas en requerimientos

Un principio general de buena práctica en la ingeniería de requerimientos es que éstos deben ser comprobables; esto es, los requerimientos tienen que escribirse de forma que pueda diseñarse una prueba para dicho requerimiento. Luego, un examinador comprueba que el requerimiento se cumpla. En consecuencia, las pruebas basadas en requerimientos son un enfoque sistemático al diseño de casos de prueba, donde se considera cada requerimiento y se deriva un conjunto de pruebas para éste. Las pruebas basadas en requerimientos son pruebas de validación más que de defecto: se intenta demostrar que el sistema implementó adecuadamente sus requerimientos.

Por ejemplo, considere los requerimientos relacionados para el MHC-PMS (presentado en el capítulo 1), que se enfocan a la comprobación de alergias a medicamentos:

> *Si se sabe que un paciente es alérgico a algún fármaco en particular, entonces la prescripción de dicho medicamento dará como resultado un mensaje de advertencia que se emitirá al usuario del sistema.*
>
> *Si quien prescribe ignora una advertencia de alergia, deberá proporcionar una razón para ello.*

Para comprobar si estos requerimientos se cumplen, tal vez necesite elaborar muchas pruebas relacionadas:

1. Configurar un registro de un paciente sin alergias conocidas. Prescribir medicamentos para alergias que se sabe que existen. Comprobar que el sistema no emite un mensaje de advertencia.
2. Realizar un registro de un paciente con una alergia conocida. Prescribir el medicamento al que es alérgico y comprobar que el sistema emite la advertencia.
3. Elaborar un registro de un paciente donde se reporten alergias a dos o más medicamentos. Prescribir dichos medicamentos por separado y comprobar que se emite la advertencia correcta para cada medicamento.
4. Prescribir dos medicamentos a los que sea alérgico el paciente. Comprobar que se emiten correctamente dos advertencias.
5. Prescribir un medicamento que emite una advertencia y pasar por alto dicha advertencia. Comprobar que el sistema solicita al usuario proporcionar información que explique por qué pasó por alto la advertencia.

A partir de esto se puede ver que probar un requerimiento no sólo significa escribir una prueba. Por lo general, usted deberá escribir muchas pruebas para garantizar que cubrió los requerimientos. También hay que mantener el rastreo de los registros de sus pruebas basadas en requerimientos, que vinculan las pruebas con los requerimientos específicos que se ponen a prueba.

8.3.2 Pruebas de escenario

Las pruebas de escenario son un enfoque a las pruebas de versión donde se crean escenarios típicos de uso y se les utiliza en el desarrollo de casos de prueba para el sistema. Un escenario es una historia que describe una forma en que puede usarse el sistema. Los escenarios deben ser realistas, y los usuarios reales del sistema tienen que relacionarse con ellos. Si usted empleó escenarios como parte del proceso de ingeniería de requerimientos (descritos en el capítulo 4), entonces podría reutilizarlos como escenarios de prueba.

En un breve ensayo sobre las pruebas de escenario, Kaner (2003) sugiere que una prueba de escenario debe ser una historia narrativa que sea creíble y bastante compleja. Tiene que motivar a los participantes; esto es, deben relacionarse con el escenario y creer que es importante que el sistema pase la prueba. También sugiere que debe ser fácil de

Kate es enfermera con especialidad en atención a la salud mental. Una de sus responsabilidades es visitar a domicilio a los pacientes, para comprobar la efectividad de su tratamiento y que no sufran de efectos colaterales del fármaco.

En un día de visitas domésticas, Kate ingresa al MHC-PMS y lo usa para imprimir su agenda de visitas domiciliarias para ese día, junto con información resumida sobre los pacientes por visitar. Solicita que los registros para dichos pacientes se descarguen a su laptop. Se le pide la palabra clave para cifrar los registros en la laptop.

Uno de los pacientes a quienes visita es Jim, quien es tratado con medicamentos antidepresivos. Jim siente que el medicamento le ayuda, pero considera que el efecto colateral es que se mantiene despierto durante la noche. Kate observa el registro de Jim y se le pide la palabra clave para descifrar el registro. Comprueba el medicamento prescrito y consulta sus efectos colaterales. El insomnio es un efecto colateral conocido, así que anota el problema en el registro de Jim y sugiere que visite la clínica para que cambien el medicamento. Él está de acuerdo, así que Kate ingresa un recordatorio para llamarlo en cuanto ella regrese a la clínica, para concertarle una cita con un médico. Termina la consulta y el sistema vuelve a cifrar el registro de Jim.

Más tarde, al terminar sus consultas, Kate regresa a la clínica y sube los registros de los pacientes visitados a la base de datos. El sistema genera para Kate una lista de aquellos pacientes con quienes debe comunicarse, para obtener información de seguimiento y concertar citas en la clínica.

Figura 8.10 Escenario de uso para el MHC-PMS

evaluar. Si hay problemas con el sistema, entonces el equipo de pruebas de versión tiene que reconocerlos. Como ejemplo de un posible escenario para el MHC-PMS, la figura 8.10 describe una forma de utilizar el sistema en una visita domiciliaria, que pone a prueba algunas características del MHC-PMS:

1. Autenticación al ingresar al sistema.
2. Descarga y carga registros de paciente específicos desde una laptop.
3. Agenda de visitas a domicilio.
4. Cifrado y descifrado de registros de pacientes en un dispositivo móvil.
5. Recuperación y modificación de registros.
6. Vinculación con la base de datos de medicamentos que mantenga información acerca de efectos colaterales.
7. Sistema para recordatorio de llamadas.

Si usted es examinador de versión, opere a través de este escenario, interprete el papel de Kate y observe cómo se comporta el sistema en respuesta a las diferentes entradas. Como "Kate", usted puede cometer errores deliberados, como ingresar la palabra clave equivocada para decodificar registros. Esto comprueba la respuesta del sistema ante los errores. Tiene que anotar cuidadosamente cualquier problema que surja, incluidos problemas de rendimiento. Si un sistema es muy lento, esto cambiará la forma en que se usa. Por ejemplo, si se tarda mucho al cifrar un registro, entonces los usuarios que tengan poco tiempo pueden saltar esta etapa. Si pierden su laptop, una persona no autorizada podría ver entonces los registros de los pacientes.

Cuando se usa un enfoque basado en escenarios, se ponen a prueba por lo general varios requerimientos dentro del mismo escenario. Por lo tanto, además de comprobar

requerimientos individuales, también demuestra que las combinaciones de requerimientos no causan problemas.

8.3.3 Pruebas de rendimiento

Una vez integrado completamente el sistema, es posible probar propiedades emergentes, como el rendimiento y la confiabilidad. Las pruebas de rendimiento deben diseñarse para garantizar que el sistema procese su carga pretendida. Generalmente, esto implica efectuar una serie de pruebas donde se aumenta la carga, hasta que el rendimiento del sistema se vuelve inaceptable.

Como con otros tipos de pruebas, las pruebas de rendimiento se preocupan tanto por demostrar que el sistema cumple con sus requerimientos, como por descubrir problemas y defectos en el sistema. Para probar si los requerimientos de rendimiento se logran, quizá se deba construir un perfil operativo. Un perfil operativo (capítulo 15) es un conjunto de pruebas que reflejan la mezcla real de trabajo que manejará el sistema. Por consiguiente, si el 90% de las transacciones en un sistema son del tipo A, el 5% del tipo B, y el resto de los tipos C, D y E, entonces habrá que diseñar el perfil operativo de modo que la gran mayoría de pruebas sean del tipo A. De otra manera, no se obtendrá una prueba precisa del rendimiento operativo del sistema.

Desde luego, este enfoque no necesariamente es el mejor para pruebas de defecto. La experiencia demuestra que una forma efectiva de descubrir defectos es diseñar pruebas sobre los límites del sistema. En las pruebas de rendimiento, significa estresar el sistema al hacer demandas que estén fuera de los límites de diseño del software. Esto se conoce como "prueba de esfuerzo". Por ejemplo, digamos que usted prueba un sistema de procesamiento de transacciones que se diseña para procesar hasta 300 transacciones por segundo. Comienza por probar el sistema con menos de 300 transacciones por segundo. Luego aumenta gradualmente la carga del sistema más allá de 300 transacciones por segundo, hasta que está muy por arriba de la carga máxima de diseño del sistema y el sistema falla. Este tipo de pruebas tiene dos funciones:

1. Prueba el comportamiento de falla del sistema. Pueden surgir circunstancias a través de una combinación inesperada de eventos donde la carga colocada en el sistema supere la carga máxima anticipada. Ante tales circunstancias, es importante que la falla del sistema no cause corrupción de datos o pérdida inesperada de servicios al usuario. Las pruebas de esfuerzo demuestran que la sobrecarga del sistema hace que "falle poco" en vez de colapsar bajo su carga.

2. Fuerza al sistema y puede hacer que salgan a la luz defectos que no se descubrirían normalmente. Aunque se puede argumentar que esos defectos probablemente no causen fallas en el sistema en uso normal, pudiera haber una serie de combinaciones inusuales de circunstancias normales que requieren pruebas de esfuerzo.

Las pruebas de esfuerzo son particularmente relevantes para los sistemas distribuidos basados en redes de procesadores. Dichos sistemas muestran con frecuencia degradación severa cuando se cargan en exceso. La red se empantana con la coordinación de datos que deben intercambiar los diferentes procesos. Éstos se vuelven cada vez más lentos conforme esperan los datos requeridos de otros procesos. Las pruebas de esfuerzo ayudan a descubrir cuándo comienza la degradación, de manera que se puedan adicionar comprobaciones al sistema para rechazar transacciones más allá de este punto.

8.4 Pruebas de usuario

Las pruebas de usuario o del cliente son una etapa en el proceso de pruebas donde los usuarios o clientes proporcionan entrada y asesoría sobre las pruebas del sistema. Esto puede implicar probar de manera formal un sistema que se comisionó a un proveedor externo, o podría ser un proceso informal donde los usuarios experimentan con un nuevo producto de software, para ver si les gusta y si hace lo que necesitan. Las pruebas de usuario son esenciales, aun cuando se hayan realizado pruebas abarcadoras de sistema y de versión. La razón de esto es que la influencia del entorno de trabajo del usuario tiene un gran efecto sobre la fiabilidad, el rendimiento, el uso y la robustez de un sistema.

Es casi imposible que un desarrollador de sistema replique el entorno de trabajo del sistema, pues las pruebas en el entorno del desarrollador forzosamente son artificiales. Por ejemplo, un sistema que se pretenda usar en un hospital se usa en un entorno clínico donde suceden otros hechos, como emergencias de pacientes, conversaciones con familiares del paciente, etcétera. Todo ello afecta el uso de un sistema, pero los desarrolladores no pueden incluirlos en su entorno de pruebas.

En la práctica, hay tres diferentes tipos de pruebas de usuario:

1. Pruebas alfa, donde los usuarios del software trabajan con el equipo de diseño para probar el software en el sitio del desarrollador.
2. Pruebas beta, donde una versión del software se pone a disposición de los usuarios, para permitirles experimentar y descubrir problemas que encuentran con los desarrolladores del sistema.
3. Pruebas de aceptación, donde los clientes prueban un sistema para decidir si está o no listo para ser aceptado por los desarrolladores del sistema y desplegado en el entorno del cliente.

En las pruebas alfa, los usuarios y desarrolladores trabajan en conjunto para probar un sistema a medida que se desarrolla. Esto significa que los usuarios pueden identificar problemas y conflictos que no son fácilmente aparentes para el equipo de prueba de desarrollo. Los desarrolladores en realidad sólo pueden trabajar a partir de los requerimientos, pero con frecuencia esto no refleja otros factores que afectan el uso práctico del software. Por lo tanto, los usuarios brindan información sobre la práctica que ayuda con el diseño de pruebas más realistas.

Las pruebas alfa se usan a menudo cuando se desarrollan productos de software que se venden como sistemas empaquetados. Los usuarios de dichos productos quizás estén satisfechos de intervenir en el proceso de pruebas alfa porque esto les da información oportuna acerca de las características del nuevo sistema que pueden explotar. También reduce el riesgo de que cambios no anticipados al software tengan efectos perturbadores para su negocio. Sin embargo, las pruebas alfa también se utilizan cuando se desarrolla software personalizado. Los métodos ágiles, como XP, abogan por la inclusión del usuario en el proceso de desarrollo y que los usuarios tengan un papel activo en el diseño de pruebas para el sistema.

Las pruebas beta tienen lugar cuando una versión temprana de un sistema de software, en ocasiones sin terminar, se pone a disposición de clientes y usuarios para evaluación.

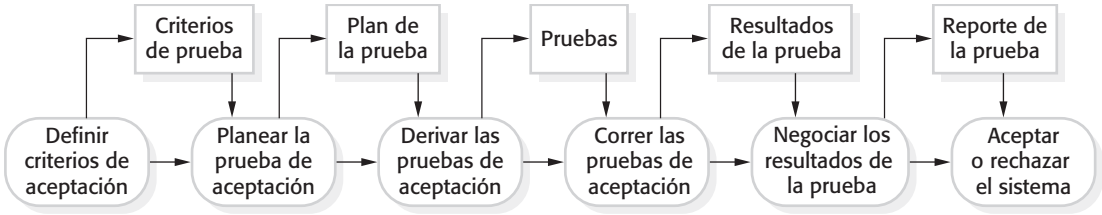


Figura 8.11 Proceso de prueba de aceptación

Los examinadores beta pueden ser un grupo selecto de clientes que sean adoptadores tempranos del sistema. De manera alternativa, el software se pone a disposición pública para uso de quienquiera que esté interesado en él. Las pruebas beta se usan sobre todo para productos de software que se emplean en entornos múltiples y diferentes (en oposición a los sistemas personalizados, que se utilizan por lo general en un entorno definido). Es imposible que los desarrolladores de producto conozcan y repliquen todos los entornos donde se usará el software. En consecuencia, las pruebas beta son esenciales para descubrir problemas de interacción entre el software y las características del entorno donde se emplea. Las pruebas beta también son una forma de comercialización: los clientes aprenden sobre su sistema y lo que puede hacer por ellos.

Las pruebas de aceptación son una parte inherente del desarrollo de sistemas personalizados. Tienen lugar después de las pruebas de versión. Implican a un cliente que prueba de manera formal un sistema, para decidir si debe o no aceptarlo del desarrollador del sistema. La aceptación implica que debe realizarse el pago por el sistema.

Existen seis etapas en el proceso de pruebas de aceptación, como se muestra en la figura 8.11. Éstas son:

1. *Definir los criterios de aceptación* Esta etapa debe, de manera ideal, anticiparse en el proceso, antes de firmar el contrato por el sistema. Los criterios de aceptación forman parte del contrato del sistema y tienen que convenirse entre el cliente y el desarrollador. Sin embargo, en la práctica suele ser difícil definir los criterios de manera tan anticipada en el proceso. Es posible que no estén disponibles requerimientos detallados y que haya cambios significativos en los requerimientos durante el proceso de desarrollo.

2. *Plan de pruebas de aceptación* Esto incluye decidir sobre los recursos, el tiempo y el presupuesto para las pruebas de aceptación, así como establecer un calendario de pruebas. El plan de pruebas de aceptación debe incluir también la cobertura requerida de los requerimientos y el orden en que se prueban las características del sistema. Tiene que definir riesgos al proceso de prueba, como caídas del sistema y rendimiento inadecuado, y resolver cómo mitigar dichos riesgos.

3. *Derivar pruebas de aceptación* Una vez establecidos los criterios de aceptación, tienen que diseñarse pruebas para comprobar si un sistema es aceptable o no. Las pruebas de aceptación deben dirigirse a probar tanto las características funcionales como las no funcionales del sistema (por ejemplo, el rendimiento). Lo ideal sería que dieran cobertura completa a los requerimientos del sistema. En la práctica, es difícil establecer criterios de aceptación completamente objetivos. Con frecuencia hay espacio para argumentar sobre si las pruebas deben mostrar o no que un criterio se cubre de manera definitiva.

4. *Correr pruebas de aceptación* Las pruebas de aceptación acordadas se ejecutan sobre el sistema. De manera ideal, esto debería ocurrir en el entorno real donde se usará el sistema, pero esto podría ser perturbador y poco práctico. En consecuencia, quizá deba establecerse un entorno de pruebas de usuario para efectuar dichas pruebas. Es difícil automatizar este proceso, ya que parte de las pruebas de aceptación podría necesitar poner a prueba las interacciones entre usuarios finales y el sistema. Es posible que se requiera cierta capacitación de los usuarios finales.

5. *Negociar los resultados de las pruebas* Es poco probable que se pasen todas las pruebas de aceptación definidas y que no haya problemas con el sistema. Si éste es el caso, entonces las pruebas de aceptación están completas y el sistema está listo para entregarse. Con mayor regularidad se descubrirán algunos problemas. En tales casos, el desarrollador y el cliente tienen que negociar para decidir si el sistema es suficientemente adecuado para ponerse en uso. También deben acordar sobre la respuesta del desarrollador para identificar problemas.

6. *Rechazo/aceptación del sistema* Esta etapa incluye una reunión entre los desarrolladores y el cliente para decidir si el sistema debe aceptarse o no. Si el sistema no es suficientemente bueno para usarse, entonces se requiere mayor desarrollo para corregir los problemas identificados. Una vez completo, se repite la fase de pruebas de aceptación.

En los métodos ágiles, como XP, las pruebas de aceptación tienen un significado un tanto diferente. En principio, comparten la noción de que son los usuarios quienes deciden si el sistema es aceptable o no. Sin embargo, en XP, el usuario forma parte del equipo de desarrollo (es decir, es un examinador alfa) y proporciona los requerimientos del sistema en términos de historias de usuario. También es responsable de definir las pruebas, que permiten determinar si el software desarrollado soporta o no la historia del usuario. Las pruebas son automatizadas y el desarrollo no avanza sino hasta que se pasan las pruebas de aceptación históricas. Por consiguiente, no hay una actividad separada de pruebas de aceptación.

Como se estudió en el capítulo 3, un problema con la participación del usuario es garantizar que quien se inserte en el equipo de desarrollo sea un usuario "típico" con conocimiento general de cómo se usará el sistema. Quizá sea difícil encontrar a tal usuario y, por lo tanto, las pruebas de aceptación en realidad tal vez no sean un verdadero reflejo de la práctica. Más aún, el requerimiento de pruebas automatizadas limita severamente la flexibilidad de los sistemas interactivos de pruebas. Para tales sistemas, las pruebas de aceptación podrían requerir que grupos de usuarios finales usen el sistema como si fuera parte de su trabajo cotidiano.

Usted puede considerar que las pruebas de aceptación son un conflicto contractual tajante. Si un sistema no pasa sus pruebas de aceptación, debe rechazarse y el pago no se realiza. Sin embargo, la realidad es más compleja. Los clientes quieren usar el software tan pronto como puedan debido a los beneficios de su despliegue inmediato. Ellos quizá compraron un nuevo hardware, capacitaron al personal y modificaron sus procesos. Tal vez están deseosos de aceptar el software, sin importar los problemas, ya que los costos por no usar el software serían mayores que los de trabajar en torno a los problemas. Por consiguiente, el resultado de las negociaciones podría ser la aceptación condicional del sistema. El cliente acepta tal sistema para comenzar el despliegue. El proveedor del sistema acuerda reparar los problemas urgentes y entregar una nueva versión al cliente tan rápido como sea posible.

PUNTOS CLAVE

- Las pruebas sólo pueden mostrar la presencia de errores en un programa. Si embargo, no pueden garantizar que no surjan fallas posteriores.
- Las pruebas de desarrollo son responsabilidad del equipo de desarrollo del software. Un equipo independiente debe responsabilizarse de probar un sistema antes de darlo a conocer a los clientes. En el proceso de pruebas de usuario, clientes o usuarios del sistema brindan datos de prueba y verifican que las pruebas sean exitosas.
- Las pruebas de desarrollo incluyen pruebas de unidad, donde se examinan objetos y métodos individuales; pruebas de componente, donde se estudian grupos de objetos relacionados; y pruebas del sistema, donde se analizan sistemas parciales o completos.
- Cuando pruebe software, debe tratar de "romperlo" mediante la experiencia y los lineamientos que elijan los tipos de casos de prueba que hayan sido efectivos para descubrir defectos en otros sistemas.
- Siempre que sea posible, se deben escribir pruebas automatizadas. Las pruebas se incrustan en un programa que puede correrse cada vez que se hace un cambio al sistema.
- El desarrollo de la primera prueba es un enfoque de desarrollo, donde las pruebas se escriben antes de que se pruebe el código. Se realizan pequeños cambios en el código, y éste se refactoriza hasta que todas las pruebas se ejecuten exitosamente.
- Las pruebas de escenario son útiles porque imitan el uso práctico del sistema. Implican trazar un escenario de uso típico y utilizarlo para derivar casos de prueba.
- Las pruebas de aceptación son un proceso de prueba de usuario, donde la meta es decidir si el software es suficientemente adecuado para desplegarse y utilizarse en su entorno operacional.

LECTURAS SUGERIDAS

"How to design practical test cases". Un artículo práctico sobre el diseño de casos de prueba elaborado por un publicista de una compañía japonesa, que tiene una muy buena reputación debido a que entrega el software con muy pocas fallas. (T. Yamaura, *IEEE Software*, **15** (6), noviembre 1998.) http://dx.doi.org/10.1109/52.730835.

How to Break Software: A Practical Guide to Testing. Se trata de un libro más práctico que teórico, sobre las pruebas de software. El autor presenta un conjunto de lineamientos basados en su experiencia relativa al diseño de pruebas, que probablemente sean efectivas en la detección de fallas del sistema. (J. A. Whittaker, Addison-Wesley, 2002.)

"Software Testing and Verification". Este número especial del *IBM Systems Journal* comprende algunos ensayos de pruebas, incluido un buen panorama. Además, incluye ensayos de métricas de prueba y automatización de pruebas. (*IBM Systems Journal*, **41** (1), enero 2002.)

"Test-driven development". Este número especial es acerca del desarrollo dirigido por pruebas, el cual incluye un buen panorama general del TDD, así como ensayos de experiencia sobre cómo se usó el TDD para diferentes tipos de software. (*IEEE Software*, **24** (3) mayo/junio 2007.)

REFERENCIAS

Andrea, J. (2007). "Envisioning the Next Generation of Functional Testing Tools". *IEEE Software*, 24 (3), 58–65.

Beck, K. (2002). *Test Driven Development: By Example*. Boston: Addison-Wesley.

Bezier, B. (1990). *Software Testing Techniques, 2nd edition*. New York: Van Nostrand Rheinhold.

Boehm, B. W. (1979). "Software engineering; R & D Trends and defense needs." In *Research Directions in Software Technology*. Wegner, P. (ed.). Cambridge, Mass.: MIT Press. 1–9.

Cusamano, M. y Selby, R. W. (1998). *Microsoft Secrets*. New York: Simon and Shuster.

Dijkstra, E. W., Dahl, O. J. y Hoare, C. A. R. (1972). *Structured Programming*. Londres: Academic Press.

Fagan, M. E. (1986). "Advances in Software Inspections". *IEEE Trans. on Software Eng.*, S**E-12** (7), 744–51.

Jeffries, R. y Melnik, G. (2007). "TDD: The Art of Fearless Programming". *IEEE Software*, **24**, 24–30.

Kaner, C. (2003). "The power of 'What If . . .' and nine ways to fuel your imagination: Cem Kaner on scenario testing". *Software Testing and Quality Engineering*, **5** (5), 16–22.

Lutz, R. R. (1993). "Analyzing Software Requirements Errors in Safety-Critical Embedded Systems". RE'93, San Diego, Calif.: IEEE.

Martin, R. C. (2007). "Professionalism and Test-Driven Development". *IEEE Software*, **24** (3), 32–6.

Massol, V. y Husted, T. (2003). *JUnit in Action*. Greenwich, Conn.: Manning Publications Co.

Prowell, S. J., Trammell, C. J., Linger, R. C. y Poore, J. H. (1999). *Cleanroom Software Engineering: Technology and Process*. Reading, Mass.: Addison-Wesley.

Whittaker, J. W. (2002). *How to Break Software: A Practical Guide to Testing*. Boston: Addison-Wesley.

PARTE

Tres

ADMINISTRACIÓN DE LA CALIDAD

En esta parte de *Ingeniería del software*, aprenderá los principios, conceptos y técnicas que se aplican para administrar y controlar la calidad del software. En los próximos capítulos se responderán preguntas como las siguientes:

- ¿Cuáles son las características generales del software de alta calidad?
- ¿Cómo se revisa la calidad y de qué manera se llevan a cabo revisiones eficaces?
- ¿En qué consiste el aseguramiento de la calidad del software?
- ¿Qué estrategias son aplicables para probar el software?
- ¿Qué métodos se utilizan para diseñar casos de prueba eficaces?
- ¿Hay métodos realistas que aseguren que el software es correcto?
- ¿Cómo pueden administrarse y controlarse los cambios que siempre ocurren cuando se elabora el software?
- ¿Qué medidas y unidades de medición se usan para evaluar la calidad de los modelos de requerimientos y diseño, código fuente y casos de prueba?

Una vez respondidas estas preguntas, el lector estará mejor preparado para asegurar que se ha producido software de alta calidad.

CAPÍTULO

14 CONCEPTOS DE CALIDAD

CONCEPTOS CLAVE

acciones de administración . . 349
calidad 339
costo de la calidad. 346
dilema de la calidad. 345
dimensiones de la calidad. . . 341
factores de la calidad. 342
punto de vista cuantitativo . 344
responsabilidad. 348
riesgos. 348
seguridad 349
suficientemente bueno. 345

El redoble de tambores para mejorar la calidad del software comenzó tan luego que éste empezó a integrarse en cada faceta de nuestras vidas. En la década de 1990, las principales corporaciones reconocieron que cada año se desperdiciaban miles de millones de dólares en software que no tenía las características ni la funcionalidad que se habían prometido. Lo que era peor, tanto el gobierno como la industria se preocupaban por la posibilidad de que alguna falla de software pudiera afectar infraestructura importante y provocara pérdidas de decenas de miles de millones de dólares. Al despuntar el nuevo siglo, *CIO Magazine* [Lev01] dio la alerta: "Dejemos de desperdiciar $78 mil millones de dólares al año", y lamentaba el hecho de que "las empresas estadounidenses gastan miles de millones de dólares en software que no hace lo que se supone que debe hacer". *InformationWeek* [Ric01] se hizo eco de la misma preocupación:

> A pesar de las buenas intenciones, el código defectuoso sigue siendo el duende de la industria del software, es responsable hasta de 45% del tiempo que están fuera los sistemas basados en computadoras y costó a las empresas estadounidenses alrededor de $100 mil millones de dólares el último año en pérdidas de productividad y reparaciones, afirma Standish Group, empresa de investigación de mercados. Eso no incluye el costo que implica perder a los clientes disgustados. Como los productores de tecnologías de información escriben aplicaciones que se basan en software empacado en infraestructura, el código defectuoso también puede inutilizar aplicaciones personalizadas...
>
> Pero, ¿cuán malo es el software defectuoso? Las respuestas varían, mas los expertos dicen que sólo se requiere de tres a cuatro defectos por cada 1 000 líneas de código para que un programa tenga mal desempeño. Hay que pensar que la mayoría de los programadores cometen un error en cada 10 líneas de código que escriben, lo que, multiplicado por los millones de líneas que hay en muchos productos comerciales, permite imaginar que la corrección de los errores cuesta a los vendedores de software al menos la mitad de sus presupuestos de desarrollo durante las pruebas. ¿Comprende lo que esto significa?

UNA MIRADA RÁPIDA

¿Qué es? La respuesta no es tan fácil como quizá se piense. La calidad se reconoce cuando se ve, por lo que puede ser algo difícil de definir. Pero para el software de computadora, la calidad es algo que debe definirse, y eso es lo que haremos en este capítulo.

¿Quién lo hace? Los involucrados en el proceso del software —ingenieros, gerentes y todos los participantes— son los responsables de la calidad.

¿Por qué es importante? Puede hacerse bien o puede repetirse. Si un equipo de software hace énfasis en la calidad de todas las actividades de la ingeniería de software, se reduce el número de repeticiones que deben hacerse. Esto da como resultado menores costos y, lo que es más importante, mejora el tiempo de llegada al mercado.

¿Cuáles son los pasos? Para lograr software de alta calidad, deben ocurrir cuatro actividades: usar procesos y prácticas probados de la ingeniería de software, administrar bien el proyecto, realizar un control de calidad exhaustivo y contar con infraestructura de aseguramiento de la calidad.

¿Cuál es el producto final? Software que satisface las necesidades del consumidor, con un desempeño apropiado y confiable, y que agrega valor para todos los que lo utilizan.

¿Cómo me aseguro de que lo hice bien? Hay que dar seguimiento a la calidad, estudiando los resultados de todas las actividades de control de calidad y midiendo ésta con el estudio de los errores antes de la entrega y de los defectos detectados en el campo.

En 2005, *ComputerWorld* [Hil05] se quejaba de que "el mal software es una plaga en casi todas las organizaciones que emplean computadoras, lo que ocasiona horas de trabajo perdidas por el tiempo que están fuera de uso las máquinas, por datos perdidos o corrompidos, oportunidades de venta perdidas, costos elevados de apoyo y mantenimiento, y poca satisfacción del cliente. Un año después, *InfoWorld* [Fos06] escribió acerca del "lamentable estado de la calidad del software" e informaba que el problema de la calidad no había mejorado.

Actualmente, la calidad del software es preocupante, pero, ¿de quién es la culpa? Los clientes culpan a los desarrolladores, pues afirman que sus prácticas descuidadas producen software de mala calidad. Los desarrolladores culpan a los clientes (y a otros participantes) con la afirmación de que las fechas de entrega irracionales y un flujo continuo de cambios los obligan a entregar software antes de haber sido validado por completo. ¿Quién tiene la razón? *Ambos*, y ése es el problema. En este capítulo se analiza el concepto de calidad del software y por qué es útil estudiarlo con seriedad siempre que se apliquen prácticas de ingeniería de software.

14.1 ¿QUÉ ES CALIDAD?

En su libro místico, *El zen y el arte del mantenimiento de la motocicleta*, Robert Persig [Per74] comenta lo siguiente acerca de lo que llamamos *calidad*:

> Calidad... sabes lo que es, pero no sabes lo que es. Pero eso es una contradicción. Algunas cosas son mejores que otras; es decir, tienen más calidad. Pero cuando tratas de decir lo que es la calidad, además de las cosas que la tienen, todo se desvanece... No hay nada de qué hablar. Pero si no puede decirse qué es Calidad, ¿cómo saber lo que es, o incluso saber que existe? Si nadie sabe lo que es, entonces, para todos los propósitos prácticos, no existe en absoluto. Pero para todos los propósitos prácticos, en realidad sí existe. ¿En qué otra cosa se basan las calificaciones? ¿Por qué paga fortunas la gente por algunos artículos y tira otros a la basura? Es obvio que algunas cosas son mejores que otras... pero, ¿en qué son mejores? Y así damos vueltas y más vueltas, ruedas de metal que patinan sin nada en lo que hagan tracción. ¿Qué demonios es la Calidad? ¿Qué es?

Es cierto: ¿qué es?

CONSEJO

¿Cuáles son las diferentes maneras en las que puede verse la calidad?

En un nivel algo pragmático, David Garvin [Gar84], de Harvard Business School, sugiere que "la calidad es un concepto complejo y de facetas múltiples" que puede describirse desde cinco diferentes puntos de vista. El *punto de vista trascendental* dice (como Persig) que la calidad es algo que se reconoce de inmediato, pero que no es posible definir explícitamente. El *punto de vista del usuario* concibe la calidad en términos de las metas específicas del usuario final. Si un producto las satisface, tiene calidad. El *punto de vista del fabricante* la define en términos de las especificaciones originales del producto. Si éste las cumple, tiene calidad. El *punto de vista del producto* sugiere que la calidad tiene que ver con las características inherentes (funciones y características) de un producto. Por último, el *punto de vista basado en el valor* la mide de acuerdo con lo que un cliente está dispuesto a pagar por un producto. En realidad, la calidad incluye todo esto y más.

Cita:

"La gente olvida cuán rápido hiciste un trabajo, pero siempre recuerda cuán bien lo realizaste."

Howard Newton

La *calidad del diseño* se refiere a las características que los diseñadores especifican para un producto. El tipo de materiales, tolerancias y especificaciones del desempeño, todo contribuye a la calidad del diseño. Si se utilizan mejores materiales, tolerancias más estrictas y se especifican mayores niveles de desempeño, la calidad del diseño de un producto se incrementa si se fabrica de acuerdo con las especificaciones.

En el desarrollo del software, la *calidad del diseño* incluye el grado en el que el diseño cumple las funciones y características especificadas en el modelo de requerimientos. La *calidad de la conformidad* se centra en el grado en el que la implementación se apega al diseño y en el que el sistema resultante cumple sus metas de requerimientos y desempeño.

Pero, ¿son la calidad del diseño y de la conformidad los únicos aspectos que deben considerar los ingenieros de software? Robert Glass [Gla98] afirma que es mejor plantear una relación más intuitiva:

satisfacción del usuario = producto que funciona + buena calidad + entrega dentro del presupuesto y plazo

En última instancia, Glass sostiene que la calidad es importante, pero que si el usuario no está satisfecho, nada de lo demás importa. DeMarco [DeM98] refuerza esta opinión al decir que "la calidad de un producto está en función de cuánto cambia al mundo para bien". Este punto de vista de la calidad afirma que si un producto de software beneficia mucho a los usuarios finales, éstos se mostrarán dispuestos a tolerar problemas ocasionales de confiabilidad o desempeño.

14.2 CALIDAD DEL SOFTWARE

Incluso los desarrolladores de software más experimentados estarán de acuerdo en que obtener software de alta calidad es una meta importante. Pero, ¿cómo se define la calidad del *software*? En el sentido más general se define[1] como: *Proceso eficaz de software que se aplica de manera que crea un producto útil que proporciona valor medible a quienes lo producen y a quienes lo utilizan*.

Hay pocas dudas acerca de que la definición anterior podría modificarse o ampliarse en un debate sin fin. Para propósitos de este libro, la misma sirve a fin de enfatizar tres puntos importantes:

1. Un proceso *eficaz de software* establece la infraestructura que da apoyo a cualquier esfuerzo de elaboración de un producto de software de alta calidad. Los aspectos de administración del proceso generan las verificaciones y equilibrios que ayudan a evitar que el proyecto caiga en el caos, contribuyente clave de la mala calidad. Las prácticas de ingeniería de software permiten al desarrollador analizar el problema y diseñar una solución sólida, ambas actividades críticas de la construcción de software de alta calidad. Por último, las actividades sombrilla, tales como administración del cambio y revisiones técnicas, tienen tanto que ver con la calidad como cualquier otra parte de la práctica de la ingeniería de software.
2. Un *producto útil* entrega contenido, funciones y características que el usuario final desea; sin embargo, de igual importancia es que entrega estos activos en forma confiable y libre de errores. Un producto útil siempre satisface los requerimientos establecidos en forma explícita por los participantes. Además, satisface el conjunto de requerimientos (por ejemplo, la facilidad de uso) con los que se espera que cuente el software de alta calidad.
3. Al *agregar valor para el productor y para el usuario* de un producto, el software de alta calidad proporciona beneficios a la organización que lo produce y a la comunidad de usuarios finales. La organización que elabora el software obtiene valor agregado porque el software de alta calidad requiere un menor esfuerzo de mantenimiento, menos errores que corregir y poca asistencia al cliente. Esto permite que los ingenieros de software dediquen más tiempo a crear nuevas aplicaciones y menos a repetir trabajos mal hechos. La comunidad de usuarios obtiene valor agregado porque la aplicación provee una capacidad útil en forma tal que agiliza algún proceso de negocios. El resultado final es 1) mayores utilidades por el producto de software, 2) más rentabilidad cuando una

1 Esta definición ha sido adaptada de [Bes04] y sustituye aquélla más orientada a la manufactura presentada en ediciones anteriores de este libro.

aplicación apoya un proceso de negocios y 3) mejor disponibilidad de información, que es crucial para el negocio.

14.2.1 Dimensiones de la calidad de Garvin

David Garvin [Gar87] sugiere que la calidad debe tomarse en cuenta, adoptando un punto de vista multidimensional que comience con la evaluación de la conformidad y termine con una visión trascendental (estética). Aunque las ocho dimensiones de Garvin de la calidad no fueron desarrolladas específicamente para el software, se aplican a la calidad de éste:

Calidad del desempeño. ¿El software entrega todo el contenido, las funciones y las características especificadas como parte del modelo de requerimientos, de manera que da valor al usuario final?

Calidad de las características. ¿El software tiene características que sorprenden y agradan la primera vez que lo emplean los usuarios finales?

Confiabilidad. ¿El software proporciona todas las características y capacidades sin fallar? ¿Está disponible cuando se necesita? ¿Entrega funcionalidad libre de errores?

Conformidad. ¿El software concuerda con los estándares locales y externos que son relevantes para la aplicación? ¿Concuerda con el diseño *de facto* y las convenciones de código? Por ejemplo, ¿la interfaz de usuario está de acuerdo con las reglas aceptadas del diseño para la selección de menú o para la entrada de datos?

Durabilidad. ¿El software puede recibir mantenimiento (cambiar) o corregirse (depurarse) sin la generación inadvertida de eventos colaterales? ¿Los cambios ocasionarán que la tasa de errores o la confiabilidad disminuyan con el tiempo?

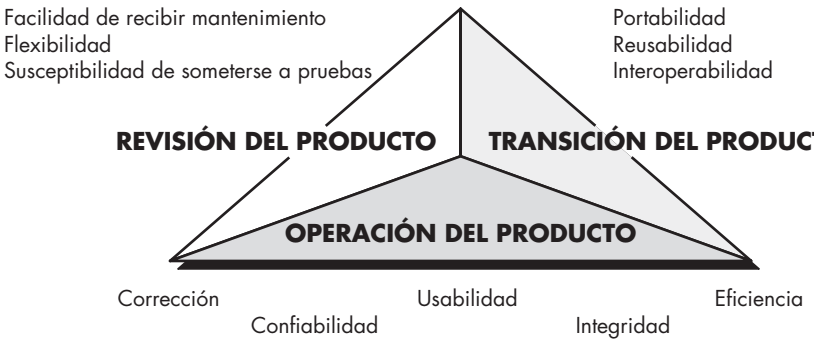
Servicio. ¿Existe la posibilidad de que el software reciba mantenimiento (cambios) o correcciones (depuración) en un periodo de tiempo aceptablemente breve? ¿El equipo de apoyo puede adquirir toda la información necesaria para hacer cambios o corregir defectos? Douglas Adams [Ada93] hace un comentario irónico que parece pertinente: "La diferencia entre algo que puede salir mal y algo que posiblemente no salga mal es que cuando esto último sale mal, por lo general es imposible corregirlo o repararlo."

Estética. No hay duda de que todos tenemos una visión diferente y muy subjetiva de lo que es estético. Aun así, la mayoría de nosotros estaría de acuerdo en que una entidad estética posee cierta elegancia, un flujo único y una "presencia" obvia que es difícil de cuantificar y que, no obstante, resulta evidente. El software estético tiene estas características.

Percepción. En ciertas situaciones, existen prejuicios que influirán en la percepción de la calidad por parte del usuario. Por ejemplo, si se introduce un producto de software elaborado por un proveedor que en el pasado ha demostrado mala calidad, se estará receloso y la percepción de la calidad del producto tendrá influencia negativa. De manera similar, si un vendedor tiene una reputación excelente se percibirá buena calidad, aun si ésta en realidad no existe.

Las dimensiones de la calidad de Garvin dan una visión "suave" de la calidad del software. Muchas de estas dimensiones (aunque no todas) sólo pueden considerarse de manera subjetiva. Por esta razón, también se necesita un conjunto de factores "duros" de la calidad que se clasifican en dos grandes grupos: 1) factores que pueden medirse en forma directa (por ejemplo, defectos no descubiertos durante las pruebas) y 2) factores que sólo pueden medirse indirectamente (como la usabilidad o la facilidad de recibir mantenimiento). En cada caso deben hacerse mediciones: debe compararse el software con algún dato para llegar a un indicador de la calidad.

FIGURA 14.1
Factores de la calidad de McCall



14.2.2 Factores de la calidad de McCall

McCall, Richards y Walters [McC77] proponen una clasificación útil de los factores que afectan la calidad del software. Éstos se ilustran en la figura 14.1 y se centran en tres aspectos importantes del producto de software: sus características operativas, su capacidad de ser modificado y su adaptabilidad a nuevos ambientes.

En relación con los factores mencionados en la figura 14.1, McCall *et al.*, hacen las descripciones siguientes:

Corrección. Grado en el que un programa satisface sus especificaciones y en el que cumple con los objetivos de la misión del cliente.

Confiabilidad. Grado en el que se espera que un programa cumpla con su función y con la precisión requerida [debe notarse que se han propuesto otras definiciones más completas de la confiabilidad (véase el capítulo 25)].

Eficiencia. Cantidad de recursos de cómputo y de código requeridos por un programa para llevar a cabo su función.

Integridad. Grado en el que es posible controlar el acceso de personas no autorizadas al software o a los datos.

Usabilidad. Esfuerzo que se requiere para aprender, operar, preparar las entradas e interpretar las salidas de un programa.

Facilidad de recibir mantenimiento. Esfuerzo requerido para detectar y corregir un error en un programa (ésta es una definición muy limitada).

Flexibilidad. Esfuerzo necesario para modificar un programa que ya opera.

Susceptibilidad de someterse a pruebas. Esfuerzo que se requiere para probar un programa a fin de garantizar que realiza la función que se pretende.

Portabilidad. Esfuerzo que se necesita para transferir el programa de un ambiente de sistema de hardware o software a otro.

Reusabilidad. Grado en el que un programa (o partes de uno) pueden volverse a utilizar en otras aplicaciones (se relaciona con el empaque y el alcance de las funciones que lleva a cabo el programa).

Interoperabilidad. Esfuerzo requerido para acoplar un sistema con otro.

Cita:

"La amargura de la mala calidad permanece mucho tiempo después de que ya se ha olvidado la dulzura de haber cumplido el plazo programado."

Karl Weigers (cita sin acreditación)

Es difícil — y, en ciertos casos, imposible— desarrollar mediciones directas[2] de estos factores de la calidad. En realidad, muchas de las unidades de medida definidas por McCall *et al.*, sólo

2 Una *medición directa* implica que hay un solo valor cuantificable que da una indicación directa del atributo en estudio. Por ejemplo, el "tamaño" de un programa se mide directamente, contando el número de sus líneas de código.

pueden obtenerse de manera indirecta. Sin embargo, la evaluación de la calidad de una aplicación por medio de estos factores dará un indicio sólido de ella.

14.2.3 Factores de la calidad ISO 9126

El estándar ISO 9126 se desarrolló con la intención de identificar los atributos clave del software de cómputo. Este sistema identifica seis atributos clave de la calidad:

Funcionalidad. Grado en el que el software satisface las necesidades planteadas según las establecen los atributos siguientes: adaptabilidad, exactitud, interoperabilidad, cumplimiento y seguridad.

Confiabilidad. Cantidad de tiempo que el software se encuentra disponible para su uso, según lo indican los siguientes atributos: madurez, tolerancia a fallas y recuperación.

Usabilidad. Grado en el que el software es fácil de usar, según lo indican los siguientes subatributos: entendible, aprendible y operable.

Eficiencia. Grado en el que el software emplea óptimamente los recursos del sistema, según lo indican los subatributos siguientes: comportamiento del tiempo y de los recursos.

Facilidad de recibir mantenimiento. Facilidad con la que pueden efectuarse reparaciones al software, según lo indican los atributos que siguen: analizable, cambiable, estable, susceptible de someterse a pruebas.

Portabilidad. Facilidad con la que el software puede llevarse de un ambiente a otro según lo indican los siguientes atributos: adaptable, instalable, conformidad y sustituible.

CONSEJO

Aunque resulta tentador desarrollar mediciones cuantitativas para los factores de calidad mencionados aquí, también puede crearse una lista de comprobación de atributos que den una indicación sólida de la presencia del factor.

Igual que otros factores de la calidad del software estudiados en las subsecciones anteriores, los factores ISO 9126 no necesariamente conducen a una medición directa. Sin embargo, proporcionan una base útil para hacer mediciones indirectas y una lista de comprobación excelente para evaluar la calidad del sistema.

14.2.4 Factores de calidad que se persiguen

Las dimensiones y factores de la calidad presentados en las secciones 14.2.1 y 14.2.2 se centran en el software como un todo y pueden utilizarse como indicación general de la calidad de una aplicación. Un equipo de software puede desarrollar un conjunto de características de la calidad y las preguntas asociadas correspondientes que demuestren[3] el grado en el que se satisface cada factor. Por ejemplo, McCall identifica la *usabilidad* como un factor importante de la calidad. Si se pidiera revisar una interfaz de usuario para evaluar su usabilidad, ¿cómo se haría? Se comenzaría con los subatributos propuestos por McCall —entendible, aprendible y operable— pero en un sentido práctico: ¿qué significan éstos?

Para hacer la evaluación, se necesita determinar atributos específicos y medibles (o al menos reconocibles) de la interfaz. Por ejemplo [Bro03]:

Intuitiva. Grado en el que la interfaz sigue patrones esperados de uso, de modo que hasta un novato la pueda utilizar sin mucha capacitación.

- ¿La interfaz lleva hacia una comprensión fácil?
- ¿Todas las operaciones son fáciles de localizar e iniciar?
- ¿La interfaz usa una metáfora reconocible?
- ¿La entrada está especificada de modo que economiza el uso del teclado o del ratón?

Cita:

"Cualquier actividad se vuelve creativa cuando a quien la realiza le importa hacerla bien, o mejor."

John Updike

3 Estas características y preguntas se plantearían como parte de la revisión del software (véase el capítulo 15).

- ¿La entrada sigue las tres reglas de oro? (véase el capítulo 11)
- ¿La estética ayuda a la comprensión y uso?

Eficiencia. Grado en el que es posible localizar o iniciar las operaciones y la información.

- ¿La distribución y estilo de la interfaz permite que un usuario introduzca con eficiencia las operaciones y la información?
- ¿Una secuencia de operaciones (o entrada de datos) puede realizarse con economía de movimientos?
- ¿Los datos de salida o el contenido están presentados de modo que se entienden de inmediato?
- ¿Las operaciones jerárquicas están organizadas de manera que minimizan la profundidad con la que debe navegar el usuario para hacer que alguna se ejecute?

Robustez. Grado en el que el software maneja entradas erróneas de datos o en el que se presenta interacción inapropiada por parte del usuario.

- ¿El software reconocerá el error si entran datos en el límite de lo permitido o más allá y, lo que es más importante, continuará operando sin fallar ni degradarse?
- ¿La interfaz reconocerá los errores cognitivos o de manipulación y guiará en forma explícita al usuario de vuelta al camino correcto?
- ¿La interfaz da un diagnóstico y guía útiles cuando se descubre una condición de error (asociada con la funcionalidad del software)?

Riqueza. Grado en el que la interfaz provee un conjunto abundante de características.

- ¿Puede personalizarse la interfaz según las necesidades específicas del usuario?
- ¿La interfaz tiene gran capacidad para permitir al usuario identificar una secuencia de operaciones comunes con una sola acción o comando?

A medida que se desarrolla el diseño de la interfaz, el equipo del software revisa el prototipo del diseño y plantea las preguntas anteriores. Si la respuesta a la mayor parte de éstas es "sí", es probable que la interfaz de usuario sea de buena calidad. Para cada factor de la calidad que se desee evaluar se desarrollan preguntas similares.

14.2.5 Transición a un punto de vista cuantitativo

En las subsecciones anteriores se presentaron varios factores cualitativos para la "medición" de la calidad del software. La comunidad de la ingeniería de software trata de obtener mediciones precisas de la calidad de éste y a veces se ve frustrada por la naturaleza subjetiva de la actividad. Cavano y McCall [Cav78] analizan esta situación:

> La determinación de la calidad es un factor clave en los eventos cotidianos: concursos para catar vinos, eventos deportivos [como gimnasia], competencias de talento, etc. En estas situaciones se juzga la calidad del modo más fundamental y directo: la comparación directa de objetos en condiciones idénticas y con conceptos predeterminados. El vino se juzga de acuerdo con su claridad, color, buqué, sabor, etc. Sin embargo, este tipo de juicio es muy subjetivo; para que tenga algún valor, debe ser hecho por un experto.
>
> La subjetividad y la especialización también se aplican a la determinación de la calidad del software. Para ayudar a resolver este problema, es necesario tener una definición más precisa de la calidad del software, así como una forma de realizar mediciones cuantitativas de la calidad a fin de hacer análisis objetivos... Como no existe algo parecido al conocimiento absoluto, no debe esperarse medir con toda exactitud la calidad del software, porque toda medición es imperfecta. Jacob Bronkowski

> describió esta paradoja del conocimiento del modo siguiente: "Año con año desarrollamos instrumentos más precisos para observar la naturaleza con más nitidez. Y cuando vemos las observaciones, nos decepcionamos porque son borrosas y sentimos que son tan inciertas como siempre".

En el capítulo 23 se presenta un conjunto de unidades de medida aplicables a la evaluación cuantitativa de la calidad del software. En todos los casos, las unidades representan mediciones indirectas, es decir, nunca miden realmente la *calidad*, sino alguna manifestación de ella. El factor que complica todo es la relación precisa entre la variable que se mide y la calidad del software.

14.3 El dilema de la calidad del software



Cuando se enfrente al dilema de la calidad (y todos lo hacen en un momento u otro), trate de alcanzar el balance: suficiente esfuerzo para producir una calidad aceptable sin que sepulte al proyecto.

En una entrevista [Ven03] publicada en la web, Bertrand Meyer analiza lo que se denomina el *dilema de la calidad:*

> Si produce un sistema de software de mala calidad, usted pierde porque nadie lo querrá comprar. Por otro lado, si dedica un tiempo infinito, demasiado esfuerzo y enormes sumas de dinero para obtener un elemento perfecto de software, entonces tomará tanto tiempo terminarlo y será tan caro de producir que de todos modos quedará fuera del negocio. En cualquier caso, habrá perdido la ventana de mercado, o simplemente habrá agotado sus recursos. De modo que las personas de la industria tratan de situarse en ese punto medio mágico donde el producto es suficientemente bueno para no ser rechazado de inmediato, no en la evaluación, pero tampoco es un objeto perfeccionista ni con demasiado trabajo que lo convierta en algo que requiera demasiado tiempo o dinero para ser terminado.

Es correcto afirmar que los ingenieros de software deben tratar de producir sistemas de alta calidad. Es mejor aplicar buenas prácticas al intento de lograrlo. Pero la situación descrita por Meyer proviene de la vida real y representa un dilema incluso para las mejores organizaciones de ingeniería de software.

14.3.1 Software "suficientemente bueno"

En palabras sencillas, si damos por válido el argumento de Meyer, ¿es aceptable producir software "suficientemente bueno"? La respuesta a esta pregunta debe ser "sí", porque las principales compañías de software lo hacen a diario. Crean software con errores detectados y lo distribuyen a una gran población de usuarios finales. Reconocen que algunas de las funciones y características de la versión 1.0 tal vez no sean de la calidad más alta y planean hacer mejoras en la versión 2.0. Hacen esto, sabiendo que algunos clientes se quejarán; reconocen que el tiempo para llegar al mercado actúa contra la mejor calidad, y liberan el software, siempre y cuando el producto entregado sea "suficientemente bueno".

Exactamente, ¿qué significa "suficientemente bueno"? El software suficientemente bueno contiene las funciones y características de alta calidad que desean los usuarios, pero al mismo tiempo tiene otras más oscuras y especializadas que contienen errores conocidos. El vendedor de software espera que la gran mayoría de usuarios finales perdone los errores gracias a que estén muy contentos con la funcionalidad de la aplicación.

Esta idea resulta familiar para muchos lectores. Si usted es uno de ellos, le pido que considere algunos de los argumentos contra lo "suficientemente bueno".

Es verdad que lo "suficientemente bueno" puede funcionar en ciertos dominios de aplicación y para unas cuantas compañías grandes de software. Después de todo, si una empresa tiene un presupuesto enorme para mercadotecnia y convence a suficientes personas de que compren la versión 1.0, habrá tenido éxito en capturarlos. Como ya se dijo, puede sostener que en las versiones posteriores mejorará la calidad. Al entregar la versión 1.0 suficientemente buena, habrá capturado al mercado.

Si el lector trabaja para una compañía pequeña, debe tener cuidado con esta filosofía. Al entregar un producto suficientemente bueno (defectuoso), corre el riesgo de causar un daño permanente a la reputación de su compañía. Tal vez nunca tenga la oportunidad de entregar una versión 2.0 porque los malos comentarios quizá ocasionen que las ventas se desplomen y que la empresa desaparezca.

Si trabaja en ciertos dominios de aplicación (por ejemplo, software incrustado en tiempo real) o si construye software de aplicación integrado con hardware (como el software automotriz o de telecomunicaciones), entregar software con errores conocidos es una negligencia y deja expuesta a su compañía a litigios costosos. En ciertos casos, incluso, puede ser un delito. ¡Nadie quiere tener software suficientemente bueno en los aviones!

Así que proceda con cautela si piensa que lo "suficientemente bueno" es un atajo que puede resolver los problemas de calidad de su software. Tal vez funcione, pero sólo para unos cuantos y en un conjunto limitado de dominios de aplicación.[4]

14.3.2 El costo de la calidad

El argumento es algo parecido a esto: *sabemos que la calidad es importante, pero cuesta tiempo y dinero —demasiado tiempo y dinero— lograr el nivel de calidad en el software que en realidad queremos.* Visto así, este argumento parece razonable (véanse los comentarios anteriores de Meyer en esta sección). No hay duda de que la calidad tiene un costo, pero la mala calidad también lo tiene —no sólo para los usuarios finales que deban vivir con el software defectuoso, sino también para la organización del software que lo elaboró y que debe darle mantenimiento—. La pregunta real es ésta: *¿por cuál costo debemos preocuparnos?* Para responder a esta pregunta debe entenderse tanto el costo de tener calidad como el del software de mala calidad.

El *costo de la calidad* incluye todos los costos en los que se incurre al buscar la calidad o al realizar actividades relacionadas con ella y los costos posteriores de la falta de calidad. Para entender estos costos, una organización debe contar con unidades de medición que provean el fundamento del costo actual de la calidad, que identifiquen las oportunidades para reducir dichos costos y que den una base normalizada de comparación. El costo de la calidad puede dividirse en los costos que están asociados con la prevención, la evaluación y la falla.



No tema incurrir en costos significativos por la prevención. Esté seguro de que su inversión tendrá un rendimiento excelente.

Los *costos de prevención* incluyen lo siguiente: 1) el costo de las actividades de administración requeridas para planear y coordinar todas las actividades de control y aseguramiento de la calidad, 2) el costo de las actividades técnicas agregadas para desarrollar modelos completos de los requerimientos y del diseño, 3) los costos de planear las pruebas y 4) el costo de toda la capacitación asociada con estas actividades.

Los *costos de evaluación* incluyen las actividades de investigación de la condición del producto la "primera vez" que pasa por cada proceso. Algunos ejemplos de costos de evaluación incluyen los siguientes:

- El costo de efectuar revisiones técnicas (véase el capítulo 15) de los productos del trabajo de la ingeniería de software.
- El costo de recabar datos y unidades de medida para la evaluación (véase el capítulo 23)
- El costo de hacer las pruebas y depurar (véanse los capítulos 18 a 21)

Los *costos de falla* son aquellos que se eliminarían si no hubiera errores antes o después de enviar el producto a los consumidores. Los costos de falla se subdividen en internos y externos. Se incurre en *costos internos de falla* cuando se detecta un error en un producto antes del envío. Los costos internos de falla incluyen los siguientes:

4 Un análisis útil de los pros y contras del software "suficientemente bueno" se encuentra en [Bre02].

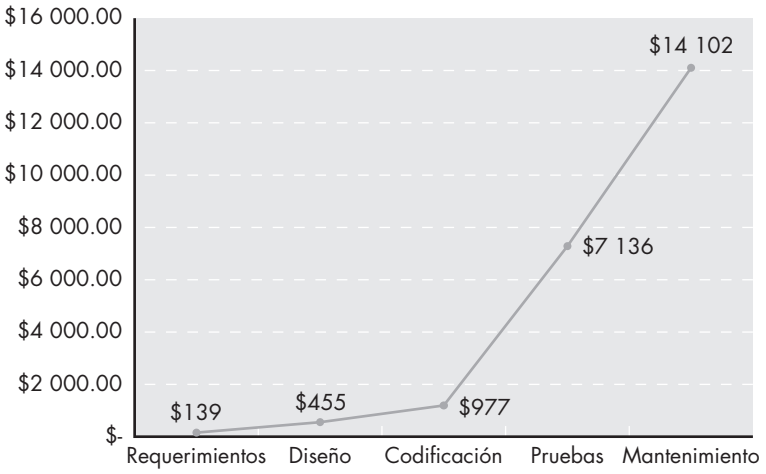


FIGURA 14.2
Costo relativo de corregir errores y defectos (cifras en dólares estadounidenses)
Fuente: Adaptado de [Boe01b].

- El costo requerido por efectuar repeticiones (reparaciones para corregir un error).
- El costo en el que se incurre cuando una repetición genera inadvertidamente efectos colaterales que deban mitigarse.
- Los costos asociados con la colección de las unidades de medida de la calidad que permitan que una organización evalúe los modos de la falla.

Cita:

"Toma menos tiempo hacer algo bien que explicar por qué se hizo mal."

H. W. Longfellow

Los *costos externos de falla* se asocian con defectos encontrados después de que el producto se envió a los consumidores. Algunos ejemplos de costos externos de falla son los de solución de quejas, devolución y sustitución del producto, ayuda en línea y trabajo asociado con la garantía. La mala reputación y la pérdida resultante de negocios es otro costo externo de falla que resulta difícil de cuantificar y que, sin embargo, es real. Cuando se produce software de mala calidad, suceden cosas malas.

En lo que constituye una acusación contra los desarrolladores de software que se rehúsan a considerar los costos de falla externos, Cem Kaner [Kan95] afirma lo siguiente:

> Muchos de los costos de falla externos, tales como los fondos de comercialización, son difíciles de cuantificar, por lo que muchas compañías los ignoran cuando calculan sus relaciones costo-beneficio. Otros costos externos de falla pueden reducirse (al dar un apoyo barato debido a la mala calidad después de hacer la venta, o al cobrar el apoyo a los consumidores) sin que se incremente la satisfacción del cliente. Al ignorar los costos que los malos productos generan a nuestros compradores, los ingenieros de la calidad estimulan una toma de decisiones que los hace víctimas en lugar de satisfacerlos.

Como es de esperar, los costos relacionados con la detección y la corrección de errores o defectos se incrementan en forma abrupta cuando se pasa de la prevención a la detección, a la falla interna y a la externa. La figura 14.2, basada en datos obtenidos por Boehm y Basili [Boe01b] y elaborada por Cigital, Inc. [Cig07], ilustra este fenómeno.

El costo promedio de la industria por corregir un defecto durante la generación de código es aproximadamente de US$977 por error. El promedio del costo en el que incurre la industria por corregir el mismo error si se descubre durante las pruebas del sistema es de US$7 136. Cigital, Inc. [Cig07] tome en cuenta que una aplicación grande contiene 200 errores introducidos durante la codificación.

> De acuerdo con datos promedio, el costo de encontrar y corregir defectos durante la fase de codificación es de US$977 por defecto. Entonces, el costo total por corregir los 200 errores "críticos" durante esta fase es de (200 × US$977) US$195 400, aproximadamente.

> Los datos promedio de la industria indican que el costo de encontrar y corregir defectos durante la fase de pruebas del sistema es de US$7 136 por cada uno. En este caso, si se supone que en dicha fase se descubren aproximadamente 50 defectos críticos (tan sólo 25% de los descubiertos por Cigital en la fase de codificación), el costo de encontrarlos y corregirlos (50 × US$7 136) sería aproximadamente de US$356 800. Esto también habría resultado en 150 errores críticos no detectados ni corregidos. El costo de encontrar y corregir estos 150 defectos en la fase de mantenimiento (150 × US$14 102) habría sido de US$2 115 300. Entonces, el costo total de encontrar y corregir los 200 defectos (US$2 115 300 + US$356 800) después de la fase de codificación habría sido de US$2 472 100.

Aun si la organización de software tuviera costos que fueran la mitad del promedio de la industria (la mayor parte de compañías no tiene idea de cuáles son sus costos), los ahorros asociados con el control de calidad temprano y las actividades para su aseguramiento (efectuadas durante el análisis de los requerimientos y el diseño) serían notables.

14.3.3 Riesgos

En el capítulo 1 de este libro se dijo que "la gente basa su trabajo, confort, seguridad, entretenimiento, decisiones y su propia vida, en software de cómputo. Más vale que esté bien hecho". La implicación es que el software de mala calidad aumenta los riesgos tanto para el desarrollador como para el usuario final. En la subsección anterior se analizó uno de dichos riesgos (el costo). Pero lo perjudicial de las aplicaciones mal diseñadas e implementadas no siempre se mide en dólares y tiempo. Un ejemplo extremo [Gag04] servirá para ilustrar esto.

En el mes de noviembre de 2000, en un hospital de Panamá, 28 pacientes recibieron dosis masivas de rayos gama durante su tratamiento contra diversos tipos de cáncer. En los meses que siguieron, 5 de estos pacientes murieron por envenenamiento radiactivo y 15 más sufrieron complicaciones serias. ¿Qué fue lo que ocasionó esta tragedia? Un paquete de software, desarrollado por una compañía estadounidense, que fue modificado por técnicos del hospital para calcular las dosis de radiación para cada paciente.

Los tres médicos panameños que "pellizcaron" el software para que diera capacidad adicional fueron acusados de asesinato en segundo grado. La empresa de Estados Unidos enfrentó litigios serios en los dos países. Gage y McCormick comentan lo siguiente:

> Éste no es un relato para prevenir a los médicos, aun cuando luchen por estar fuera de la cárcel si no entienden o hacen mal uso de la tecnología. Tampoco es la narración de cómo pueden salir heridos, o algo peor, los seres humanos a causa del software mal diseñado o poco explicado, aunque hay muchos ejemplos al respecto. Ésta es la alerta para cualquier creador de programas de cómputo: la calidad del software importa, las aplicaciones deben ser a prueba de tontos y el código mal desplegado —ya sea incrustado en el motor de un automóvil, un brazo robótico o un dispositivo de curación en un hospital— puede matar.

La mala calidad conlleva riesgos, algunos muy serios.

14.3.4 Negligencia y responsabilidad

La historia es muy común. Una entidad gubernamental o corporativa contrata a una compañía importante de desarrollo de software o a una consultoría para que analice los requerimientos y luego diseñe y construya un "sistema" basado en software para apoyar alguna actividad de importancia. El sistema debe auxiliar a una función corporativa principal (como la administración de pensiones) o a alguna función gubernamental (por ejemplo, la administración del cuidado de la salud o los créditos hipotecarios).

El trabajo comienza con las mejores intenciones por ambas partes, pero en el momento en el que el sistema se entrega, las cosas han marchado mal. El sistema va retrasado, no da los resultados y funciones deseadas, comete errores y no cuenta con la aprobación del cliente. Comienzan los litigios.

En la mayor parte de los casos, el cliente afirma que el desarrollador ha sido negligente (en cuanto a la manera en la que aplicó las prácticas del software), por lo que no merece el pago. Es frecuente que el desarrollador diga que el cliente ha cambiado repetidamente sus requerimientos y trastornado de diversas maneras los acuerdos para el trabajo. En cualquier caso, es la calidad del sistema lo que está en entredicho.

14.3.5 Calidad y seguridad

A medida que aumenta la importancia crítica de los sistemas y aplicaciones basados en web, la seguridad de las aplicaciones se ha vuelto más importante. En pocas palabras, el software que no tiene alta calidad es fácil de penetrar por parte de intrusos y, en consecuencia, el software de mala calidad aumenta indirectamente el riesgo de la seguridad, con todos los costos y problemas que eso conlleva.

En una entrevista para *ComputerWorld*, el autor y experto en seguridad Gary McGraw comenta lo siguiente [Wil05]:

> La seguridad del software se relaciona por completo con la calidad. Debe pensarse en seguridad, confiabilidad, disponibilidad y dependencia, en la fase inicial, en la de diseño, en la de arquitectura, pruebas y codificación, durante todo el ciclo de vida del software [proceso]. Incluso las personas conscientes del problema de la seguridad del software se centran en las etapas finales del ciclo de vida. Entre más pronto se detecte un problema en el software, mejor. Y hay dos clases de problemas. Uno son los errores, que son problemas de implementación. El otro son las fallas del software: problemas de arquitectura en el diseño. La gente presta demasiada atención a los errores pero no la suficiente a las fallas.

Para construir un sistema seguro hay que centrarse en la calidad, y eso debe comenzar durante el diseño. Los conceptos y métodos analizados en la parte 2 del libro llevan a una arquitectura del software que reduce las "fallas". Al eliminar las fallas de arquitectura (con lo que mejora la calidad del software) será más difícil que intrusos penetren en el software.

14.3.6 El efecto de las acciones de la administración

Es frecuente que la calidad del software reciba influencia tanto de las decisiones administrativas como de las tecnológicas. Incluso las mejores prácticas de la ingeniería de software pueden ser arruinadas por malas decisiones gerenciales y por acciones cuestionables de la administración del proyecto.

En la parte 4 de este libro se analiza la administración del proyecto en el contexto del proceso del software. Al iniciar toda tarea del proyecto, el líder de éste tomará decisiones que tienen un efecto significativo en la calidad del producto.

Decisiones de estimación. Como se dice en el capítulo 26, un equipo de software rara vez puede darse el lujo de dar una estimación para el proyecto *antes* de que se hayan establecido las fechas de entrega y especificado un presupuesto general. En vez de ello, el equipo realiza un "filtro sanitario" para garantizar que las fechas de entrega y puntos de revisión son racionales. En muchos casos, hay una presión enorme del tiempo para entrar al mercado que fuerza al equipo a aceptar fechas de entrega irreales. En consecuencia, se toman atajos, se pasan por alto las actividades que elevan la calidad del software y disminuye la calidad del producto. Si una fecha de entrega es irracional, es importante poner los pies sobre la tierra. Explique por qué se necesita más tiempo o, alternativamente, sugiera un subconjunto de funciones que puedan entregarse (sin demasiada calidad) en el tiempo programado.

Decisiones de programación. Cuando se establece un programa de desarrollo de un proyecto de software (véase el capítulo 27), se establece la secuencia de las tareas con base en dependencias. Por ejemplo, como el componente **A** depende del procesamiento que ocurra

dentro de los componentes **B**, **C** y **D**, el componente **A** no puede programarse para ser probado hasta que los componentes **B**, **C** y **D** no hayan sido probados por completo. La programación del proyecto reflejaría esto. Pero si el tiempo es demasiado escaso y debe disponerse de **A** para realizar pruebas de importancia crítica, puede decidirse a probar **A** sin sus componentes subordinados (que están un poco retrasados) a fin de que esté disponible para otras pruebas que se realicen antes de la entrega. Después de todo, el plazo final se acerca. En consecuencia, **A** podría tener defectos ocultos que sólo se descubrirían mucho tiempo después. La calidad bajaría.

Decisiones orientadas al riesgo. La administración del riesgo (véase el capítulo 28) es uno de los atributos clave de un proyecto exitoso de software. En realidad se necesita saber lo que puede salir mal y establecer un plan de contingencia para ese caso. Demasiados equipos de software prefieren un optimismo ciego y establecen un programa de desarrollo con la suposición de que nada saldrá mal. Lo que es peor, no tienen manera de manejar las cosas que salgan mal. En consecuencia, cuando un riesgo se convierte en realidad, reina el caos y aumenta el grado de locuras que se cometen, con lo que invariablemente la calidad se desploma.

El dilema de la calidad del software se resume mejor con el enunciado de la Ley de Meskimen: *Nunca hay tiempo para hacerlo bien, pero siempre hay tiempo para hacerlo otra vez.* Mi consejo es: tomarse el tiempo para hacerlo bien casi nunca es la decisión equivocada.

14.4 LOGRAR LA CALIDAD DEL SOFTWARE

La calidad del software no sólo se ve. Es el resultado de la buena administración del proyecto y de una correcta práctica de la ingeniería de software. La administración y práctica se aplican en el contexto de cuatro actividades principales que ayudan al equipo de software a lograr una alta calidad en éste: métodos de la ingeniería de software, técnicas de administración de proyectos, acciones de control de calidad y aseguramiento de la calidad del software.

14.4.1 Métodos de la ingeniería de software

? ¿Qué necesito hacer para influir en la calidad de manera positiva?

Si espera construir software de alta calidad, debe entender el problema que se quiere resolver. También debe ser capaz de crear un diseño que esté de acuerdo con el problema y que al mismo tiempo tenga características que lleven al software a las dimensiones y factores de calidad que se estudiaron en la sección 14.2.

En la parte 2 de este libro se presentó una amplia variedad de conceptos y métodos que conducen a una comprensión razonablemente completa del problema y al diseño exhaustivo que establece un fundamento sólido para la actividad de construcción. Si el lector aplica estos conceptos y adopta métodos apropiados de análisis y diseño, se eleva sustancialmente la probabilidad de crear software de alta calidad.

14.4.2 Técnicas de administración de proyectos

El efecto de las malas decisiones de administración sobre la calidad del software se estudió en la sección 14.3.6. Las implicaciones son claras: si 1) un gerente de proyecto usa estimaciones para verificar que las fechas pueden cumplirse, 2) se comprenden las dependencias de las actividades programadas y el equipo resiste la tentación de usar atajos, 3) la planeación del riesgo se lleva a cabo de manera que los problemas no alienten el caos, entonces la calidad del software se verá influida de manera positiva.

Además, el plan del proyecto debe incluir técnicas explícitas para la administración de la calidad y el cambio. Las técnicas que llevan a buenas prácticas de administración de proyectos se estudian en la parte 4 de este libro.

14.4.3 Control de calidad

? ¿Qué es el control de calidad del software?

El control de calidad incluye un conjunto de acciones de ingeniería de software que ayudan a asegurar que todo producto del trabajo cumpla sus metas de calidad. Los modelos se revisan para garantizar que están completos y que son consistentes. El código se inspecciona con objeto de descubrir y corregir errores antes de que comiencen las pruebas. Se aplica una serie de etapas de prueba para detectar los errores en procesamiento lógico, manipulación de datos y comunicación con la interfaz. La combinación de mediciones con retroalimentación permite que el equipo del software sintonice el proceso cuando cualquiera de estos productos del trabajo falla en el cumplimiento de las metas de calidad. Las actividades de control de calidad se estudian en detalle en lo que resta de la parte 3 de este libro.

14.4.4 Aseguramiento de la calidad

WebRef

Pueden encontrarse vínculos útiles acerca de técnicas de aseguramiento de la calidad en la dirección **www.niwotridge.com/Resources/PM-SWEResources/SoftwareQualityAssurance.htm**

El aseguramiento de la calidad establece la infraestructura de apoyo a los métodos sólidos de la ingeniería de software, la administración racional de proyectos y las acciones de control de calidad, todo de importancia crucial si se trata de elaborar software de alta calidad. Además, el aseguramiento de la calidad consiste en un conjunto de funciones de auditoría y reportes para evaluar la eficacia y completitud de las acciones de control de calidad. La meta del aseguramiento de la calidad es proveer al equipo administrativo y técnico los datos necesarios para mantenerlo informado sobre la calidad del producto, con lo que obtiene perspectiva y confianza en que las acciones necesarias para lograr la calidad del producto funcionan. Por supuesto, si los datos provistos a través del aseguramiento de la calidad identifican los problemas, es responsabilidad de la administración enfrentarlos y aplicar los recursos necesarios para resolver los correspondientes a la calidad. En el capítulo 16 se estudia en detalle el aseguramiento de la calidad del software.

14.5 Resumen

La preocupación por la calidad de los sistemas basados en software ha aumentado a medida que éste se integra en cada aspecto de nuestras vidas cotidianas. Pero es difícil hacer la descripción exhaustiva de la calidad del software. En este capítulo se define la calidad como un proceso eficaz del software aplicado de modo que crea un producto útil que da un valor medible a quienes lo generan y a quienes lo utilizan.

Con el tiempo se han propuesto varias dimensiones y factores de calidad del software. Todos ellos tratan de definir un conjunto de características que, si se logran, llevarán a un software de alta calidad. McCall y los factores de calidad de la norma ISO 9126 establecen características tales como confiabilidad, usabilidad, facilidad de dar mantenimiento, funcionalidad y portabilidad, como indicadores de la existencia de calidad.

Toda organización de software se enfrenta al dilema de la calidad del software. En esencia, todos quieren elaborar sistemas de alta calidad, pero en un mundo dirigido por el mercado, sencillamente no se dispone del tiempo y el esfuerzo requeridos para producir software "perfecto". La cuestión es la siguiente: ¿debe elaborarse software que sea "suficientemente bueno"? Aunque muchas compañías hacen eso, hay una desventaja notable que debe tomarse en cuenta.

Sin importar el enfoque que se elija, la calidad tiene un costo que puede estudiarse en términos de prevención, evaluación y falla. Los costos de prevención incluyen todas las acciones de la ingeniería de software diseñadas para prevenir los defectos. Los costos de evaluación están asociados con aquellas acciones que evalúan los productos del trabajo de software para determinar su calidad. Los costos de falla incluyen el precio interno de fallar y los efectos externos que precipitan la mala calidad.

La calidad del software se consigue por medio de la aplicación de métodos de ingeniería de software, prácticas adecuadas de administración y un control de calidad exhaustivo, todo lo cual es apoyado por la infraestructura de aseguramiento de la calidad. En los capítulos que siguen se estudian con cierto detalle el control y aseguramiento de la calidad.

Problemas y puntos por evaluar

14.1. Describa cómo evaluaría la calidad de una universidad antes de inscribirse. ¿Cuáles factores serían importantes? ¿Cuáles tendrían importancia crítica?

14.2. Garvin [Gar84] describe cinco puntos de vista distintos sobre la calidad. Dé un ejemplo de cada uno con el uso de uno o más productos electrónicos conocidos con los que esté familiarizado.

14.3. Con el uso de la definición de calidad del software propuesta en la sección 14.2, diga si cree posible crear un producto útil que genere valor medible sin el uso de un proceso eficaz. Explique su respuesta.

14.4. Agregue dos preguntas adicionales a cada una de las dimensiones de la calidad de Garvin presentadas en la sección 14.2.1.

14.5. Los factores de calidad de McCall se desarrollaron en la década de 1970. Casi todos los aspectos de la computación han cambiado mucho desde entonces, no obstante lo cual aún se aplican al software moderno. ¿Qué conclusiones saca con base en ello?

14.6. Con el empleo de los subatributos mencionados en la sección 14.2.3 para el factor de calidad llamado "facilidad de recibir mantenimiento", de la ISO 9126, desarrolle preguntas que exploren si estos atributos existen o no. Continúe el ejemplo presentado en la sección 14.2.4.

14.7. Describa con sus propias palabras el dilema de la calidad del software.

14.8. ¿Qué es un software "suficientemente bueno"? Mencione una compañía dada y productos específicos que crea que fueron desarrollados con el uso de la filosofía de lo suficientemente bueno.

14.9. Considere cada uno de los cuatro aspectos de la calidad y diga cuál piensa que es el más caro y por qué.

14.10. Haga una búsqueda en web y encuentre otros tres ejemplos de "riesgos" para el público que puedan atribuirse directamente a la mala calidad de un software. Comience la búsqueda en **http://catless.ncl.ac.uk/risks.**

14.11. ¿Son lo mismo *calidad* y *seguridad*? Explique su respuesta.

14.12. Explique por qué es que muchos de nosotros utilizamos la ley de Meskimen. ¿Qué ocurre con el software de negocios que causa esto?

Lecturas y fuentes de información adicionales

Los conceptos básicos de calidad del software se estudian en los libros de Henry y Hanlon (*Software Quality Assurance*, Prentice-Hall, 2008), Kahn *et al.* (*Software Quality: Concepts and Practice*, Alpha Science International, Ltd., 2006), O'Regan (*A Practical Approach to Software Quality*, Springer, 2002) y Daughtrey (*Fundamental Concepts for the Software Quality Engineer*, ASQ Quality Press, 2001).

Duvall *et al.* (*Continuous Integration: Improving Software Quality and Reducing Risk*, Addison-Wesley, 2007), Tian (*Software Quality Engineering*, Wiley-IEEE Computer Society Press, 2005), Kandt (*Software Engineering Quality Practices*, Auerbach, 2005), Godbole (*Software Quality Assurance: Principles and Practice*, Alpha Science International, Ltd., 2004) y Galin (*Software Quality Assurance: From Theory to Implementation*, Addison-Wesley, 2003) presentan estudios detallados del aseguramiento de la calidad del software. Stamelos y Sfetsos (*Agile Software Development Quality Assurance*, IGI Global, 2007) estudian el aseguramiento de la calidad en el contexto del proceso ágil.

El diseño sólido conduce a una alta calidad del software. Jayasawal y Patton (*Design for Trustworthy Software*, Prentice-Hall, 2006) y Ploesch (*Contracts, Scenarios and Prototypes*, Springer, 2004) analizan las herramientas y técnicas para desarrollar software "robusto".

La medición es un componente importante de la ingeniería de calidad del software. Ejiogu (*Software Metrics: The Discipline of Software Quality*, BookSurge Publishing, 2005), Kan (*Metrics and Models in Software*

Quality Engineering, Addison-Wesley, 2002) y Nance y Arthur (*Managing Software Quality*, Springer, 2002) estudian unidades de medida y modelos importantes relacionados con la calidad. Los aspectos de la calidad del software orientados al equipo los estudia Evans (*Achieving Software Quality through Teamwork*, Artech House Publishers, 2004).

En internet existe una amplia variedad de fuentes de información acerca de la calidad del software. En el sitio web del libro, en la dirección: **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**, se encuentra una lista actualizada de referencias existentes en la red mundial y que son relevantes para la calidad del software.

CAPÍTULO

16 ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE

CONCEPTOS CLAVE

confiabilidad del software . . 376
elementos del ACS. 370
estadístico 374
plan 379
tareas 371
enfoques formales. 373
estándar ISO 9001-2000 . . 378
metas. 371
seguridad del software 378
Seis Sigma 375

El enfoque de la ingeniería de software descrito en este libro se dirige a una sola meta: producir software a tiempo y de alta calidad. Pero muchos lectores se preguntarán: "¿Qué es calidad del software?".

Philip Crosby [Cro79], en su libro clásico sobre calidad, da una respuesta irónica a esta pregunta:

> El problema de la administración de la calidad no es lo que la gente ignora de ella. El problema es lo que piensan que saben...
>
> En ese sentido, la calidad tiene mucho en común con el sexo. Todo mundo lo busca (en ciertas condiciones, por supuesto). Todos creen que lo entienden (aunque no querrían explicarlo). Todos piensan que su ejecución sólo consiste en seguir las inclinaciones naturales (después de todo, lo hacemos de algún modo). Y, por supuesto, la mayoría de la gente siente que los problemas en esta área los causan las demás personas (si sólo se dieran el tiempo de hacer las cosas bien).

En realidad, la calidad es un concepto difícil (se abordó con cierto detalle en el capítulo 14).[1]

Algunos desarrolladores de software todavía creen que la calidad del software es algo por lo que hay que empezar a preocuparse una vez generado el código. Nada podría estar más lejos de la verdad... El *aseguramiento de la calidad del software* (con frecuencia llamado *administración*

UNA MIRADA RÁPIDA

¿Qué es? No basta hablar por hablar para decir que la calidad del software es importante. Tiene que 1) definirse explícitamente lo que quiere decir "calidad del software", 2) crearse un conjunto de actividades que ayuden a garantizar que todo producto de la ingeniería de software tenga alta calidad, 3) desarrollarse el control de calidad y las actividades para asegurar ésta en todo proyecto de software, 4) usarse métricas para desarrollar estrategias a fin de mejorar el proceso del software y, en consecuencia, la calidad del producto final.

¿Quién lo hace? Todos los involucrados en el proceso de ingeniería de software son los responsables de la calidad.

¿Por qué es importante? Las cosas pueden hacerse bien o pueden volverse a hacer. Si un equipo de software pone el énfasis en la calidad en todas las actividades de la ingeniería de software, se reduce la cantidad de repeticiones que debe hacer. Eso da como resultado costos más bajos y, lo que es más importante, un mejor tiempo para llegar al mercado.

¿Cuáles son las etapas? Antes de iniciar las actividades de aseguramiento de la calidad del software (ACS), es importante definir la *calidad del software* en varios niveles diferentes de abstracción. Una vez que se entiende lo que es la calidad, el equipo de software debe identificar un conjunto de actividades de ACS que filtren los errores de los productos del trabajo antes de que se aprueben.

¿Cuál es el producto final? Se crea un Plan de Aseguramiento de la Calidad del Software para definir una estrategia de ACS del equipo. Durante la modelación y codificación, el producto principal del ACS es la salida de las revisiones técnicas (véase el capítulo 15). Durante las pruebas (capítulos 17 a 20), se generan los planes y procedimientos de prueba, así como otros productos del trabajo asociados con el proceso de mejora.

¿Cómo me aseguro de que lo hice bien? Hay que encontrar los errores antes de que se vuelvan defectos... Es decir, debe trabajarse para mejorar la eficiencia en la eliminación de defectos (capítulo 23), a fin de reducir la cantidad de repeticiones que tenga que hacer el equipo del software.

1 Si no ha leído el capítulo 14, debe leerlo ahora.

de la calidad) es una actividad sombrilla (véase el capítulo 2) que se aplica en todo el proceso del software.

El aseguramiento de la calidad del software (ACS) incluye lo siguiente: 1) un proceso de ACS, 2) tareas específicas de aseguramiento y control de la calidad (incluidas revisiones técnicas y una estrategia de pruebas relacionadas entre sí), 3) prácticas eficaces de ingeniería de software (métodos y herramientas), 4) control de todos los productos del trabajo de software y de los cambios que sufren (véase el capítulo 22), 5) un procedimiento para garantizar el cumplimiento de los estándares del desarrollo de software (cuando sea aplicable) y 6) mecanismos de medición y reporte.

Este capítulo se centra en aspectos de la administración y en las actividades específicas del proceso que permiten a una organización de software garantizar que hace "las cosas correctas en el momento correcto y de la forma correcta".

16.1 ANTECEDENTES

El control y aseguramiento de la calidad son actividades esenciales para cualquier negocio que genere productos que utilicen otras personas. Antes del siglo XX, el control de calidad era responsabilidad única del artesano que elaboraba el producto. Cuando pasó el tiempo y las técnicas de la producción en masa se hicieron comunes, el control de calidad se convirtió en una actividad ejecutada por personas diferentes de aquellas que elaboraban el producto.

Cita:

"Cometes demasiados errores equivocados."

Yogi Berra

La primera función formal de aseguramiento y control de la calidad se introdujo en los laboratorios Bell en 1916 y se difundió con rapidez al resto del mundo de la manufactura. Durante la década de 1940, sugirieron enfoques más formales del control de calidad. Éstos se basaban en la medición y en el proceso de la mejora continua [Dem86] como elementos clave de la administración de la calidad.

Actualmente, toda compañía tiene mecanismos para asegurar la calidad en sus productos. En realidad, en las últimas décadas, las afirmaciones explícitas del compromiso de una compañía con la calidad se han vuelto un mantra de la mercadotecnia.

La historia del aseguramiento de la calidad en el desarrollo del software corre de manera paralela con la historia de la calidad en la manufactura del hardware. En los primeros días de la computación (décadas de 1950 y 1960), la calidad era responsabilidad única del programador. Los estándares para asegurar la calidad del software se introdujeron en los contratos para desarrollar software militar en la década de 1970 y se extendieron con rapidez al desarrollo de software en el mundo comercial [IEE93]. Si se amplía la definición presentada al principio, el aseguramiento de la calidad del software es un "patrón planeado y sistemático de acciones" [Sch98c] que se requieren para garantizar alta calidad en el software. El alcance de la responsabilidad del aseguramiento de la calidad se caracteriza mejor si se parafrasea un comercial de un automóvil popular: "La calidad es el empleo número 1." La implicación para el software es que muchas entidades diferentes tienen responsabilidad en el aseguramiento de la calidad del software: ingenieros de software, gerentes de proyecto, clientes, vendedores y los individuos que trabajan en el grupo de ACS.

El grupo de ACS funciona como representante del cliente en el interior de la empresa. Es decir, la gente que realiza el ACS debe ver al software desde el punto de vista del cliente. ¿El software cumple adecuadamente los factores de calidad mencionados en el capítulo 14? ¿El desarrollo del software se condujo de acuerdo con estándares preestablecidos? ¿Las disciplinas técnicas han cumplido con sus roles como parte de la actividad de ACS? El grupo de ACS trata de responder éstas y otras preguntas para garantizar que se mantenga la calidad del software.

16.2 ELEMENTOS DE ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE

WebRef

En la dirección **www.swqual.com/newsletter/vol2/no1/vol2no1.html**, se encuentra un análisis profundo del ACS, que incluye una amplia variedad de definiciones.

El aseguramiento de la calidad del software incluye un rango amplio de preocupaciones y actividades que se centran en la administración de la calidad del software. Éstas se resumen como sigue [Hor03]:

Estándares. El IEEE, ISO y otras organizaciones que establecen estándares han producido una amplia variedad de ellos para ingeniería de software y documentos relacionados. Los estándares los adopta de manera voluntaria una organización de software o los impone el cliente u otros participantes. El trabajo del ACS es asegurar que los estándares que se hayan adoptado se sigan, y que todos los productos del trabajo se apeguen a ellos.

Revisiones y auditorías. Las revisiones técnicas son una actividad del control de calidad que realizan ingenieros de software para otros ingenieros de software (véase el capítulo 15). Su objetivo es detectar errores. Las auditorías son un tipo de revisión efectuada por personal de ACS con objeto de garantizar que se sigan los lineamientos de calidad en el trabajo de la ingeniería de software. Por ejemplo, una auditoría del proceso de revisión se efectúa para asegurar que las revisiones se lleven a cabo de manera que tengan la máxima probabilidad de descubrir errores.

Pruebas. Las pruebas del software (capítulos 17 a 20) son una función del control de calidad que tiene un objetivo principal: detectar errores. El trabajo del ACS es garantizar que las pruebas se planeen en forma apropiada y que se realicen con eficiencia, de modo que la probabilidad de que logren su objetivo principal sea máxima.

Colección y análisis de los errores. La única manera de mejorar es medir cómo se está haciendo algo. El ACS reúne y analiza errores y datos acerca de los defectos para entender mejor cómo se cometen los errores y qué actividades de la ingeniería de software son más apropiadas para eliminarlos.

Administración del cambio. El cambio es uno de los aspectos que más irrumpe en cualquier proyecto de software. Si no se administra en forma adecuada, lleva a la confusión y ésta casi siempre genera mala calidad. El ACS asegura que se hayan instituido prácticas adecuadas de administración del cambio (véase el capítulo 22).

Educación. Toda organización de software quiere mejorar sus prácticas de ingeniería de software. Un contribuyente clave de la mejora es la educación de los ingenieros de software, de sus gerentes y de otros participantes. La organización de ACS lleva el liderazgo en la mejora del proceso de software (capítulo 30) y es clave para proponer y patrocinar programas educativos.

Cita:

"La excelencia es la capacidad ilimitada de mejorar la calidad de lo que se tenga para ofrecer."

Rick Petin

Administración de los proveedores. Son tres las categorías de software que se adquieren a proveedores externos: *paquetes contenidos en una caja* (por ejemplo, *Office*, de Microsoft); *un shell personalizado* [Hor03], que da una estructura básica, tipo esqueleto, que se adapta de manera única a las necesidades del comprador; y *software contratado*, que se diseña y construye especialmente a partir de especificaciones provistas por la organización cliente. El trabajo de la organización de ACS es garantizar que se obtenga software de alta calidad a partir de las sugerencias de prácticas específicas de calidad que el proveedor debe seguir (cuando sea posible) y de la incorporación de cláusulas de calidad como parte de cualquier contrato con un proveedor externo.

Administración de la seguridad. Con el aumento de los delitos cibernéticos y de las nuevas regulaciones gubernamentales respecto de la privacidad, toda organización de software debe instituir políticas para proteger los datos en todos los niveles, establecer cortafuegos de protección para las *webapps* y asegurar que el software no va a ser vulnerado in-

ternamente. El ACS garantiza que para lograr la seguridad del software, se utilicen el proceso y la tecnología apropiados.

Seguridad. Debido a que el software casi siempre es un componente crucial de los sistemas humanos (como aplicaciones automotrices o aeronáuticas), la consecuencia de defectos ocultos puede ser catastrófica. El ACS es responsable de evaluar el efecto de las fallas del software y de dar los pasos que se requieren para disminuir el riesgo.

Administración de riesgos. Aunque el análisis y la mitigación de riesgos (véase el capítulo 28) es asunto de los ingenieros de software, la organización del ACS garantiza que las actividades de administración de riesgos se efectúen en forma apropiada y que se establezcan planes de contingencia relacionados con los riesgos.

Además de cada una de estas preocupaciones y actividades, el ACS tiene como preocupación dominante asegurar que las actividades de apoyo del software (como mantenimiento, líneas de ayuda, documentación y manuales) se lleven a cabo o se produzcan con calidad.

INFORMACIÓN



Recursos para la administración de la calidad

En la red mundial existen decenas de recursos para la administración de la calidad, incluidas sociedades profesionales, organizaciones emisoras de estándares y fuentes de información general. Los sitios siguientes constituyen un buen punto de partida:

American Society for Quality (ASQ), División de Software, **www.asq.org/software**
Association for Computer Machinery, **www.acm.org**, Centro de Datos y Análisis del Software (DACS), **www.dacs.dtic.mil/**
International Organization for Standardization (ISO), **www.iso.ch**
ISO SPIECE, **www.isospiece.com**
Malcolm Baldridge National Quality Award, **www.quality.nist.gov**
Software Engineering Institute, **www.sei.cmu.edu/**
Software Testing and Quality Engineering, **www.stickyminds.com**
Six Sigma Resources, **www.isixsigma.com/** **www.asq.org/sixsigma/**
TickIT International, temas sobre certificación de la calidad, **www.tickit.org/international.htm**
Total Quality Management (TQM)
Información general: **www.gslis.utexas.edu/~rpollock/tqm.html**
Artículos: **www.work911.com/tqmarticles.htm**
Glosario: **www.quality.org/TQM-MSI/TQM-glossary.html**

16.3 TAREAS, METAS Y MÉTRICAS DEL ACS

El aseguramiento de la calidad del software se compone de varias tareas asociadas con dos entidades diferentes: los ingenieros de software que hacen el trabajo técnico y un grupo de ACS que tiene la responsabilidad de planear, supervisar, registrar, analizar y hacer reportes acerca de la calidad.

Los ingenieros de software abordan la calidad (y ejecutan actividades para controlarla), aplicando métodos y medidas técnicas sólidos, realizando revisiones técnicas y haciendo pruebas de software bien planeadas.

16.3.1 Tareas del ACS

El objetivo del grupo de ACS es auxiliar al equipo del software para lograr un producto final de alta calidad. El Instituto de Ingeniería de Software recomienda un conjunto de acciones de ACS que se dirigen a la planeación, supervisión, registro, análisis y elaboración de reportes para el aseguramiento de la calidad. Estas acciones son realizadas (o facilitadas) por un grupo independiente de ACS que hace lo siguiente:

¿Cuál es el rol de un grupo de ACS?

Prepara el plan de ACS para un proyecto. El plan se desarrolla como parte de la preparación del proyecto y es revisado por todos los participantes. Las acciones de aseguramiento de la calidad efectuadas por el equipo de ingeniería de software y por el grupo de ACS son dirigidas por el plan. Éste identifica las evaluaciones que se van a realizar, las auditorías y revisiones por efectuar, los estándares aplicables al proyecto, los procedimientos para reportar y dar seguimiento a los errores, los productos del trabajo que genera el grupo de ACS y la retroalimentación que se dará al equipo del software.

Participa en el desarrollo de la descripción del software del proyecto. El equipo de software selecciona un proceso para el trabajo que se va a realizar. El grupo de ACS revisa la descripción del proceso a fin de cumplir con la política organizacional, los estándares internos para el software, los estándares impuestos desde el exterior (como la norma ISO-9001) y otras partes del plan del proyecto de software.

Revisa las actividades de la ingeniería de software a fin de verificar el cumplimiento mediante el proceso definido para el software. El grupo de ACS identifica, documenta y da seguimiento a las desviaciones del proceso y verifica que se hayan hecho las correcciones pertinentes.

Audita los productos del trabajo de software designados para verificar que se cumpla con aquellos definidos como parte del proceso de software. El grupo de ACS revisa productos del trabajo seleccionados; identifica, documenta y da seguimiento a las desviaciones; verifica que se hayan hecho las correcciones necesarias y reporta periódicamente los resultados de su trabajo al gerente del proyecto.

Asegura que las desviaciones en el trabajo de software y sus productos se documenten y manejen de acuerdo con un procedimiento documentado. Las desviaciones pueden encontrarse en el plan del proyecto, la descripción del proceso, los estándares aplicables o los productos del trabajo de la ingeniería de software.

Registra toda falta de cumplimiento y la reporta a la alta dirección. Se da seguimiento a los incumplimientos hasta que son resueltos.

Además de estas acciones, el grupo de ACS coordina el control y administración del cambio (véase el capítulo 22) y ayuda a recabar y analizar métricas para el software.

16.3.2 Metas, atributos y métricas

Cita:

"La calidad nunca es un accidente; siempre es el resultado de una intención clara, un esfuerzo sincero, una dirección inteligente y una ejecución hábil; representa la elección sabia de muchas alternativas".

William A. Foster

Las acciones de ACS descritas en la sección anterior se realizan con objeto de alcanzar un conjunto de metas pragmáticas:

Calidad de los requerimientos. La corrección, completitud y consistencia del modelo de requerimientos tendrá una gran influencia en la calidad de todos los productos del trabajo que sigan. El ACS debe garantizar que el equipo de software ha revisado en forma apropiada el modelo de requerimientos a fin de alcanzar un alto nivel de calidad.

Calidad del diseño. Todo elemento del modelo del diseño debe ser evaluado por el equipo del software para asegurar que tenga alta calidad y que el diseño en sí se apegue a los requerimientos. El ACS busca atributos del diseño que sean indicadores de la calidad.

Calidad del código. El código fuente y los productos del trabajo relacionados (por ejemplo, otra información descriptiva) deben apegarse a los estándares locales de codificación y tener características que faciliten darle mantenimiento. El ACS debe identificar aquellos atributos que permitan hacer un análisis razonable de la calidad del código.

Eficacia del control de calidad. Un equipo de software debe aplicar recursos limitados, en forma tal que tenga la máxima probabilidad de lograr un resultado de alta calidad. El

FIGURA 16.1 **Metas atributos y métricas de la calidad del software**
Fuente: Adaptado de [Hya96].

Meta	Atributo	Métrica
Calidad de los requerimientos	Ambigüedad	Número de modificadores ambiguos (por ejemplo, muchos, grande, amigable, etc.)
	Completitud	Número de TBA y TBD
	Comprensibilidad	Número de secciones y subsecciones
	Volatilidad	Número de cambios por requerimiento
		Tiempo (por actividad) cuando se solicita un cambio
	Trazabilidad	Número de requerimientos no trazables hasta el diseño o código
	Claridad del modelo	Número de modelos UML
		Número de páginas descriptivas por modelo
		Número de errores de UML
Calidad del diseño	Integridad arquitectónica	Existencia del modelo arquitectónico
	Completitud de componentes	Número de componentes que se siguen hasta el modelo arquitectónico
		Complejidad del diseño del procedimiento
	Complejidad de la interfaz	Número promedio de pasos para llegar a una función o contenido normal
		Distribución apropiada
	Patrones	Número de patrones utilizados
Calidad del código	Complejidad	Complejidad ciclomática
	Facilidad de mantenimiento	Factores de diseño (capítulo 8)
	Comprensibilidad	Porcentaje de comentarios internos
		Convenciones variables de nomenclatura
	Reusabilidad	Porcentaje de componentes reutilizados
	Documentación	Índice de legibilidad
Eficacia del control de calidad	Asignación de recursos	Porcentaje de personal por hora y por actividad
	Tasa de finalización	Tiempo de terminación real *versus* lo planeado
	Eficacia de la revisión	Ver medición de la revisión (capítulo 14)
	Eficacia de las pruebas	Número de errores de importancia crítica encontrados
		Esfuerzo requerido para corregir un error
		Origen del error

ACS analiza la asignación de recursos para las revisiones y pruebas a fin de evaluar si se asignan en la forma más eficaz.

La figura 16.1 (adaptada de [Hya96]) identifica los atributos que son indicadores de la existencia de la calidad para cada una de las metas mencionadas. También se presentan las métricas que se utilizan para indicar la fortaleza relativa de un atributo.

16.4 ENFOQUES FORMALES AL ACS

En las secciones anteriores, se dijo que la calidad del software es el trabajo de cada quien y que puede lograrse por medio de una práctica competente de la ingeniería de software, así como de la aplicación de revisiones técnicas, de una estrategia de pruebas con relaciones múltiples, de un mejor control de los productos del trabajo de software y de los cambios efectuados sobre

WebRef

En la dirección **www.gslis.utexas.edu/~rpollock/tqm.html**, se encuentra información útil acerca del ACS y de los métodos formales de la calidad.

ellos, así como de la aplicación de estándares aceptados de la ingeniería de software. Además, la calidad se define en términos de una amplia variedad de atributos de la calidad y se mide (indirectamente) con el empleo de varios índices y métricas.

En las últimas tres décadas, un segmento pequeño pero sonoro de la comunidad de la ingeniería de software ha afirmado que se requiere un enfoque más formal para el ACS. Puede decirse que un programa de cómputo es un objeto matemático. Para cada lenguaje de programación, es posible definir una sintaxis y semántica rigurosas, y se dispone de un enfoque igualmente riguroso para la especificación de los requerimientos del software (véase el capítulo 21). Si el modelo de los requerimientos (especificación) y el lenguaje de programación se representan en forma rigurosa, debe ser posible usar una demostración matemática para la corrección, de modo que se confirme que un programa se ajusta exactamente a sus especificaciones.

Los intentos de demostrar la corrección de un programa no son nuevos. Dijkstra [Dij76a] y Linger, Mills y Witt [Lin79], entre otros, han invocado pruebas de la corrección de programas y las han relacionado con el uso de conceptos de programación estructurada (véase el capítulo 10).

16.5 ASEGURAMIENTO ESTADÍSTICO DE LA CALIDAD DEL SOFTWARE

El aseguramiento estadístico de la calidad del software refleja una tendencia creciente en la industria para que se vuelva más cuantitativo respecto de la calidad. Para el software, el aseguramiento estadístico de la calidad implica los pasos siguientes:

¿Qué pasos se requieren para efectuar el ACS estadístico?

1. Se recaba y clasifica la información acerca de errores y defectos del software.
2. Se hace un intento por rastrear cada error y defecto hasta sus primeras causas (por ejemplo, no conformidad con las especificaciones, error de diseño, violación de los estándares, mala comunicación con el cliente, etc.).
3. Con el uso del Principio de Pareto (80 por ciento de los defectos se debe a 20 por ciento de todas las causas posibles), se identifica 20 por ciento de las causas de errores y defectos (las *pocas vitales*).
4. Una vez identificadas las pocas causas vitales, se corrigen los problemas que han dado origen a los errores y defectos.

Este concepto relativamente simple representa un paso importante hacia la creación de un proceso adaptativo del software en el que se hacen cambios para mejorar aquellos elementos del proceso que introducen errores.

16.5.1 Ejemplo general

Cita:

"Un análisis estadístico, si se realiza en forma apropiada, es una disección delicada de las incertidumbres, una cirugía de las suposiciones."

M. J. Moroney

A fin de ilustrar el uso de los métodos estadísticos para el trabajo de ingeniería de software, suponga que una organización de ingeniería de software recaba información sobre los errores y defectos cometidos en un periodo de un año. Algunos de dichos errores se descubren a medida que se desarrolla el software. Otros (defectos) se encuentran después de haber liberado el software a sus usuarios finales. Aunque se descubren cientos de problemas diferentes, todos pueden rastrearse hasta una (o más) de las causas siguientes:

- Especificaciones erróneas o incompletas (EEI)
- Mala interpretación de la comunicación con el cliente (MCC)
- Desviación intencional de las especificaciones (DIE)
- Violación de los estándares de programación (VEP)
- Error en la representación de los datos (ERD)

FIGURA 16.2
Colección de datos para hacer ACS estadístico

	Total		Serio		Moderado		Menor	
Error	**No.**	**%**	**No.**	**%**	**No.**	**%**	**No.**	**%**
IES	205	22%	34	27%	68	18%	103	24%
MCC	156	17%	12	9%	68	18%	76	17%
IDS	48	5%	1	1%	24	6%	23	5%
VPS	25	3%	0	0%	15	4%	10	2%
EDR	130	14%	26	20%	68	18%	36	8%
ICI	58	6%	9	7%	18	5%	31	7%
EDL	45	5%	14	11%	12	3%	19	4%
IET	95	10%	12	9%	35	9%	48	11%
IID	36	4%	2	2%	20	5%	14	3%
PLT	60	6%	15	12%	19	5%	26	6%
HCI	28	3%	3	2%	17	4%	8	2%
MIS	56	6%	0	0%	15	4%	41	9%
Totales	942	100%	128	100%	379	100%	435	100%

- Interfaz componente inconsistente (ICI)
- Error en el diseño lógico (EDL)
- Pruebas incompletas o erróneas (PIE)
- Documentación inexacta o incompleta (DII)
- Error en la traducción del lenguaje de programación del diseño (LPD)
- Interfaz humano/computadora ambigua o inconsistente (IHC)
- Varios (V)

Cita:

"20 por ciento del código tiene 80 por cierto de los errores. Encuéntrelos, corríjalos".

Lowell Arthur

Para aplicar el ACS estadístico, se elabora la tabla de la figura 16.2. La tabla indica que EEI, MCC y ERD son las pocas causas vitales que originan 53 por ciento de todos los errores. Sin embargo, debe notarse que EEI, ERD, LPD y EDL se habrían seleccionado como las pocas causas vitales si se consideran sólo errores serios. Una vez que las pocas causas vitales han sido determinadas, la organización de ingeniería de software comienza su acción correctiva. Por ejemplo, a fin de corregir el MCC, deben implementarse técnicas para recabar requerimientos (capítulo 5) que mejoren la calidad de la comunicación y las especificaciones con el cliente. Para mejorar el ERD, deben adquirirse herramientas para desarrollar la modelación de casos y realizar datos y revisiones del diseño más significativos.

Es importante notar que la acción correctiva se centra sobre todo en las pocas causas vitales. En tanto éstas se corrigen, nuevas candidatas se van a la cumbre de la pila.

Las técnicas para el aseguramiento correctivo han sido propuestas para dar una mejora sustancial de la calidad [Art97]. En ciertos casos, las organizaciones de software han tenido una reducción anual de 50 por ciento en defectos después de aplicar esta técnica.

La aplicación del ACS estadístico y el Principio de Pareto se resumen en una sola oración: *Pasa tu tiempo viendo las cosas que realmente importan, pero primero asegúrate de que entiendes lo que realmente importa...*

16.5.2 Seis Sigma para la ingeniería de software

Seis Sigma es la estrategia más ampliamente usada hoy para el aseguramiento estadístico de la calidad en la industria. La estrategia Seis Sigma fue popularizada originalmente por Motorola en la década de 1980 y "es una metodología rigurosa y disciplinada que usa datos y análisis estadísticos para medir y mejorar el desempeño operativo de una compañía, identificando y eliminando defectos en procesos de manufactura y servicios" [ISI08]. El término Seis Sigma se deriva

de seis desviaciones estándar —3.4 casos (defectos) por millón de ocurrencias—, lo que implica un estándar de calidad extremadamente alto. La metodología Seis Sigma define tres etapas fundamentales:

¿Cuáles son las etapas fundamentales de la metodología Seis Sigma?

- *Definir* los requerimientos del cliente y los que se le entregan, así como las metas del proyecto a través de métodos bien definidos de comunicación con el cliente.
- *Medir* el proceso existente y su resultado para determinar el desempeño actual de la calidad (recabar métricas para los defectos).
- *Analizar* las métricas de los defectos y determinar las pocas causas vitales.

Si se trata de un proceso de software existente que se requiere mejorar, Seis Sigma sugiere dos etapas adicionales:

- *Mejorar* el proceso, eliminando las causas originales de los defectos.
- *Controlar* el proceso para asegurar que el trabajo futuro no vuelva a introducir las causas de los defectos.

Estas etapas fundamentales y adicionales en ocasiones son conocidas como método DMAMC (definir, medir, analizar, mejorar y controlar).

Si una organización va a desarrollar un proceso de software (en vez de mejorar uno existente), a las etapas fundamentales se agregan las siguientes:

- *Diseñar* el proceso para 1) evitar las causas originales de los defectos y 2) cumplir los requerimientos del cliente.
- *Verificar* que el modelo del proceso en realidad evite los defectos y cumpla los requerimientos del cliente.

Esta variación en ocasiones es denominada método DMADV (definir, medir, analizar, diseñar y verificar).

El estudio detallado de Seis Sigma se deja a fuentes dedicadas a ese tema. Si el lector tiene interés al respecto, consulte [ISI08], [Pyz303] y [Sne03].

16.6 CONFIABILIDAD DEL SOFTWARE

Cita:

"El precio inevitable de la confiabilidad es la simplicidad."

C. A. R. Hoare

No hay duda de que la confiabilidad de un programa de cómputo es un elemento importante de su calidad general. Si un programa falla repetida y frecuentemente en su desempeño, importa poco si otros factores de la calidad del software son aceptables.

La confiabilidad del software, a diferencia de muchos otros factores de la calidad, se mide y estima directamente mediante el uso de datos históricos del desarrollo. La *confiabilidad del software* se define en términos estadísticos como "la probabilidad que tiene un programa de cómputo de operar sin fallas en un ambiente específico por un tiempo específico" [Mus87]. Para ilustrar lo anterior, digamos que se estima que el programa *X* tiene una confiabilidad de 0.999 durante ocho horas de procesamiento continuo. En otras palabras, si el programa *X* fuera a ejecutarse 1 000 veces y requiriera un total de ocho horas de tiempo de procesamiento continuo (tiempo de procesamiento), es probable que operara correctamente (sin fallas) 999 veces.

Siempre que se trate de la confiabilidad del software, surge una pregunta crucial: ¿qué significa el término *falla*? En el contexto de cualquier análisis de la calidad y confiabilidad del software, la falla significa la falta de conformidad con los requerimientos del software. Pero, incluso con esta definición, hay gradaciones. Las fallas pueden ser leves o catastróficas. Una falla podría corregirse en segundos, mientras que otra tal vez requiera de varias semanas o meses de trabajo para ser corregida. Para complicar más el asunto, la corrección de una falla quizá dé como resultado la introducción de otros errores que a su vez originen otras fallas.

16.6.1 Mediciones de la confiabilidad y disponibilidad

PUNTO CLAVE

Los problemas de confiabilidad del software casi siempre pueden seguirse hasta encontrar defectos en el diseño o en la implementación.

Los primeros trabajos sobre confiabilidad del software trataban de extrapolar la teoría matemática de la confiabilidad del hardware a la predicción de la confiabilidad del software. La mayor parte de modelos relacionados con el hardware se abocan a la falla debida al uso, en lugar de a la que tiene su origen en los defectos de diseño. En el hardware, las fallas debidas al uso físico (por ejemplo, los efectos de temperatura, corrosión y golpes) son más probables que las debidas al diseño. Desafortunadamente, con el software ocurre lo contrario. En realidad, todas las fallas del software pueden rastrearse en problemas de diseño o de implementación; el uso (véase el capítulo 1) no entra en el escenario.

Ha habido un debate permanente acerca de la relación que existe entre los conceptos clave en la confiabilidad del hardware y su aplicabilidad al software. Aunque es posible establecer un vínculo irrefutable, es útil considerar algunos conceptos sencillos que se aplican a ambos elementos del sistema.

PUNTO

TMEF: El tiempo promedio para que un sistema funcióne sin averiarse

Si se considera un sistema basado en computadora, una medida sencilla de su confiabilidad es el *tiempo medio entre fallas* (TMEF):

$$\text{TMEF} = \text{TMPF} + \text{TMPR}$$

donde las siglas TMPF y TMPR significan *tiempo medio para la falla* y *tiempo medio para la reparación*,[2] respectivamente.

CPU, no en tiempo de reloj.

Muchos investigadores afirman que el TMEF es una medición más útil que otras relacionadas con la calidad del software que se estudian en el capítulo 23. En pocas palabras, a un usuario final le preocupan las fallas, no la cuenta total de defectos. Como cada defecto contenido en un programa no tiene la misma tasa de fallas, la cuenta total de defectos indica muy poco acerca de la confiabilidad del sistema. Por ejemplo, considere un programa que haya estado en operación durante 3 000 horas de procesador sin falla. Muchos defectos de este programa estarían sin detectar durante decenas de miles de horas antes de ser descubiertos. El TMEF de tales errores oscuros podría ser de 30 000 o hasta 60 000 horas de procesador. Otros defectos, no descubiertos, podrían tener una tasa de fallas de 4 000 a 5 000 horas. Aun si cada uno de los errores en esta categoría (los que tienen un TMEF largo) se eliminara, el efecto que tendrían sobre el software sería despreciable.

CONSEJO

Algunos aspectos de la disponibilidad

La disponibilidad: Te indica de ese tiempo que el software fallo y se reparo cuanto tiempo estubo activo

Sin embargo, el TMEF puede ser problemático por dos razones: 1) proyecta un tiempo entre fallas, pero no da una tasa de fallas proyectada y 2) puede interpretarse mal, como la vida promedio, cuando *no* es esto lo que implica.

Una medición alternativa de confiabilidad es la de las *fallas en el tiempo* (FET): medición estadística de cuántas fallas tendrá un componente en mil millones de horas de operación. Por tanto, 1 FET es equivalente a una falla en cada mil millones de horas de operación.

Además de una medida de la confiabilidad, también debe desarrollarse otra para la disponibilidad. La *disponibilidad del software* es la probabilidad de que un programa opere de acuerdo con los requerimientos en un momento determinado de tiempo, y se define así:

disponible.

$$\text{Disponibilidad} = \frac{\text{TMPF}}{\text{TMPF} + \text{TMPR}} \times 100\%$$

La medición del TMEF para la confiabilidad es igualmente sensible al TMPF y al TMPR. La medición de la disponibilidad es un poco más sensible al TMPR, que es una medición indirecta de la facilidad que tiene el software para recibir mantenimiento.

---

2 Aunque tal vez se requiera depurar (y hacer otras correcciones relacionadas) como consecuencia de la falla, en muchos casos el software funcionará de manera apropiada después de reiniciar, sin ningún otro cambio.

16.6.2 Seguridad del software

Cita:

"La seguridad de las personas debe ser la ley máxima."

Cicerón

La *seguridad del software* es una actividad del aseguramiento del software que se centra en la identificación y evaluación de los peligros potenciales que podrían afectarlo negativamente y que podrían ocasionar que falle todo el sistema. Si los peligros se identifican al principio del proceso del software, las características de su diseño se especifican de modo que los eliminen o controlen.

Como parte de la seguridad del software, se lleva a cabo un proceso de modelado y análisis. Inicialmente se identifican los peligros y se clasifican según su riesgo. Por ejemplo, algunos de los peligros asociados con un control de crucero basado en computadora para un automóvil podrían ser los siguientes: 1) ocasionar una aceleración incontrolada que no pudiera detenerse, 2) no responder a la presión en el pedal de frenado (porque se apague), 3) no encender cuando se active el interruptor y 4) perder o ganar velocidad poco a poco. Una vez identificados estos peligros en el nivel del sistema, se utilizan técnicas de análisis para asignar severidad y probabilidad de ocurrencia a cada uno.[3] Para ser eficaz, el software debe analizarse en el contexto de todo el sistema. Por ejemplo, un error sutil en la entrada de un usuario (las personas son componentes del sistema) podría ampliarse por una falla del software y producir datos de control que situaran equivocadamente un dispositivo mecánico. Si y sólo si se encontrara un único conjunto de condiciones ambientales externas, la posición falsa del dispositivo mecánico ocasionaría una falla desastrosa. Podrían usarse técnicas de análisis [Eri05], tales como árbol de fallas, lógica en tiempo real y modelos de red de Petri, para predecir la cadena de eventos que ocasionarían los peligros, así como la probabilidad de ocurrir que tendría cada uno de los eventos para generar la cadena.

Cita:

"No puedo imaginar ninguna condición que hiciera que esta nave se hundiera. La construcción naval moderna ha llegado más allá de eso".

E. I. Smith, capitán del *Titanic*

Una vez identificados y analizados los peligros, pueden especificarse requerimientos relacionados con la seguridad para el software. Es decir, la especificación contendría una lista de eventos indeseables y las respuestas deseadas del sistema ante ellos. Después se indicaría el papel del software en la administración indeseable de los mismos.

Aunque la confiabilidad y la seguridad del software están muy relacionadas, es importante entender la sutil diferencia entre ellas. La primera utiliza técnicas de análisis estadístico para determinar la probabilidad de que ocurra una falla del software. Sin embargo, la ocurrencia de una falla no necesariamente da como resultado un peligro o riesgo. La seguridad del software examina las formas en las que las fallas generan condiciones que llevan a un peligro. Es decir, las fallas no se consideran en el vacío, sino que se evalúan en el contexto de la totalidad del sistema basado en computadora y de su ambiente.

WebRef

En la dirección **www.safeware-eng.com/**, se encuentran varios artículos sobre seguridad del software.

El estudio exhaustivo de la seguridad del software está más allá del alcance de este libro. Si el lector está interesado en la seguridad del software y en otros aspectos relacionados, consulte [Smi05], [Dun02] y [Lev95].

16.7 Las normas de calidad ISO 9000[4]

Un *sistema de aseguramiento de la calidad* se define como la estructura organizacional, responsabilidades, procedimientos, procesos y recursos necesarios para implementar la administración de la calidad [ANS87]. Los sistemas de aseguramiento de la calidad se crean para ayudar a las organizaciones a asegurar que sus productos y servicios satisfagan las expectativas del con-

3 Este enfoque es similar a los métodos de análisis del riesgo descritos en el capítulo 28. La diferencia principal es el énfasis que se pone en aspectos de la tecnología en lugar de en los relacionados con el proyecto.

4 Esta sección, escrita por Michael Stovski, ha sido adaptada a partir de "Fundamentos de ISO 9000", libro de trabajo desarrollado para *Essential Software Engineering*, video desarrollado por R. S. Pressman & Associates, Inc. Se reimprime con su autorización.

sumidor gracias a que cumplan con sus especificaciones. Estos sistemas cubren una amplia variedad de actividades, que contemplan todo el ciclo de vida del producto, incluidos planeación, control, medición, pruebas e informes, así como la mejora de los niveles de calidad en todo el proceso de desarrollo y manufactura. La norma ISO 9000 describe en términos generales los elementos de aseguramiento de la calidad que se aplican a cualquier negocio, sin importar los productos o servicios ofrecidos.

Para registrarse en alguno de los modelos del sistema de aseguramiento de la calidad contenidos en la ISO 9000, por medio de auditores externos se revisan en detalle el sistema y las operaciones de calidad de una compañía, respecto del cumplimiento del estándar y de la operación eficaz. Después de un registro exitoso, el grupo de registro representado por los auditores emite un certificado para la compañía. Auditorías semestrales de supervisión aseguran el cumplimiento continuo de la norma.

WebRef

En la dirección **www.tantara.ab.ca/info.htm**, se encuentran muchos vínculos hacia los recursos de la norma ISO 9000/9001.

Los requerimientos esbozados por la norma ISO 9001:2000 se dirigen a temas tales como responsabilidad de la administración, sistema de calidad, revisión del contrato, control del diseño, documentación y control de datos, identificación del producto y su seguimiento, control del proceso, inspección y pruebas, acciones correctivas y preventivas, registros del control de calidad, auditorías internas de calidad, capacitación, servicio y técnicas estadísticas. A fin de que una organización de software se registre en la ISO 9001:2000, debe establecer políticas y procedimientos que cumplan cada uno de los requerimientos mencionados (y otros más), y después demostrar que sigue dichas políticas y procedimientos. Si el lector desea más información sobre la norma ISO 9001:2000, consulte [Ant06], [Mut03] o [Dob04].

INFORMACIÓN



La norma ISO 9001:2000

La descripción siguiente define los elementos básicos de la norma ISO 9001:2000. Información completa sobre la misma se obtiene en la Organización Internacional de Normas (**www.iso.ch**) y en otras fuentes de internet (como en **www.praxiom.com**).

Establecer los elementos de un sistema de administración de la calidad.
- Desarrollar, implementar y mejorar el sistema.
- Definir una política que ponga el énfasis en la importancia del sistema.

Documentar el sistema de calidad.
- Describir el proceso.
- Producir un manual de operación.
- Desarrollar métodos para controlar (actualizar) documentos.
- Establecer métodos de registro.

Apoyar el control y aseguramiento de la calidad.
- Promover la importancia de la calidad entre todos los participantes.
- Centrarse en la satisfacción del cliente.
- Definir un plan de calidad que se aboque a los objetivos, responsabilidades y autoridad.
- Definir mecanismos de comunicación entre los participantes.

Establecer mecanismos de revisión para el sistema de administración de la calidad.
- Identificar métodos de revisión y mecanismos de retroalimentación.
- Definir procedimientos para dar seguimiento.

Identificar recursos para la calidad, incluidos personal, capacitación y elementos de la infraestructura.
- Establecer mecanismos de control.
- Para la planeación
- Para los requerimientos del cliente
- Para las actividades técnicas (tales como análisis, diseño y pruebas)
- Para la vigilancia y administración del proyecto

Definir métodos de corrección.
- Evaluar datos y métricas de la calidad.
- Definir el enfoque para la mejora continua del proceso y la calidad.

16.8 EL PLAN DE ACS

El *Plan de ACS* proporciona un mapa de ruta para instituir el aseguramiento de la calidad del software. Desarrollado por el grupo de ACS (o por el equipo del software si no existe un grupo de ACS), el plan funciona como plantilla para las actividades de ACS que se instituyen para cada proyecto de software.

La IEEE [IEEE93] ha publicado una norma para el ACS. Ésta recomienda una estructura que identifica lo siguiente: 1) propósito y alcance del plan, 2) descripción de todos los productos del trabajo de ingeniería de software (tales como modelos, documentos, código fuente, etc.) que se ubiquen dentro del ámbito del ACS, 3) todas las normas y prácticas aplicables que se utilicen durante el proceso del software, 4) acciones y tareas del ACS (incluidas revisiones y auditorías) y su ubicación en el proceso del software, 5) herramientas y métodos que den apoyo a las acciones y tareas de ACS, 6) procedimientos para la administración de la configuración del software (véase el capítulo 22), 7) métodos para unificar las salvaguardas y para mantener todos los registros relacionados con el ACS y 8) roles y responsabilidades relacionados con la calidad del producto.

HERRAMIENTAS DE SOFTWARE



Administración de la calidad del software

Objetivos: El objetivo de las herramientas del ACS es ayudar al equipo del proyecto a evaluar y mejorar la calidad del producto del trabajo de software.

Mecánica: La mecánica de las herramientas varía. En general, el objetivo consiste en evaluar la calidad de un producto específico. Nota: Es frecuente que dentro de la categoría de herramientas para el ACS, se incluya una amplia variedad de herramientas para someter a prueba al software (véanse los capítulos 17 a 20).

Herramientas representativas[5]

ARM, desarrollada por la NASA (**state.gsfc.nasa.gov/tools/index.html**), proporciona mediciones que se utilizan para evaluar la calidad de un documento de requerimientos de software.

QPR ProcessGuide and Scorecard, desarrollada por QPR Software (**www.qpronline.com**), da apoyo para establecer Seis Sigma y otros enfoques de administración de la calidad.

Quality Tools and Templates, desarrollada por iSixSigma (**www.isixsigma.com/tt/**), describe un amplio abanico de herramientas y métodos útiles para la administración de la calidad.

NASA Quality Resources, desarrollada por el Centro Coddard de Vuelos Espaciales (**sw-assurance.gsfc.nasa.gov/index.php**), contiene formatos, plantillas, listas de verificación y herramientas que son útiles para el ACS.

16.9 RESUMEN

El aseguramiento de la calidad del software es una actividad sombrilla de la ingeniería de software que se aplica en cada etapa del proceso del software. El ACS incluye procedimientos para la aplicación eficaz de métodos y herramientas, supervisa las actividades de control de calidad, tales como las revisiones técnicas y las pruebas del software, procedimientos para la administración del cambio, y procedimientos para asegurar el cumplimiento de las normas y mecanismos de medición y elaboración de reportes.

Para llevar a cabo el aseguramiento de la calidad del software de manera adecuada, deben recabarse, evaluarse y divulgarse datos sobre el proceso de la ingeniería de software. Los métodos estadísticos aplicados al ACS ayudan a mejorar la calidad del producto y del proceso de software mismo. Los modelos de confiabilidad del software amplían las mediciones, lo que permite que los datos obtenidos acerca de los defectos se extrapolen hacia tasas de falla proyectadas y hacia la elaboración de pronósticos de confiabilidad.

En resumen, deben tomarse en cuenta las palabras de Dunn y Ullman [Dun82]: "El aseguramiento de la calidad del software es el mapeo de los preceptos administrativos y de las disciplinas de diseño del aseguramiento de la calidad, en el ámbito administrativo y tecnológico aplicable a la ingeniería de software." La capacidad de asegurar la calidad es la medida de una

5 Las herramientas mencionadas aquí no son obligatorias, sino una muestra de las que hay en esta categoría. En la mayoría de casos, los nombres de las herramientas son marcas registradas por sus respectivos desarrolladores.

disciplina madura de la ingeniería. Cuando el mapeo se lleva a cabo con éxito, el resultado es una ingeniería de software madura.

Problemas y puntos por evaluar

16.1. Algunas personas afirman que "el control de la variación es el corazón del control de calidad". Como todo programa que se crea es diferente de cualquier otro programa, ¿cuáles son las variaciones que se buscan y cómo se controlan?

16.2. ¿Es posible evaluar la calidad del software si el cliente cambia continuamente lo que se supone que debe hacerse?

16.3. La calidad y confiabilidad son conceptos relacionados, pero difieren en lo fundamental por varias razones. Analice las diferencias.

16.4. ¿Un programa puede corregirse y aún así ser confiable? Explique su respuesta.

16.5. ¿Un programa puede corregirse y tener buena calidad? Explique lo que responda.

16.6. ¿Por qué es frecuente que haya tensiones entre el grupo de ingeniería de software y el del aseguramiento de la calidad? ¿Es saludable eso?

16.7. El lector tiene la responsabilidad de mejorar la calidad del software en su organización. ¿Qué es lo primero que debe hacer? ¿Qué es lo siguiente?

16.8. Además de contar los errores y defectos, ¿hay otras características cuantificables de software que impliquen calidad? ¿Cuáles son y cómo podrían medirse directamente?

16.9. El concepto del tiempo medio para la falla del software es objeto de críticas. Explique por qué.

16.10. Considere dos sistemas cuya seguridad sea crítica y que estén controlados por computadora. Enliste al menos tres peligros que se relacionen directamente con fallas del software.

16.11. Obtenga una copia de las normas ISO 9001:2000 e ISO 9000-3. Prepare una presentación que analice tres requerimientos de ISO 9001 y la forma en la que se apliquen en el contexto del software.

Lecturas y fuentes de información adicionales

Los libros de Hoyle (*Quality Management Fundamentals*, Butterworth-Heinemann, 2007), Tian (*Software Quality Engineering*, Wiley-IEEE Computer Society Press, 2005), El Emam (*The ROI from Software Quality*, Auerbach, 2005) y Horch (*Practical Guide to Software Quality Management*, Artech House, 2003), y Nance y Arthur (*Managing Software Quality*, Springer, 2002) son presentaciones excelentes en el nivel de administración acerca de los beneficios de los programas formales de aseguramiento de la calidad del software de computadora. Las obras de Deming [Dem86], Juran (*Juran on Quality by Design*, Free Press, 1992) y Crosby ([Cro79], así como *Quality is Still Free*, McGraw-Hill, 1995) no se abocan al software, pero son una lectura obligada para los altos directivos que tengan responsabilidades en el desarrollo del software. Gluckman y Roome (*Everyday Heroes of the Quality Movement*, Dorset House, 1993) humanizan los aspectos de la calidad a través de la historia de los actores participantes en el proceso. Kan (*Metrics and Models in Software Quality Engineering*, Addison-Wesley, 1995) presenta un enfoque cuantitativo de la calidad del software.

Los libros de Evans (*Total Quality: Management, Organization and Strategy*, 4a. ed., South Western College Publishing, 2004), Bru (*Six Sigma for Managers*, McGraw-Hill, 2005) y Dobb (*ISO 9001:2000 Quality Registration Step-by-Step*, 3a. ed., Butterworth-Heinemann, 2004) son representativos de los muchos que se han escrito sobre Seis Sigma e ISO 9001:2000, respectivamente.

Pham (*System Software Reliability,* Springer, 2006), Musa (*Software Reliability Engineering: More Reliable Software, Faster Development and Testing*, 2a. ed., McGraw-Hill, 2004) y Peled (*Software Reliability Methods*, Springer, 2001) proporcionan guías prácticas que describen los métodos para medir y analizar la confiabilidad del software.

Vincoli (*Basic Guide to System Safety*, Wiley 2006), Dhillon (*Engineering Safety*, World Scientific Publishing Co., Inc., 2003), Hermann (*Software Safety and Reliability*, Wiley-IEEE Computer Society Press, 2000), Storey (*Safety-Critical Computer Systems*, Addison-Wesley, 1996) y Leveson [Lev95] aportan los análisis más exhaustivos que se hayan publicado hasta la fecha acerca de la seguridad del software y del sistema. Además, Van

der Meulen (*Definitions for Hardware and Software Safety Engineers*, Springer-Verlag, 2000) ofrece un compendio completo de conceptos y términos importantes para la confiabilidad y la seguridad; Gartner (*Testing Safety-Related Software*, Springer-Verlag, 1999) ofrece una guía especializada para probar sistemas cuya seguridad sea crítica; Friedman y Voas (*Software Assesment: Reliability Safety and Testability*, Wiley, 1995) proveen modelos útiles para evaluar la confiabilidad y la seguridad. Ericson (*Hazard Analysis Techniques for System Safety*, Wiley, 2005) estudia el dominio cada vez más importante del análisis de los peligros.

En internet, hay una amplia variedad de fuentes de información sobre el aseguramiento de la calidad del software y otros temas relacionados. En el sitio web del libro, **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**, existe una lista actualizada de referencias existentes en la red mundial que son relevantes para el ACS.

CAPÍTULO

17

ESTRATEGIAS DE PRUEBA DE SOFTWARE

La estrategia de pruebas es un plan para implementar las pruebas correctamente.

CONCEPTOS CLAVE

depuración 404
grupo de prueba independiente 386
prueba alfa 400
prueba beta 400
prueba de clase 398
prueba de despliegue 403
prueba de integración 391
prueba de regresión 398
prueba de unidad 389
prueba de validación 399
prueba del sistema 401
revisión de la configuración 400
V&V 387

Una estrategia de prueba de software proporciona una guía que describe los pasos que deben realizarse como parte de la prueba, cuándo se planean y se llevan a cabo dichos pasos, y cuánto esfuerzo, tiempo y recursos se requerirán. Por tanto, cualquier estrategia de prueba debe incorporar la planificación de la prueba, el diseño de casos de prueba, la ejecución de la prueba y la recolección y evaluación de los resultados.

Una estrategia de prueba de software debe ser suficientemente flexible para promover un uso personalizado de la prueba. Al mismo tiempo, debe ser suficientemente rígida para alentar la planificación razonable y el seguimiento de la gestión conforme avanza el proyecto. Shooman [Sho83] analiza estos temas:

> En muchas formas, la prueba es un proceso de individualización, y el número de tipos diferentes de pruebas varía tanto como los diferentes acercamientos para su desarrollo. Durante muchos años, la única defensa contra los errores de programación fue el diseño cuidadoso y la inteligencia natural del programador. Ahora estamos en una era en la que modernas técnicas de diseño (y revisiones técnicas) ayudan a reducir el número de errores iniciales que son inherentes al código. De igual modo, diferentes métodos de prueba comienzan a agruparse en métodos y filosofías distintos.

Estos "enfoques y filosofías" a los que denomino *estrategias* son el tema que se presenta en este capítulo. En los capítulos 18, 19 y 20 se exponen los métodos y técnicas de prueba que permiten desarrollar la estrategia.

Las pruebas permiten descubrir errores.

UNA MIRADA RÁPIDA

¿Qué es? El software se prueba para descubrir errores que se cometieron de manera inadvertida conforme se diseñó y construyó. Pero, ¿cómo se realizan las pruebas? ¿Debe realizarse un plan formal para las mismas? ¿Debe probarse el programa completo, como un todo, o aplicar pruebas sólo sobre una pequeña parte de él? ¿Debe volverse a aplicar las pruebas que ya se realizaron mientras se agregan nuevos componentes a un sistema grande? ¿Cuándo debe

Las pruebas pasan errores por alto sino se planifican

de software.

¿Quién lo hace? El gerente de proyecto, los ingenieros de software y los especialistas en pruebas desarrollan una estrategia para probar el software.

¿Por qué es importante? Con frecuencia, la prueba requiere más esfuerzo que cualquiera otra acción de ingeniería del software. Si se realiza sin orden, se desperdicia tiempo, se emplea esfuerzo innecesario y, todavía peor, es posible que algunos errores pasen desapercibidos. Por tanto, parecería razonable establecer una estrategia sistemática para probar el software.

¿Cuáles son los pasos? La prueba comienza "por lo pequeño" y avanza "hacia lo grande". Es decir que las primeras etapas de prueba se enfocan sobre un solo componente o un pequeño grupo de componentes relacionados y se aplican pruebas para descubrir errores en los datos y en la lógica de procesamiento que se encapsularon en los componentes. Después de probar éstos, deben integrarse hasta que se construya el sistema completo. En este punto, se ejecuta una serie de pruebas de orden superior para descubrir errores en la satisfacción de los requerimientos del cliente. Conforme se descubren, los errores deben diagnosticarse y corregirse usando un proceso que se llama depuración.

¿Cuál es el producto final? Una *Especificación pruebas* documenta la forma en la que el equipo de software prepara la prueba al definir un plan que describe una estrategia global y un procedimiento con pasos de prueba específicos y los tipos de pruebas que se realizarán.

¿Cómo me aseguro de que lo hice bien? Al revisar la *Especificación pruebas* antes de realizar las pruebas, es posible valorar si están completos los casos de prueba y las tareas de la misma. Un plan de prueba y procedimientos efectivos conducirán a la construcción ordenada del software y al descubrimiento de errores en cada etapa del proceso de construcción.

17.1 UN ENFOQUE ESTRATÉGICO PARA LA PRUEBA DE SOFTWARE

La prueba es un conjunto de actividades que pueden planearse por adelantado y realizarse de manera sistemática. Por esta razón, durante el proceso de software, debe definirse una plantilla para la prueba del software: un conjunto de pasos que incluyen métodos de prueba y técnicas de diseño de casos de prueba específicos.

En la literatura sobre el tema, se han propuesto algunas estrategias de prueba de software. Todas proporcionan una plantilla para la prueba y tienen las siguientes características genéricas:

Pasos clave de cualquier estrategia de prueba

- Para realizar una prueba efectiva, debe realizar revisiones técnicas efectivas (capítulo 15). Al hacerlo, eliminará muchos errores antes de comenzar la prueba.
- La prueba comienza en los componentes y opera "hacia afuera", hacia la integración de todo el sistema de cómputo.
- Diferentes técnicas de prueba son adecuadas para distintos enfoques de ingeniería de software y en diferentes momentos en el tiempo.
- Las pruebas las realiza el desarrollador del software y (para proyectos grandes) un grupo de prueba independiente.
- Prueba y depuración son actividades diferentes, pero la depuración debe incluirse en cualquier estrategia de prueba.

WebRef

En **www.mtsu.edu/~storm** pueden encontrarse útiles recursos para la prueba de software.

Una estrategia para la prueba de software debe incluir pruebas de bajo nivel, que son necesarias para verificar que un pequeño segmento de código fuente se implementó correctamente, así como pruebas de alto nivel, que validan las principales funciones del sistema a partir de los requerimientos del cliente. Una estrategia debe proporcionar una guía para el profesional y un conjunto de guías para el jefe de proyecto. Puesto que los pasos de la estrategia de prueba ocurren cuando comienza a aumentar la presión por las fechas límite, el avance debe ser medible y los problemas deben salir a la superficie tan pronto como sea posible.

17.1.1 Verificación y validación

Que rol juegan las pruebas en el proceso de Verificación y Validación

"Probar es la parte inevitable de cualquier esfuerzo responsable por desarrollar un sistema de software".

William Howden

La prueba de software es un elemento de un tema más amplio que usualmente se conoce como verificación y validación (V&V). La *verificación* se refiere al conjunto de tareas que garantizan que el software implementa correctamente una función específica. La *validación* es un conjunto diferente de tareas que aseguran que el software que se construye sigue los requerimientos del cliente. Boehm [Boe81] afirma esto de esta forma:

> Verificación: "¿Construimos el producto correctamente?"
>
> Validación: "¿Construimos el producto correcto?"

La definición de V&V abarca muchas actividades de aseguramiento de calidad del software (capítulo 16).[1]

La verificación y la validación incluyen un amplio arreglo de actividades SQA: revisiones técnicas, auditorías de calidad y configuración, monitoreo de rendimiento, simulación, estudio de factibilidad, revisión de documentación, revisión de base de datos, análisis de algoritmos, pruebas de desarrollo, pruebas de usabilidad, pruebas de calificación, pruebas de aceptación y

1 Debe notarse que hay una fuerte divergencia de opinión acerca de qué tipos de pruebas constituyen la "validación". Algunas personas creen que *todas* las pruebas sirven para la verificación y que la validación se lleva a cabo cuando los requerimientos se revisan y aprueban, y, más tarde, por el usuario, cuando el sistema resulta operativo. Otras personas ven las pruebas de unidad y de integración (secciones 17.3.1 y 17.3.2) como verificación y las de orden superior (secciones 17.6 y 17.7) como validación.

pruebas de instalación. Aunque las pruebas juegan un papel extremadamente importante en V&V, también son necesarias muchas otras actividades.

CONSEJO

Es un error pensar que las pruebas son una "red de seguridad" que atrapará todos los errores que ocurran como producto de deficientes prácticas de ingeniería de software. No lo hará. Enfatice la calidad y la detección de errores a lo largo del proceso de software.

Las pruebas representan el último bastión desde donde puede valorarse la calidad y, de manera más pragmática, descubrirse errores. Pero las pruebas no deben verse como una red de seguridad. Como se dice: "no se puede probar la calidad. Si no está ahí antes de comenzar las pruebas, no estará cuando termine de probar". La calidad se incorpora en el software a lo largo de todo el proceso de ingeniería del software. La adecuada aplicación de métodos y herramientas, revisiones técnicas efectivas, y gestión y medición sólidas conducen a la calidad que se confirma durante las pruebas.

Miller [Mil77] relaciona la prueba del software con el aseguramiento de la calidad al afirmar que "la motivación subyacente de las pruebas de los programas es afirmar la claridad del software con métodos que puedan aplicarse de manera económica y efectiva a sistemas a gran y pequeña escala".

17.1.2 Organización de las pruebas del software

Cita:

"El optimismo es el riesgo ocupacional de la programación; la prueba es el tratamiento".

Kent Beck

En todo proyecto de software hay un conflicto inherente de intereses que ocurre conforme comienzan las pruebas. Hoy en día, a las personas que construyen el software se les pide probarlo. En sí, esto parece sencillo; después de todo, ¿quién conoce mejor el programa que sus desarrolladores? Por desgracia, estos mismos desarrolladores tienen mucho interés en demostrar que el programa está libre de errores, que funciona de acuerdo con los requerimientos del cliente y que se completará a tiempo y dentro del presupuesto. Cada uno de estos intereses tienen un efecto negativo sobre las pruebas más cuidadosas.

Desde un punto de vista psicológico, el análisis y diseño de software (junto con la codificación) son tareas constructivas. El ingeniero de software analiza, modela y luego crea un programa de computadora y su documentación. Como cualquier constructor, el ingeniero de software está orgulloso del edificio que construyó y ve con desconfianza a quien intente derrumbarlo. Cuando comienzan las pruebas, hay un sutil, pero definitivo, intento por "romper" lo que construyó el ingeniero de software. Desde el punto de vista del constructor, las pruebas pueden considerarse como (psicológicamente) destructivas. De modo que el constructor actuará con cuidado, y diseñará y ejecutará pruebas que demostrarán que el programa funciona, en lugar de descubrir errores. Desafortunadamente, los errores estarán presentes. Y si el ingeniero de software no los encuentra, ¡el cliente lo hará!

Con frecuencia, existen algunas malas interpretaciones que pueden inferirse de manera errónea a partir de la discusión anterior: 1) que el desarrollador de software no debe hacer pruebas en absoluto, 2) que el software debe "ponerse tras una pared" que lo separe de los extraños que lo probarán sin misericordia, 3) que quienes realicen las pruebas deben involucrarse con el proyecto sólo cuando los pasos de las pruebas estén por comenzar. Cada uno de estos enunciados es incorrecto.

El desarrollador de software siempre es responsable de probar las unidades individuales (componentes) del programa y de asegurarse de que cada una desempeña la función o muestra el comportamiento para el cual se diseñó. En muchos casos, el desarrollador también realiza pruebas de integración, una etapa en las pruebas que conduce a la construcción (y prueba) de la arquitectura completa del software. Sólo después de que la arquitectura de software está completa se involucra un grupo de prueba independiente (GPI).

PUNTO CLAVE

Un grupo de prueba independiente no tiene el "conflicto de intereses" que pueden experimentar los constructores del software.

El papel de un *grupo de prueba independiente* (GPI) es remover los problemas inherentes que están asociados con dejar al constructor probar lo que construyó. Las pruebas independientes remueven el conflicto de intereses que de otro modo puede estar presente. Después de todo, al personal del GPI se le paga por encontrar errores.

Sin embargo, el desarrollador no da el software al GPI y se retira. Él y el GPI trabajan de manera cercana a lo largo del proyecto de software para garantizar que se realizarán pruebas ex-

Cita:

"El primer error que comete la gente es creer que el equipo de prueba es responsable de asegurar la calidad."

Brian Marick

haustivas. Mientras se realizan éstas, el desarrollador debe estar disponible para corregir los errores que se descubran.

El GPI es parte del equipo de proyecto de desarrollo del software pues se involucra durante el análisis y el diseño, y sigue involucrado (mediante planificación y especificación de procedimientos de prueba) a lo largo de un proyecto grande. No obstante, en muchos casos, el GPI reporta a la organización de aseguramiento de calidad del software, y por tanto logra un grado de independencia que no puede existir si fuese parte de la organización de ingeniería del software.

17.1.3 Estrategia de prueba del software. Visión general

Relación entre el proceso de desarrollo y el proceso de pruebas

El proceso de software puede verse como la espiral que se ilustra en la figura 17.1. Inicialmente, la ingeniería de sistemas define el papel del software y conduce al análisis de los requerimientos del mismo, donde se establecen los criterios de dominio, función, comportamiento, desempeño, restricciones y validación de información para el software. Al avanzar hacia adentro a lo largo de la espiral, se llega al diseño y finalmente a la codificación. Para desarrollar software de computadoras, se avanza en espiral hacia adentro (contra las manecillas del reloj) a lo largo de una línea que reduce el nivel de abstracción en cada vuelta.

¿Cuál es la estrategia global para la prueba del software?

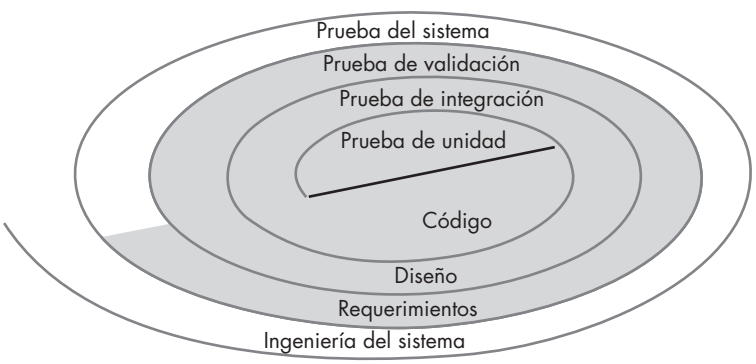
Una estrategia para probar el software también puede verse en el contexto de la espiral (figura 17.1). La *prueba de unidad* comienza en el vértice de la espiral y se concentra en cada unidad (por ejemplo, componente, clase o un objeto de contenido de una *webapp*) del software como se implementó en el código fuente. La prueba avanza al moverse hacia afuera a lo largo de la espiral, hacia la *prueba de integración*, donde el enfoque se centra en el diseño y la construcción de la arquitectura del software. Al dar otra vuelta hacia afuera de la espiral, se encuentra la *prueba de validación*, donde los requerimientos establecidos como parte de su modelado se validan confrontándose con el software que se construyó. Finalmente, se llega a la *prueba del sistema*, donde el software y otros elementos del sistema se prueban como un todo. Para probar el software de cómputo, se avanza en espiral hacia afuera en dirección de las manecillas del reloj a lo largo de líneas que ensanchan el alcance de las pruebas con cada vuelta.

WebRef

Quienes prueban software pueden encontrar recursos útiles en **www.SQAtester.com**

Al considerar el proceso desde un punto de vista procedural, las pruebas dentro del contexto de la ingeniería del software en realidad son una serie de cuatro pasos que se implementan de manera secuencial. Éstos se muestran en la figura 17.2. Inicialmente, las pruebas se enfocan en cada componente de manera individual, lo que garantiza que funcionan adecuadamente como unidad. De ahí el nombre de *prueba de unidad*. Esta prueba utiliza mucho de las técnicas de prueba que ejercitan rutas específicas en una estructura de control de componentes para asegurar una cobertura completa y la máxima detección de errores. A continuación, los componentes deben ensamblarse o integrarse para formar el paquete de software completo. La *prueba de integración* aborda los conflictos asociados con los problemas duales de verificación y construcción de programas. Durante la integración, se usan más las técnicas de diseño de casos de

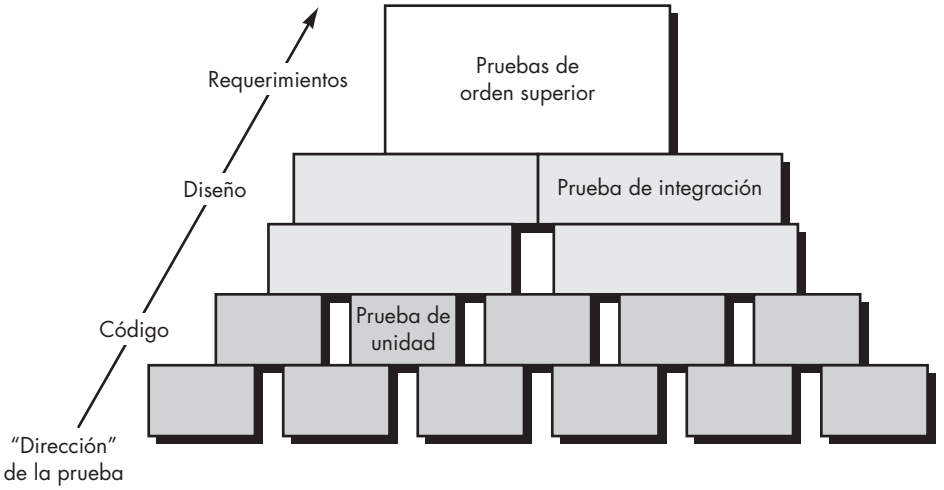
FIGURA 17.1
Estrategia de pruebas



Para cada fase de desarrollo existe su fase en el proceso de pruebas.
Esto es lo mismo que el modelo en V visto en unidades enteriores.

FIGURA 17.2
Pasos de la prueba del software

El desarrollo va desde lo general hasta lo especifico mientras que las pruebas van al revez.



prueba que se enfocan en entradas y salidas, aunque también pueden usarse técnicas que ejercitan rutas de programa específicas para asegurar la cobertura de las principales rutas de control. Después de integrar (construir) el software, se realiza una serie de *pruebas de orden superior*. Deben evaluarse criterios de validación (establecidos durante el análisis de requerimientos). La *prueba de validación* proporciona la garantía final de que el software cumple con todos los requerimientos informativos, funcionales, de comportamiento y de rendimiento.

El último paso de la prueba de orden superior cae fuera de las fronteras de la ingeniería de software y en el contexto más amplio de la ingeniería de sistemas de cómputo. El software, una vez validado, debe combinarse con otros elementos del sistema (por ejemplo, hardware, personal, bases de datos). La *prueba del sistema* verifica que todos los elementos se mezclan de manera adecuada y que se logra el funcionamiento/rendimiento global del sistema.

CASASEGURA



Preparación para la prueba

La escena: Oficina de Doug Miller, mientras continúa el diseño en el nivel de componentes y comienza la construcción de algunos de ellos.

Participantes: Doug Miller, jefe de ingeniería de software; Vinod, Jamie, Ed y Shakira, miembros del equipo de ingeniería de software de CasaSegura.

La conversación:

Doug: Me parece que no hemos dedicado suficiente tiempo para hablar de las pruebas.

Vinod: Cierto, pero todos hemos estado un poco ocupados. Y además hemos pensado en ello... en realidad, hemos hecho más que pensar.

Doug (sonríe): Lo sé... tenemos exceso de trabajo, pero todavía debemos pensar en las cosas importantes.

Shakira: Me gusta la idea de diseñar pruebas de unidad antes de comenzar a codificar cualquiera de mis componentes, así que eso es lo que he intentado hacer. Tengo un archivo de pruebas bastante grande para aplicar cuando codifique mis componentes por completo.

Doug: Ése es un concepto de programación extrema [proceso de desarrollo de software ágil, véase el capítulo 3], ¿o no?

Ed: Lo es. Aun cuando no usamos programación extrema *per se*, decidimos que sería buena idea diseñar pruebas de unidad antes de construir el componente; el diseño nos dará la información que necesitamos.

Jamie: Yo he hecho lo mismo.

Vinod: Y yo tomé el papel de integrador, así que cada vez que uno de los muchachos me pase un componente, lo integraré y correré una serie de pruebas de regresión sobre el programa parcialmente integrado. He trabajado para diseñar un conjunto de pruebas adecuadas para cada función en el sistema.

Doug (a Vinod): ¿Con qué frecuencia corres las pruebas?

Vinod: Todos los días... hasta que el sistema esté integrado... bueno, quiero decir hasta que esté integrado el incremento de software que planeamos entregar.

Doug: ¡Muchachos, van adelante de mí!

Vinod (ríe): La anticipación lo es todo en el negocio del software, jefe.

El uso de la estadistica para saber cuando terminar la fase de pruebas.

17.1.4 Criterios para completar las pruebas

¿Cuándo terminan las pruebas?

Cada vez que se analiza la prueba del software, surge una pregunta clásica: "¿cuándo terminan las pruebas?, ¿cómo se sabe que se ha probado lo suficiente?". Lamentablemente, no hay una respuesta definitiva a esta pregunta, pero existen algunas respuestas pragmáticas e intentos tempranos a manera de guía empírica.

Una respuesta a la pregunta es: "nunca se termina de probar; la carga simplemente pasa de usted (el ingeniero de software) al usuario final". Cada vez que el usuario ejecuta un programa de cómputo, el programa se pone a prueba. Este instructivo hecho subraya la importancia de otras actividades a fin de garantizar la calidad del software. Otra respuesta (un tanto cínica, mas no obstante precisa) es: "las pruebas terminan cuando se agota el tiempo o el dinero".

Aunque algunos profesionales usarían estas respuestas, se necesitan criterios más rigurosos para determinar cuándo se han realizado suficientes pruebas. El enfoque de *ingeniería de software de salas limpias* (capítulo 21) sugiere el uso de técnicas estadísticas [Kel00] que ejecutan una serie de pruebas derivadas de una muestra estadística de todas las posibles ejecuciones de programa por parte de todos los usuarios de una población objetivo. Otros (por ejemplo, [Sin99]) abogan por el uso del modelado estadístico y la teoría de confiabilidad del software para predecir cuándo están completas las pruebas.

WebRef

Un glosario amplio de términos de pruebas puede encontrarse en el sitio **www.testingstandards.co.uk/living_glossary.htm**

Al coleccionar estadísticas durante las pruebas del software y usar los modelos existentes de confiabilidad del mismo, es posible desarrollar lineamientos significativos para responder la pregunta: "¿cuándo terminan las pruebas?". Hay poco debate acerca de que todavía queda mucho trabajo por hacer antes de poder establecer reglas cuantitativas para las pruebas, pero los acercamientos empíricos que existen en la actualidad son considerablemente mejores que la intuición pura.

17.2 Aspectos estratégicos

Más adelante en este capítulo, se presenta una estrategia sistemática para probar el software. Pero incluso la mejor estrategia fracasará si no se aborda una serie de aspectos decisivos. Tom Gilb [Gil95] arguye que una estrategia de software triunfará cuando quienes prueban el software:

¿Qué lineamientos conducen a una exitosa estrategia de prueba del software?

Especifican los requerimientos del producto en forma cuantificable mucho antes de comenzar con las pruebas. Aunque el objetivo predominante de una prueba es encontrar errores, una buena estrategia de prueba también valora otras características de la calidad, como la portabilidad, el mantenimiento y la facilidad de uso (capítulo 14). Esto debe especificarse en una forma medible, de modo que los resultados de las pruebas no sean ambiguos.

Establecen de manera explícita los objetivos de las pruebas. Los objetivos específicos de las pruebas deben enunciarse en términos medibles. Por ejemplo, la efectividad de las pruebas, su cobertura, el tiempo medio antes de aparecer una falla, el costo por descubrir y corregir defectos, la densidad de defectos restantes o la frecuencia de ocurrencia, y las horas de trabajo de prueba deben enunciarse dentro del plan de la prueba.

WebRef

Una excelente lista de recursos de prueba puede encontrarse en el sitio **www.io.com/~wazmo/qa**

Entienden a los usuarios del software y desarrollan un perfil para cada categoría de usuario. Los casos de uso que describen el escenario de interacción para cada clase de usuario pueden reducir el esfuerzo de prueba global al enfocar las pruebas en el uso real del producto.

Desarrollan un plan de prueba que enfatice "pruebas de ciclo rápido". Gilb [Gil95] recomienda que un equipo de software "aprenda a probar en ciclos rápidos (2 por ciento del esfuerzo del proyecto) de cliente-utilidad al menos la 'comprobabilidad' en campo, los incrementos de funcionalidad y/o la mejora de la calidad". La retroalimentación generada a partir de estas pruebas de ciclo rápido puede usarse para controlar niveles de calidad y las correspondientes estrategias de prueba.

Construyen software "robusto" que esté diseñado para probarse a sí mismo. El software debe diseñarse en forma que use técnicas antierrores (sección 17.3.1), es decir, el software debe poder diagnosticar ciertas clases de errores. Además, el diseño debe incluir pruebas automatizadas y pruebas de regresión.

Usan revisiones técnicas efectivas como filtro previo a las pruebas. Las revisiones técnicas (capítulo 15) pueden ser tan efectivas como probar para descubrir errores. Por esta razón, las revisiones pueden reducir la cantidad del esfuerzo de pruebas que se requieren para producir software de alta calidad.

Realizan revisiones técnicas para valorar la estrategia de prueba y los casos de prueba. Las revisiones de prueba pueden descubrir inconsistencias, omisiones y errores evidentes en el abordaje de las pruebas. Esto ahorra tiempo y también mejora la calidad del producto.

Desarrollan un enfoque de mejora continuo para el proceso de prueba. La estrategia de pruebas debe medirse. Las métricas recopiladas durante las pruebas deben usarse como parte de un enfoque de control de proceso estadístico para la prueba del software.

Cita:

"Probar sólo los **requerimientos** del usuario final es como inspeccionar un edificio con base en el trabajo realizado por el decorador de interiores a costa de cimientos, vigas y plomería."

Boris Beizer

17.3 ESTRATEGIAS DE PRUEBA PARA SOFTWARE CONVENCIONAL[2]

Existen muchas estrategias que pueden usarse para probar el software. En un extremo, puede esperarse hasta que el sistema esté completamente construido y luego realizar las pruebas sobre el sistema total, con la esperanza de encontrar errores. Este enfoque, aunque atractivo, simplemente no funciona. Dará como resultado software defectuoso que desilusionará a todos los participantes. En el otro extremo, podrían realizarse pruebas diariamente, siempre que se construya alguna parte del sistema. Este enfoque, aunque menos atractivo para muchos, puede ser muy efectivo. Por desgracia, algunos desarrolladores de software son reacios a usarlo. ¿Qué hacer?

Una estrategia de prueba que eligen la mayoría de los equipos de software se coloca entre los dos extremos. Toma una visión incremental de las pruebas, comenzando con la de unidades de programa individuales, avanza hacia pruebas diseñadas para facilitar la integración de las unidades y culmina con pruebas que ejercitan el sistema construido. Cada una de estas clases de pruebas se describe en las secciones que siguen.

17.3.1 Prueba de unidad

La *prueba de unidad* enfoca los esfuerzos de verificación en la unidad más pequeña del diseño de software: el componente o módulo de software. Al usar la descripción del diseño de componente como guía, las rutas de control importantes se prueban para descubrir errores dentro de la frontera del módulo. La relativa complejidad de las pruebas y los errores que descubren están limitados por el ámbito restringido que se establece para la prueba de unidad. Las pruebas de unidad se enfocan en la lógica de procesamiento interno y de las estructuras de datos dentro de las fronteras de un componente. Este tipo de pruebas puede realizarse en paralelo para múltiples componentes.

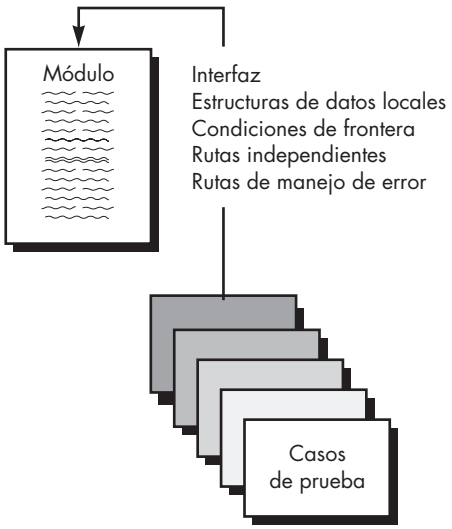
CONSEJO

No es mala idea diseñar casos de prueba de unidad antes de desarrollar el código para un componente. Eso ayuda a garantizar que se desarrollará un código que pasará las pruebas.

Consideraciones de las pruebas de unidad. Las pruebas de unidad se ilustran de manera esquemática en la figura 17.3. La interfaz del módulo se prueba para garantizar que la información fluya de manera adecuada hacia y desde . Las estructuras de datos locales se examinan para asegurar que los datos almacenados temporal-

2 A lo largo de este libro, se usan los términos *software convencional* o *software tradicional* para referirse a arquitecturas de software jerárquica común, o de "llamar y regresar", que con frecuencia se encuentran en una variedad de dominios de aplicación. Las arquitecturas de software tradicional *no* son orientadas a objetos y no abarcan *webapps*.

FIGURA 17.3
Prueba de unidad



mente mantienen su integridad durante todos los pasos en la ejecución de un algoritmo. Todas las rutas independientes a través de la estructura de control se ejercitan para asegurar que todos los estatutos en un módulo se ejecuten al menos una vez. Las condiciones de frontera se prueban para asegurar que el módulo opera adecuadamente en las fronteras establecidas para limitar o restringir el procesamiento. Y, finalmente, se ponen a prueba todas las rutas para el manejo de errores.

El flujo de datos a través de la interfaz de un componente se prueba antes de iniciar cualquiera otra prueba. Si los datos no entran y salen de manera adecuada, todas las demás pruebas son irrelevantes. Además, deben ejercitarse las estructuras de datos locales y averiguarse (si es posible) el impacto local sobre los datos globales durante las pruebas de unidad.

? ¿Qué errores se encuentran comúnmente durante las pruebas de unidad?

La prueba selectiva de las rutas de ejecución es una tarea esencial durante la prueba de unidad. Los casos de prueba deben diseñarse para descubrir errores debidos a cálculos erróneos, comparaciones incorrectas o flujo de control inadecuado.

Las pruebas de frontera son una de las tareas de prueba de unidad más importantes. Con frecuencia, el software falla en sus fronteras. Es decir: con frecuencia los errores ocurren cuando se procesa el *en*ésimo elemento de un arreglo *ene*-dimensional, cuando se invoca la *en*ésima repetición de un bucle con *n* pasadas, cuando se encuentra el valor máximo o mínimo permisible. Es muy probable que los casos de prueba que ejercitan la estructura de datos, el flujo de control y los valores de datos justo abajo y arriba de máximos y mínimos descubran errores.

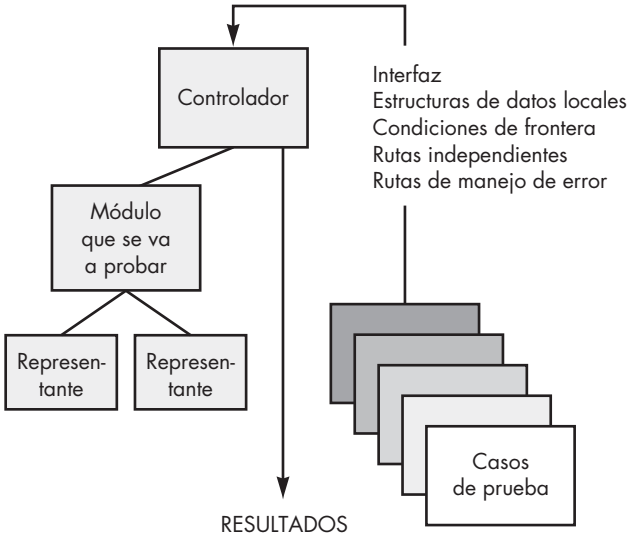
WebRef
Puede encontrar información útil acerca de una gran variedad de artículos y recursos para "prueba ágil" en **testing.com/agile**

Un buen diseño anticipa las condiciones de error y establece rutas de manejo de errores para enrutar o terminar limpiamente el procesamiento cuando ocurre un error. Yourdon [You75] llama a este enfoque *antierrores*. Desafortunadamente, hay una tendencia a incorporar el manejo de errores en el software y luego nunca probarlo. Una historia verídica puede servir como ilustración:

> Un sistema de diseño asistido por computadora se desarrolló bajo contrato. En un módulo de procesamiento de transacción, un bromista colocó el siguiente mensaje de manejo de error después de una serie de pruebas condicionales que invocaban varias ramas de flujo de control: ¡ERROR! NO HAY FORMA DE QUE PUEDA LLEGAR AQUÍ. ¡Este "mensaje de error" lo descubrió un cliente durante el entrenamiento para usuarios!

Entre los potenciales errores que deben ponerse a prueba cuando se evalúa el manejo de errores están: 1) la descripción de error ininteligible, 2) el error indicado no corresponde con el error que se encuentra, 3) la condición del error causa la intervención del sistema antes de manejar el

FIGURA 17.4
Entorno de prueba de unidad





Asegúrese de diseñar pruebas para ejecutar cada ruta de manejo de error. Si no lo hace, la ruta puede fallar cuando se invoque, lo que agrava una situación de por sí peligrosa.

error, 4) el procesamiento excepción-condición es incorrecto y 5) la descripción del error no proporciona suficiente información para auxiliar en la localización de la causa del error.

Procedimientos de prueba de unidad. Las pruebas de unidad por lo general se consideran como adjuntas al paso de codificación. El diseño de las pruebas de unidad puede ocurrir antes de comenzar la codificación o después de generar el código fuente. La revisión de la información del diseño proporciona una guía para establecer casos de prueba que es probable que descubran errores en cada una de las categorías analizadas anteriormente. Cada caso de prueba debe acoplarse con un conjunto de resultados esperados.

Es como la anlogia de la caja negra.

El controlador envia datos, el modulo a probar procesa esos datos y el representante recibe el resultado de ese procesamiento.

Puesto que un componente no es un programa independiente, con frecuencia debe desarrollarse software controlador y/o de resguardo para cada prueba de unidad. En la figura 17.4 se ilustran los entornos de prueba de unidad. En la mayoría de las aplicaciones, un *controlador* no es más que un "programa principal" que acepta datos de caso de prueba, pasa tales datos al componente (que va a ponerse a prueba) e imprime resultados relevantes. Los *representantes* (en inglés *stubs)* sirven para sustituir módulos que están subordinados al (invocados por el) componente que se va a probar. Un representante o "subprograma tonto" usa la interfaz de módulo subordinado, puede realizar mínima manipulación de datos, imprimir verificación de entradas y regresar el control al módulo sobre el que se realiza la prueba.



Existen algunas situaciones donde no se tienen los recursos para realizar una prueba de unidad amplia. Seleccione los módulos cruciales o complejos y aplique sólo en ellos las pruebas de unidad.

Los controladores y representantes añaden una "sobrecarga" a las pruebas. Es decir: ambos son software que debe escribirse (el diseño formal usualmente no se aplica), pero que no se entrega con el producto de software final. Si los controladores y representantes se mantienen simples, la sobrecarga real es relativamente baja. Por desgracia, muchos componentes no pueden tener prueba de unidad adecuada con un software de sobrecarga simple. En tales casos, la prueba completa puede posponerse hasta el paso de prueba de integración (donde también se usan controladores o representantes).

Las pruebas de unidad se simplifican cuando se diseña un componente con alta cohesión. Cuando un componente aborda una sola función, el número de casos de prueba se reduce y los errores pueden predecirse y descubrirse con mayor facilidad.

17.3.2 Pruebas de integración

Un neófito en el mundo del software podrá plantear una pregunta aparentemente legítima una vez que todos los módulos se hayan probado de manera individual: "si todos ellos funcionan

individualmente, ¿por qué dudan que funcionarán cuando se junten todos?". Desde luego, el problema es "juntarlos todos": conectarlos. Los datos pueden perderse a través de una interfaz; un componente puede tener un inadvertido efecto adverso sobre otro; las subfunciones, cuando se combinan, pueden no producir la función principal deseada; la imprecisión aceptable individualmente puede magnificarse a niveles inaceptables; las estructuras de datos globales pueden presentar problemas. Lamentablemente, la lista sigue y sigue.

Las pruebas de integración son una técnica sistemática para construir la arquitectura del software mientras se llevan a cabo pruebas para descubrir errores asociados con la interfaz. El objetivo es tomar los componentes probados de manera individual y construir una estructura de programa que se haya dictado por diseño.

CONSEJO

Tomar el enfoque de "big bang" para la integración es una estrategia perezosa condenada al fracaso. Integre de manera incremental y pruebe conforme avance.

Con frecuencia existe una tendencia a intentar la integración no incremental, es decir, a construir el programa usando un enfoque de *big bang*. Todos los componentes se combinan por adelantado. Todo el programa se prueba como un todo. ¡Y usualmente resulta el caos! Se descubre un conjunto de errores. La corrección se dificulta pues el aislamiento de las causas se complica por la vasta extensión de todo el programa. Una vez corregidos estos errores, otros nuevos aparecen y el proceso continúa en un bucle aparentemente interminable.

La integración incremental es la antítesis del enfoque *big bang*. El programa se construye y prueba en pequeños incrementos, donde los errores son más fáciles de aislar y corregir; las interfaces tienen más posibilidades de probarse por completo; y puede aplicarse un enfoque de prueba sistemático. En los siguientes párrafos se exponen algunas estrategias diferentes de integración incremental.

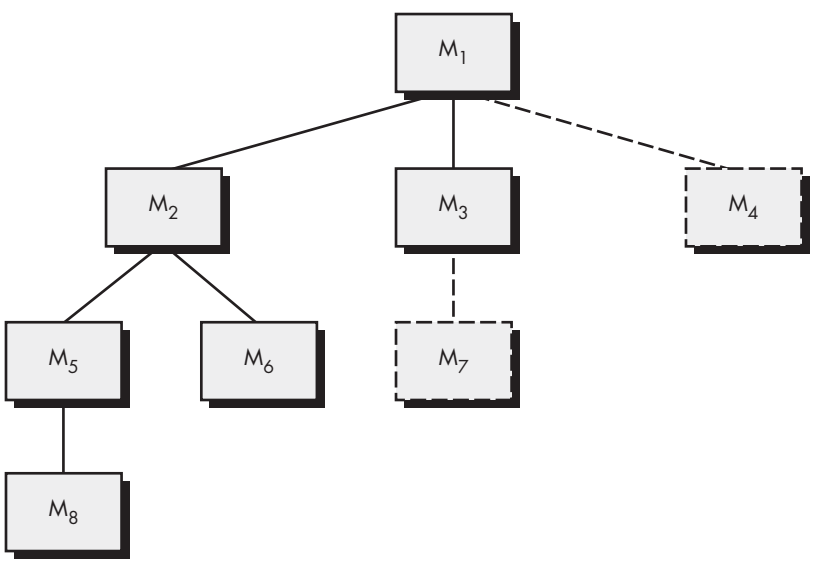
CONSEJO

Un modulo subordinado es un modulo hijo (el que esta abajo de otro modulo)

Integración descendente. La *prueba de integración descendente* es un enfoque incremental a la construcción de la arquitectura de software. Los módulos se integran al moverse hacia abajo a través de la jerarquía de control, comenzando con el módulo de control principal (programa principal). Los módulos subordinados al módulo de control principal se incorporan en la estructura en una forma de primero en profundidad o primero en anchura.

Con referencia a la figura 17.5, la *integración primero en profundidad* integra todos los componentes sobre una ruta de control mayor de la estructura del programa. La selección de una ruta mayor es un tanto arbitraria y depende de las características específicas de la aplicación. Por ejemplo, al seleccionar la ruta de la izquierda, los componentes M_1, M_2, M_5 se integrarían primero. A continuación, M_8 o (si es necesario para el adecuado funcionamiento de M_2) se inte-

FIGURA 17.5
Integración descendente



primero en profundidad llega al nivel final de una cadena de nodos y luego al nivel final de otra cadena de nodos y así
M2 y sus decendiente son unacadenadenodos.
M3 y sus decendientes son otras cadena de nodos.
M4 y sus decendientes sonotracadenadenodosyfin.

graría M_6. Luego se construyen las rutas de control central y derecha. La *integración primero en anchura* incorpora todos los componentes directamente subordinados en cada nivel, y se mueve horizontalmente a través de la estructura. De la figura, los componentes M_2, M_3 y M_4 se integrarían primero. Le sigue el siguiente nivel de control, M_5, M_6, etc. El proceso de integración se realiza en una serie de cinco pasos:

¿Cuáles son los pasos para la integración descendente?

1. El módulo de control principal se usa como un controlador de prueba y los representantes (*stubs*) se sustituyen con todos los componentes directamente subordinados al módulo de control principal.
2. Dependiendo del enfoque de integración seleccionado (es decir, primero en profundidad o anchura), los representantes subordinados se sustituyen uno a la vez con componentes reales.
3. Las pruebas se llevan a cabo conforme se integra cada componente.
4. Al completar cada conjunto de pruebas, otro representante se sustituye con el componente real.
5. Las pruebas de regresión (que se analizan más adelante en esta sección) pueden realizarse para asegurar que no se introdujeron nuevos errores.

El proceso continúa desde el paso 2 hasta que se construye todo la estructura del programa.

La estrategia de integración descendente verifica los principales puntos de control o de decisión al principio en el proceso de prueba. En una estructura de programa "bien factorizada", la toma de decisiones ocurre en niveles superiores en la jerarquía y, por tanto, se encuentra primero. Si existen grandes problemas de control, el reconocimiento temprano es esencial. Si se selecciona la integración primero en profundidad, es posible implementar y demostrar un funcionamiento completo del software. La demostración temprana de la capacidad funcional es un constructor de confianza para todos los participantes.

¿Qué problemas pueden encontrarse cuando se elige la integración descendente?

Pareciera que la estrategia descendente no tiene complicaciones, pero, en la práctica, pueden surgir problemas logísticos. El más común de éstos ocurre cuando se requiere procesamiento en niveles bajos en la jerarquía a fin de probar de manera adecuada los niveles superiores. Los representantes (*stubs*) sustituyen los módulos de bajo nivel al comienzo de la prueba descendente; por tanto, ningún dato significativo puede fluir hacia arriba en la estructura del programa. A la persona que realiza la prueba le quedan tres opciones: 1) demorar muchas pruebas hasta que los representantes se sustituyan con módulos reales, 2) desarrollar resguardos que realicen funciones limitadas que simulen al módulo real o 3) integrar el software desde el fondo de la jerarquía y hacia arriba.

El primer enfoque (demorar las pruebas hasta que los representantes se sustituyan con módulos reales) puede hacerle perder algo de control sobre la correspondencia entre pruebas específicas y la incorporación de módulos específicos. Esto puede conducir a dificultades para determinar la causa de los errores y tiende a violar la naturaleza enormemente restrictiva del enfoque descendente. El segundo enfoque vale la pena, pero puede conducir a una sobrecarga significativa conforme los representantes se vuelven cada vez más complejos. El tercero, llamado integración ascendente, se analiza en los siguientes párrafos.

Integración ascendente. La *prueba de integración ascendente*, como su nombre implica, comienza la construcción y la prueba con *módulos atómicos* (es decir, componentes en los niveles inferiores dentro de la estructura del programa). Puesto que los componentes se integran de abajo hacia arriba, la funcionalidad que proporcionan los componentes subordinados en determinado nivel siempre está disponible y se elimina la necesidad de representantes (*stubs*). Una estrategia de integración ascendente puede implementarse con los siguientes pasos:

¿Cuáles son los pasos para la integración ascendente?

1. Los componentes en el nivel inferior se combinan en grupos (en ocasiones llamados *construcciones* o *builds)* que realizan una subfunción de software específica.
2. Se escribe un *controlador* (un programa de control para pruebas) a fin de coordinar la entrada y salida de casos de prueba.
3. Se prueba el grupo.
4. Los controladores se remueven y los grupos se combinan moviéndolos hacia arriba en la estructura del programa.

PUNTO CLAVE

La integración ascendente elimina la necesidad de **representantes (*stubs*)** complejos.

La integración sigue el patrón que se ilustra en la figura 17.6. Los componentes se combinan para formar los grupos 1, 2 y 3. Cada uno de ellos se prueba usando un controlador (que se muestra como un bloque rayado). Los componentes en los grupos 1 y 2 se subordinan a M_a. Los controladores D_1 y D_2 se remueven y los grupos se ponen en interfaz directamente con M_a. De igual modo, el controlador D_3 para el grupo 3 se remueve antes de la integración con el módulo M_b. Tanto M_a como M_b al final se integrarán con el componente M_c, y así sucesivamente.

Conforme la integración avanza hacia arriba, se reduce la necesidad de controladores de prueba separados. De hecho, si los dos niveles superiores del programa se integran de manera descendente, el número de controladores puede reducirse de manera sustancial y la integración de grupos se simplifica enormemente.

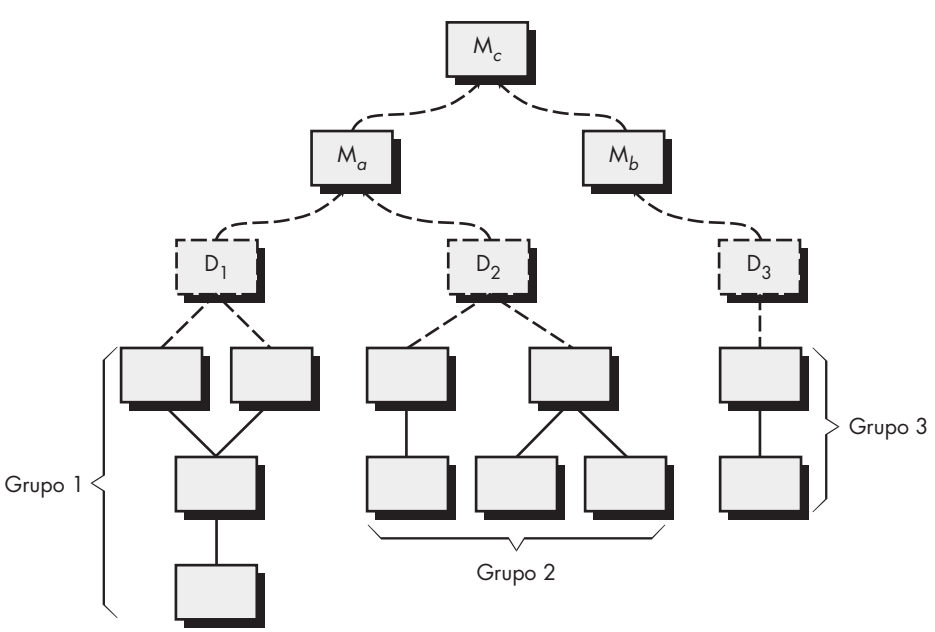
CONSEJO

La prueba de regresión es una importante estrategia para reducir "efectos colaterales". Corra pruebas de regresión cada vez que se realiza un cambio importante al software (incluida la integración de nuevos componentes).

Prueba de regresión. Cada vez que se agrega un nuevo módulo como parte de las pruebas de integración, el software cambia. Se establecen nuevas rutas de flujo de datos, ocurren nuevas operaciones de entrada/salida y se invoca nueva lógica de control. Dichos cambios pueden causar problemas con las funciones que anteriormente trabajaban sin fallas. En el contexto de una estrategia de prueba de integración, la *prueba de regresión* es la nueva ejecución de algún subconjunto de pruebas que ya se realizaron a fin de asegurar que los cambios no propagaron efectos colaterales no deseados.

En un contexto más amplio, las pruebas exitosas (de cualquier tipo) dan como resultado el descubrimiento de errores, y los errores deben corregirse. Siempre que se corrige el software, cambia algún aspecto de la configuración del software (el programa, su documentación o los datos que sustenta). Las pruebas de regresión ayudan a garantizar que los cambios (debidos

FIGURA 17.6
Integración ascendente



a pruebas o por otras razones) no introducen comportamiento no planeado o errores adicionales.

Las pruebas de regresión se pueden realizar manualmente, al volver a ejecutar un subconjunto de todos los casos de prueba o usando herramientas de captura/reproducción automatizadas. Las *herramientas de captura/reproducción* permiten al ingeniero de software capturar casos de prueba y resultados para una posterior reproducción y comparación. La *suite de prueba de regresión* (el subconjunto de pruebas que se va a ejecutar) contiene tres clases diferentes de casos de prueba:

- Una muestra representativa de pruebas que ejercitará todas las funciones de software.
- Pruebas adicionales que se enfocan en las funciones del software que probablemente resulten afectadas por el cambio.
- Pruebas que se enfocan en los componentes del software que cambiaron.

Conforme avanza la prueba de integración, el número de pruebas de regresión puede volverse muy grande. Por tanto, la suite de pruebas de regresión debe diseñarse para incluir solamente aquellas que aborden una o más clases de errores en cada una de las funciones del programa principal. Es impráctico e ineficiente volver a ejecutar toda prueba para cada función del programa cada vez que ocurre un cambio.

Prueba de humo. La *prueba de humo* es un enfoque de prueba de integración que se usa cuando se desarrolla software de producto. Se diseña como un mecanismo de ritmo para proyectos críticos en el tiempo, lo que permite al equipo del software valorar el proyecto de manera frecuente. En esencia, el enfoque de prueba de humo abarca las siguientes actividades:

PUNTO CLAVE

La prueba de humo puede caracterizarse como una estrategia de integración constante. El software se reconstruye (con el agregado de nuevos componentes) y se prueba cada día.

La construcción es el Inegraci ón de todos los modulos

1. Los componentes de software traducidos en código se integran en una *construcción*. Una construcción incluye todos los archivos de datos, bibliotecas, módulos reutilizables y componentes sometidos a ingeniería que se requieren para implementar una o más funciones del producto.
2. Se diseña una serie de pruebas para exponer los errores que evitarán a la construcción realizar adecuadamente su función. La intención debe ser descubrir errores "paralizantes" que tengan la mayor probabilidad de retrasar el proyecto.
3. La construcción se integra con otras construcciones, y todo el producto (en su forma actual) se somete a prueba de humo diariamente. El enfoque de integración puede ser descendente o ascendente.

Cita:

"Trate a la construcción diaria como al latido del proyecto. Si no hay latido, el proyecto está muerto."

Jim McCarthy

La frecuencia diaria de las pruebas de todo el producto puede sorprender a algunos lectores. Sin embargo, las pruebas constantes brindan, tanto a gerentes como a profesionales, una valoración realista del progreso de la prueba de integración. McConnell [McC96] describe la prueba de humo de la forma siguiente:

> La prueba de humo debe ejercitar todo el sistema de extremo a extremo. No tiene que ser exhaustiva, pero debe poder exponer los problemas principales. La prueba de humo debe ser suficientemente profunda para que, si la construcción pasa, pueda suponer que es suficientemente estable para probarse con mayor profundidad.

La prueba de humo proporciona algunos beneficios cuando se aplica sobre proyectos de software complejos y cruciales en el tiempo:

? ¿Qué beneficios pueden derivarse de las pruebas de humo?

- *Se minimiza el riesgo de integración*. Puesto que las pruebas de humo se realizan diariamente, las incompatibilidades y otros errores paralizantes pueden descubrirse tempranamente, lo que reduce la probabilidad de impacto severo sobre el calendario cuando se descubren errores.

- *La calidad del producto final mejora.* Es probable que la prueba de humo descubra errores funcionales así como errores de diseño arquitectónico o en el componente debido a que el enfoque está orientado a la construcción (integración). Si tales errores se corrigen temprano, se tendrá una mejor calidad del producto.
- *El diagnóstico y la corrección de errores se simplifican.* Como todo enfoque de prueba de integración, es probable que los errores descubiertos durante la prueba de humo se asocien con "nuevos incrementos de software"; es decir, el software que se acaba de agregar a la(s) construcción(es) es causa probable de un error recientemente descubierto.
- *El progreso es más fácil de valorar.* Con cada día que transcurre, más software se integra y se demuestra que funciona. Esto incrementa la moral del equipo y brinda a los gerentes un buen indicio de que se está progresando.

WebRef

En **www.qalinks.com** pueden encontrarse enlaces a comentarios acerca de estrategias de pruebas.

Opciones estratégicas. Ha habido mucha discusión (por ejemplo, [Bei84]) acerca de las relativas ventajas y desventajas de las pruebas de integración descendente en comparación con las ascendentes. En general, las ventajas de una estrategia tienden a ser desventajas para la otra. La principal desventaja del enfoque descendente es la necesidad de representantes y las dificultades de prueba que pueden asociarse con ellos. Los problemas asociados con los representantes pueden compensarse con la ventaja de probar tempranamente las principales funciones de control. La principal desventaja de la integración ascendente es que "el programa como entidad no existe hasta que se agrega el último módulo" [Mye79]. Este inconveniente se atempera con la mayor facilidad en el diseño de casos de prueba y la falta de representantes.

La selección de una estrategia de integración depende de las características del software y, en ocasiones, del calendario del proyecto. En general, un enfoque combinado (a veces llamado *prueba sándwich*), que usa pruebas descendentes para niveles superiores de la estructura del programa acopladas con pruebas ascendentes para niveles subordinados, puede ser el mejor arreglo.

? ¿Qué es un "módulo crítico" y por qué debe identificársele?

Conforme se realiza la integración, quien efectúa la prueba debe identificar los módulos críticos. Un *módulo crítico* tiene una o más de las siguientes características: 1) aborda muchos requerimientos de software, 2) tiene un alto nivel de control (reside relativamente alto en la estructura del programa), 3) es complejo o proclive al error o 4) tiene requerimientos de rendimiento definidos. Los módulos críticos deben probarse tan pronto como sea posible. Además, las pruebas de regresión deben enfocarse en la función del módulo crítico.

Productos de trabajo de las pruebas de integración. Un plan global para integración del software y una descripción de las pruebas específicas se documentan en una *Especificación de pruebas.* Este producto de trabajo incorpora un plan de prueba y un procedimiento de prueba, y se vuelve parte de la configuración del software. La prueba se divide en fases y construcciones que abordan características del software funcionales y de comportamiento específicas. Por ejemplo, la prueba de integración para el sistema de seguridad *CasaSegura* puede dividirse en las siguientes fases de prueba:

- *Interacción con el usuario* (entrada y salida de comandos, representación de despliegue, procesamiento y representación de errores)
- *Procesamiento de sensores* (adquisición de salida de sensor, determinación de condiciones del sensor, acciones requeridas como consecuencia de las condiciones)
- *Funciones de comunicación* (capacidad para comunicarse con la estación de monitoreo central)
- *Procesamiento de alarma* (pruebas de acciones del software que ocurren cuando se encuentra una alarma)

Cada una de estas fases de la prueba de integración delinea una amplia categoría funcional dentro del software y por lo general puede relacionarse con un dominio específico dentro de la arquitectura del software. Por tanto, las construcciones de programas (grupos de módulos) se crean para corresponder a cada fase. Los siguientes criterios y pruebas correspondientes se aplican a todas las fases de prueba:

¿Qué criterios deben usarse para diseñar pruebas de integración?

Integridad de interfaz. Las interfaces internas y externas se prueban conforme cada módulo (o grupo) se incorpora en la estructura.

Validez funcional. Se realizan pruebas diseñadas para descubrir errores funcionales ocultos.

Contenido de la información. Se realizan pruebas diseñadas para descubrir errores ocultos asociados con las estructuras de datos locales o globales.

Rendimiento. Se realizan pruebas diseñadas para verificar los límites del rendimiento establecidos durante el diseño del software.

Como parte del plan de prueba, también se discute un calendario para la integración, el desarrollo de software de sobrecarga del sistema y temas relacionados. Se establecen las fechas de inicio y fin de cada fase y se definen "ventanas disponibles" para módulos de prueba de unidad. Una breve descripción del software de sobrecarga (representantes y controladores) se concentra en las características que pueden requerir de un esfuerzo especial. Finalmente, se describe el entorno y los recursos de la prueba. Configuraciones inusuales de hardware, simuladores peculiares y herramientas o técnicas de prueba especial son algunos de los muchos temas que también pueden analizarse.

A continuación se describe el procedimiento de prueba detallado que se requiere para lograr el plan de prueba. Se señala el orden de la integración y las pruebas correspondientes en cada paso de ésta. También se incluye una lista de todos los casos de prueba (anotados para referencia posterior) y los resultados esperados.

En un *Reporte de prueba*, que puede anexarse a la *Especificación pruebas* si se desea, se registra una historia de resultados, problemas o peculiaridades de prueba reales. La información contenida en esta sección puede ser vital durante el mantenimiento del software. También se presentan las referencias y apéndices apropiados.

Como todos los demás elementos de una configuración de software, el formato de la especificación pruebas puede adaptarse a las necesidades locales de una organización de ingeniería de software. Sin embargo, es importante señalar que una estrategia de integración (contenida en un plan de prueba) y los detalles de la prueba (descritos en un procedimiento de prueba) son ingredientes esenciales y deben aparecer.

17.4 ESTRATEGIAS DE PRUEBA PARA SOFTWARE ORIENTADO A OBJETO[3]

Enunciado de manera simple, el objetivo de probar es encontrar el mayor número posible de errores con una cantidad manejable de esfuerzo aplicado durante un lapso realista. Aunque este objetivo fundamental se mantiene invariable para el software orientado a objeto, la naturaleza de este software cambia tanto la estrategia como las tácticas de la prueba (capítulo 19).

17.4.1 Prueba de unidad en el contexto OO

Cuando se considera software orientado a objeto, el concepto de unidad cambia. La encapsulación determina la definición de clases y objetos. Esto significa que cada clase y cada instancia de una clase empaqueta los atributos (datos) y las operaciones que manipulan estos datos. Por lo

3 En el apéndice 2 se presentan conceptos básicos orientados a objeto.

general, una clase encapsulada es el foco de la prueba de unidad. No obstante, las operaciones (métodos) dentro de la clase son las unidades comprobables más pequeñas. Puesto que una clase puede contener algunas operaciones diferentes, y una operación particular puede existir como parte de algunas clases diferentes, las tácticas aplicadas a la prueba de unidad deben cambiar.

PUNTO CLAVE

La prueba de clase para software OO es análoga a la prueba de módulo para software convencional. No es aconsejable probar operaciones en aislamiento.

Ya no es posible probar una sola operación en aislamiento (la visión convencional de la prueba de unidad) sino más bien como parte de una clase. Para ilustrarlo, considere una jerarquía de clase en la que una operación *X* se define para la superclase y la heredan algunas subclases. Cada subclase usa la operación *X*, pero se aplica dentro del contexto de los atributos y operaciones privados que se definieron para la subclase. Dado que el contexto en el que se usa la operación *X* varía sutilmente, es necesario probar la operación *X* en el contexto de cada una de las subclases. Esto significa que por lo general no es efectivo probar la operación *X* en forma aislada (el enfoque de prueba de unidad convencional) en el contexto orientado a objeto.

La prueba de clase para software OO es el equivalente de la prueba de unidad para software convencional. A diferencia de la prueba de unidad del software convencional, que tiende a enfocarse sobre el detalle algorítmico de un módulo y en los datos que fluyen a través de la interfaz de módulo, la prueba de clase para software OO la dirigen las operaciones encapsuladas por la clase y el comportamiento de estado de ésta.

17.4.2 Prueba de integración en el contexto OO

Puesto que el software orientado a objeto no tiene una estructura de control jerárquico obvia, las estrategias tradicionales descendente y ascendente (sección 17.3.2) tienen poco significado. Además, con frecuencia es imposible integrar las operaciones una a la vez en una clase (el enfoque de integración incremental convencional) debido a las "interacciones directa e indirecta de los componentes que constituyen la clase" [Ber93].

Una hebra es el camino de funciones que se ejecutan. Estando las funciones en diferentes clases.

OO es la prueba basada en hebra.

La prueba basada en uso consiste en probar todas las clases independientes, luego todas las dependientes y fin.

clases.

Existen dos estrategias diferentes para la prueba de integración de los sistemas OO [Bin94b]. La primera, la *prueba basada en hebra*, integra el conjunto de clases requeridas para responder a una entrada o evento para el sistema. Cada hebra se integra y prueba de manera individual. La prueba de regresión se aplica para asegurar que no ocurran efectos colaterales. El segundo enfoque de integración, la *prueba basada en uso*, comienza la construcción del sistema al probar dichas clases (llamadas *clases independientes*) que usan muy pocas clases *servidor* (si es que usan alguna). Después de probar las clases independientes, se prueba la siguiente capa de clases, llamadas *dependientes*, que usan las clases independientes. Esta secuencia de probar capas de clases dependientes continúa hasta que se construye todo el sistema.

El uso de controladores y representantes también cambia cuando se realiza la prueba de integración de los sistemas OO. Los controladores pueden usarse para probar operaciones en el nivel más bajo, y para la prueba de todos los grupos de clases. También puede usarse un controlador para sustituir la interfaz de usuario, de modo que las pruebas de funcionalidad del sistema puedan realizarse antes de la implementación de la interfaz. Los representantes (*stubs*) pueden usarse en situaciones donde se requiere la colaboración entre clases pero donde una o más de las clases colaboradoras todavía no se implementan por completo.

La *prueba de grupo* es un paso en la prueba de integración del software OO. Aquí, un grupo de clases colaboradoras (determinadas al examinar el CRC y el modelo objeto relacional) se ejercita al diseñar casos de prueba que intentan descubrir errores en las colaboraciones.

17.5 Estrategias de prueba para *webapps*

La estrategia para probar *webapps* adopta los principios básicos para todas las pruebas de software y aplica una estrategia y tácticas que se usan para sistemas orientados a objetos. Los siguientes pasos resumen el enfoque:

PUNTO CLAVE

La estrategia global para probar *webapps* puede resumirse en los 10 pasos que se anotan aquí.

1. El modelo de contenido para la *webapp* se revisa para descubrir errores.
2. El modelo de interfaz se revisa para garantizar que todos los casos de uso pueden adecuarse.
3. El modelo de diseño para la *webapp* se revisa para descubrir errores de navegación.
4. La interfaz de usuario se prueba para descubrir errores en los mecanismos de presentación y/o navegación.
5. A cada componente funcional se le aplica una prueba de unidad.
6. Se prueba la navegación a lo largo de toda la arquitectura.
7. La *webapp* se implementa en varias configuraciones ambientales diferentes y se prueba en su compatibilidad con cada configuración.
8. Las pruebas de seguridad se realizan con la intención de explotar vulnerabilidades en la *webapp* o dentro de su ambiente.
9. Se realizan pruebas de rendimiento.
10. La *webapp* se prueba mediante una población de usuarios finales controlada y monitoreada. Los resultados de su interacción con el sistema se evalúan por errores de contenido y navegación, preocupaciones de facilidad de uso, preocupaciones de compatibilidad, así como confiabilidad y rendimiento de la *webapp*.

WebRef

En **www.stickyminds.com/testing.asp** pueden encontrarse excelentes artículos acerca de pruebas de las *webapps*.

Puesto que muchas *webapps* evolucionan continuamente, el proceso de prueba es una actividad siempre en marcha, y se realiza para apoyar al personal que usa pruebas de regresión derivadas de las pruebas desarrolladas cuando se elaboró por primera vez la *webapp*. En el capítulo 20 se consideran métodos para probar la *webapp*.

Resumir las pruebas de validación !!!

17.6 PRUEBAS DE VALIDACIÓN

Las pruebas de validación comienzan en la culminación de las pruebas de integración, cuando se ejercitaron componentes individuales, el software está completamente ensamblado como un paquete y los errores de interfaz se descubrieron y corrigieron. En el nivel de validación o de sistema, desaparece la distinción entre software convencional, software orientado a objetos y *webapps*. Las pruebas se enfocan en las acciones visibles para el usuario y las salidas del sistema reconocibles por el usuario.

PUNTO CLAVE

Como todos los demás pasos de las pruebas, la validación intenta descubrir errores, pero el **enfoque se orienta** en **los requerimientos**: sobre las cosas que serán inmediatamente aparentes **para** el usuario final.

La validación puede definirse en muchas formas, pero una definición simple (aunque dura) es que la validación es exitosa cuando el software funciona en una forma que cumpla con las expectativas razonables del cliente. En este punto, un desarrollador de software curtido en la batalla puede protestar: "¿quién o qué es el árbitro de las expectativas razonables?". Si se desarrolló una *Especificación de requerimientos de software*, en ella se describen todos los atributos del software visibles para el usuario; contiene una sección de *Criterios de validación* que forman la base para un enfoque de pruebas de validación.

17.6.1 Criterios de pruebas de validación

La validación del software se logra a través de una serie de pruebas que demuestran conformidad con los requerimientos. Un plan de prueba subraya las clases de pruebas que se van a realizar y un procedimiento de prueba define casos de prueba específicos que se diseñan para garantizar que: se satisfacen todos los requerimientos de funcionamiento, se logran todas las características de comportamiento, todo el contenido es preciso y se presenta de manera adecuada, se logran todos los requerimientos de rendimiento, la documentación es correcta y se

satisfacen la facilidad de uso y otros requerimientos (por ejemplo, transportabilidad, compatibilidad, recuperación de error, mantenimiento).

Después de realizar cada caso de prueba de validación, existen dos posibles condiciones: 1) La característica de función o rendimiento se conforma de acuerdo con las especificaciones y se acepta, o 2) se descubre una desviación de la especificación y se crea una lista de deficiencias. Las desviaciones o errores descubiertos en esta etapa en un proyecto rara vez pueden corregirse antes de la entrega calendarizada. Con frecuencia es necesario negociar con el cliente para establecer un método para resolver deficiencias.

17.6.2 Revisión de la configuración

Un elemento importante del proceso de validación es una *revisión de la configuración*. La intención de la revisión es garantizar que todos los elementos de la configuración del software se desarrollaron de manera adecuada, y que se cataloga y se tiene el detalle necesario para reforzar las actividades de apoyo. La revisión de la configuración, en ocasiones llamada auditoría, se estudia con más detalle en el capítulo 22.

17.6.3 Pruebas alfa y beta

Cita:

"Teniendo los suficientes ojos, todos los errores son superficiales (por ejemplo, con una base suficientemente grande de personas que realizan pruebas beta y codesarrolladores, casi todo problema se caracterizará rápidamente y la corrección será obvia para alguien)."

E. Raymond

Virtualmente, es imposible que un desarrollador de software prevea cómo usará el cliente realmente un programa. Las instrucciones para usarlo pueden malinterpretarse; regularmente pueden usarse combinaciones extrañas de datos; la salida que parecía clara a quien realizó la prueba puede ser ininteligible para un usuario.

Cuando se construye software a la medida para un cliente, se realiza una serie de pruebas de aceptación a fin de permitir al cliente validar todos los requerimientos. Realizada por el usuario final en lugar de por los ingenieros de software, una prueba de aceptación puede variar desde una "prueba de conducción" informal hasta una serie de pruebas planificadas y ejecutadas sistemáticamente. De hecho, la prueba de aceptación puede realizarse durante un periodo de semanas o meses, y mediante ella descubrir errores acumulados que con el tiempo puedan degradar el sistema.

Si el software se desarrolla como un producto que va a ser usado por muchos clientes, no es práctico realizar pruebas de aceptación formales con cada uno de ellos. La mayoría de los constructores de productos de software usan un proceso llamado prueba alfa y prueba beta para descubrir errores que al parecer sólo el usuario final es capaz de encontrar.

¿Cuál es la diferencia entre una prueba alfa y una prueba beta?

La *prueba alfa* se lleva a cabo en el sitio del desarrollador por un grupo representativo de usuarios finales. El software se usa en un escenario natural con el desarrollador "mirando sobre el hombro" de los usuarios y registrando los errores y problemas de uso. Las pruebas alfa se realizan en un ambiente controlado.

La *prueba beta* se realiza en uno o más sitios del usuario final. A diferencia de la prueba alfa, por lo general el desarrollador no está presente. Por tanto, la prueba beta es una aplicación "en vivo" del software en un ambiente que no puede controlar el desarrollador. El cliente registra todos los problemas (reales o imaginarios) que se encuentran durante la prueba beta y los reporta al desarrollador periódicamente. Como resultado de los problemas reportados durante las pruebas beta, es posible hacer modificaciones y luego preparar la liberación del producto de software a toda la base de clientes.

En ocasiones se realiza una variación de la prueba beta, llamada *prueba de aceptación del cliente*, cuando el software se entrega a un cliente bajo contrato. El cliente realiza una serie de pruebas específicas con la intención de descubrir errores antes de aceptar el software del desarrollador. En algunos casos (por ejemplo, un gran corporativo o sistema gubernamental) la prueba de aceptación puede ser muy formal y abarcar muchos días o incluso semanas de prueba.

Diferencia entre beta y aceptación del cliente: Beta se utiliza en software generico, mientras que la prueba de aceptación del cliente se dirigue a un cliente especifico.

CASASEGURA



Preparación para la validación

La escena: Oficina de Doug Miller, mientras continúan tanto el diseño a nivel de componentes como la construcción de ciertos componentes.

Participantes: Doug Miller, jefe de ingeniería del software, Vinod, Jamie, Ed y Shakira, miembros del equipo de ingeniería del software *CasaSegura*.

La conversación:

Doug: El primer incremento estará listo para validación en... ¿cuánto tiempo? ¿Tres semanas?

Vinod: Más o menos. La integración va bien. Hacemos pruebas de humo todos los días y encontramos algunos *bugs*, pero nada que no podamos manejar. Hasta el momento va bien.

Doug: Háblame de la validación.

Shakira: Bueno, para el diseño de prueba, usaremos todos los casos de uso como base. Todavía no empiezo, pero desarrollaré pruebas para todos los casos de uso de las que sea responsable.

Ed: Igual yo.

Jamie: Yo también, pero debemos actuar juntos para la prueba de aceptación y también para las pruebas alfa y beta, ¿o no?

Doug: Sí. De hecho lo he pensado; podríamos traer un contratista externo para ayudarnos con la validación. Tengo dinero en el presupuesto... y él nos daría un nuevo punto de vista.

Vinod: Creo que lo tenemos bajo control.

Doug: Estoy seguro que sí, pero un GPI nos da un vistazo independiente del software.

Jamie: Estamos apretados de tiempo, Doug. Yo no tengo tiempo para vigilar a alguien que traigan para hacer el trabajo.

Doug: Lo sé, lo sé. Pero si un GPI funciona a partir de los requerimientos y los casos de uso, no necesitará demasiada vigilancia.

Vinod: Yo todavía creo que lo tenemos bajo control.

Doug: Te escuché, Vinod, pero voy a sostener mi opinión en esta ocasión. Más tarde planearemos la reunión con el representante del GPI para esta semana. Dejemos que comiencen y veamos lo que proponen.

Vinod: Muy bien, tal vez eso aligere un poco la carga.

17.7 PRUEBAS DEL SISTEMA

Cita:

"Como la muerte y los impuestos, las pruebas son desagradables e inevitables".

Ed Yourdon

Al comienzo de este libro, se resaltó el hecho de que el software sólo es un elemento de un sistema basado en computadora más grande. A final de cuentas, el software se incorpora con otros elementos del sistema (por ejemplo, hardware, personas, información), y se lleva a cabo una serie de pruebas de integración y validación del sistema. Estas pruebas quedan fuera del ámbito del proceso de software y no se llevan a cabo exclusivamente por parte de ingenieros de software. Sin embargo, los pasos que se toman durante el diseño y la prueba del software pueden mejorar enormemente la probabilidad de integración exitosa del software en el sistema más grande.

Un problema clásico en la prueba del sistema es el "dedo acusador". Esto ocurre cuando se descubre un error y los desarrolladores de diferentes elementos del sistema se culpan unos a otros por el problema. En lugar de abandonarse a tal sinsentido, deben anticiparse los potenciales problemas de interfaz y: 1) diseñar rutas de manejo de error que prueben toda la información proveniente de otros elementos del sistema, 2) realizar una serie de pruebas que simulen los datos malos u otros errores potenciales en la interfaz del software, 3) registrar los resultados de las pruebas para usar como "evidencia" si ocurre el dedo acusador, y 4) participar en planificación y diseño de pruebas del sistema para garantizar que el software se prueba de manera adecuada.

En realidad, la *prueba del sistema* es una serie de diferentes pruebas cuyo propósito principal es ejercitar por completo el sistema basado en computadora. Aunque cada prueba tenga un propósito diferente, todo él funciona para verificar que los elementos del sistema se hayan integrado de manera adecuada y que se realicen las funciones asignadas. En las secciones que siguen se estudian los tipos de pruebas del sistema que valen la pena para los sistemas basados en software.

17.7.1 Pruebas de recuperación

Muchos sistemas basados en computadora deben recuperarse de fallas y reanudar el procesamiento con poco o ningún tiempo de inactividad. En algunos casos, un sistema debe ser tole-

rante a las fallas, es decir, las fallas del procesamiento no deben causar el cese del funcionamiento del sistema global. En otros casos, la falla de un sistema debe corregirse dentro de un periodo de tiempo específico u ocurrirán severos daños económicos.

La *recuperación* es una prueba del sistema que fuerza al software a fallar en varias formas y que verifica que la recuperación se realice de manera adecuada. Si la recuperación es automática (realizada por el sistema en sí), se evalúa el reinicio, los mecanismos de puntos de verificación, la recuperación de datos y la reanudación para correcciones. Si la recuperación requiere intervención humana, se evalúa el tiempo medio de reparación (TMR) para determinar si está dentro de límites aceptables.

17.7.2 Pruebas de seguridad

Cualquier sistema basado en computadora que gestione información sensible o cause acciones que puedan dañar (o beneficiar) de manera inadecuada a individuos es un blanco de penetración inadecuada o ilegal. La penetración abarca un amplio rango de actividades: *hackers* que intentan penetrar en los sistemas por deporte, empleados resentidos que intentan penetrar por venganza, individuos deshonestos que intentan penetrar para obtener ganancia personal ilícita.

Penetración: Acceder a la estructura interna del software sin permiso.

La *prueba de seguridad* intenta verificar que los mecanismos de protección que se construyen en un sistema en realidad lo protegerán de cualquier penetración impropia. Para citar a Beizar [Bei84]: "La seguridad del sistema debe, desde luego, probarse para ser invulnerable ante ataques frontales; pero también debe probarse su invulnerabilidad contra ataques laterales y traseros."

Durante la prueba de seguridad, quien realiza la prueba juega el papel del individuo que desea penetrar al sistema. ¡Cualquier cosa vale! Quien realice la prueba puede intentar adquirir contraseñas por medios administrativos externos; puede atacar el sistema con software a la medida diseñado para romper cualquier defensa que se haya construido; puede abrumar al sistema, y por tanto negar el servicio a los demás; puede causar a propósito errores del sistema con la esperanza de penetrar durante la recuperación; puede navegar a través de datos inseguros para encontrar la llave de la entrada al sistema.

Con los suficientes tiempo y recursos, las buenas pruebas de seguridad a final de cuentas penetran en el sistema. El papel del diseñador de sistemas es hacer que el costo de la penetración sea mayor que el valor de la información que se obtendrá.

17.7.3 Pruebas de esfuerzo

Los primeros pasos de la prueba del software dieron como resultado una evaluación extensa de las funciones y el rendimiento normales del programa. Las pruebas de esfuerzo se diseñan para enfrentar los programas con situaciones anormales. En esencia, la persona que realiza las pruebas de esfuerzo pregunta: "¿cuánto podemos doblar esto antes de que se rompa?".

Cita:

"Si intenta encontrar verdaderos errores del sistema y no sujeta su software a una verdadera prueba de esfuerzo, entonces es el momento de comenzar."

Boris Beizer

La *prueba de esfuerzo* ejecuta un sistema en forma que demanda recursos en cantidad, frecuencia o volumen anormales. Por ejemplo, pueden 1) diseñarse pruebas especiales que generen diez interrupciones por segundo, cuando una o dos es la tasa promedio, (2) aumentarse las tasas de entrada de datos en un orden de magnitud para determinar cómo responderán las funciones de entrada, 3) ejecutarse casos de prueba que requieran memoria máxima y otros recursos, 4) diseñarse casos de prueba que puedan causar *thrashing* (que es un quebranto del sistema por hiperpaginación) en un sistema operativo virtual, 5) crearse casos de prueba que puedan causar búsqueda excesiva por datos residentes en disco. En esencia, la persona que realiza la prueba intenta romper el programa.

Una variación de la prueba de esfuerzo es una técnica llamada *prueba de sensibilidad*. En algunas situaciones (la más común ocurre en algoritmos matemáticos), un rango muy pequeño

de datos contenidos dentro de las fronteras de los datos válidos para un programa pueden causar procesamiento extremo, e incluso erróneo, o profunda degradación del rendimiento. La prueba de sensibilidad intenta descubrir combinaciones de datos dentro de clases de entrada válidas que puedan causar inestabilidad o procesamiento inadecuado.

17.7.4 Pruebas de rendimiento

Para sistemas en tiempo real y sistemas embebidos , el software que proporcione la función requerida, pero que no se adecue a los requerimientos de rendimiento, es inaceptable. La prueba de rendimiento se diseña para poner a prueba el rendimiento del software en tiempo de corrida, dentro del contexto de un sistema integrado. La prueba del rendimiento ocurre a lo largo de todos los pasos del proceso de prueba. Incluso en el nivel de unidad, puede accederse al rendimiento de un módulo individual conforme se realizan las pruebas. Sin embargo, no es sino hasta que todos los elementos del sistema están plenamente integrados cuando puede determinarse el verdadero rendimiento de un sistema.

Las pruebas de rendimiento con frecuencia se aparean con las pruebas de esfuerzo y por lo general requieren instrumentación de hardware y de software, es decir, con frecuencia es necesario medir la utilización de los recursos (por ejemplo, ciclos del procesador) en forma meticulosa. La instrumentación externa puede monitorear intervalos de ejecución y eventos de registro (por ejemplo, interrupciones) conforme ocurren, y los muestreos del estado de la máquina de manera regular. Con la instrumentación de un sistema, la persona que realiza la prueba puede descubrir situaciones que conduzcan a degradación y posibles fallas del sistema.

17.7.5 Pruebas de despliegue

En muchos casos, el software debe ejecutarse en varias plataformas y bajo más de un entorno de sistema operativo. La *prueba de despliegue*, en ocasiones llamada *prueba de configuración*, ejercita el software en cada entorno en el que debe operar. Además, examina todos los proce-

HERRAMIENTAS DE SOFTWARE



Planeación y administración de pruebas

Objetivo: Estas herramientas ayudan al equipo de software a planificar la estrategia de pruebas que se elija y a administrar el proceso de prueba mientras se lleva a cabo.

Mecánica: Las herramientas en esta categoría abordan la planificación de las pruebas, el almacenamiento, administración y control de las mismas; el seguimiento de los requisitos, la integración, el rastreo de errores y la generación de reportes. Los gestores de proyecto los usan para complementar las herramientas calendarizadas del proyecto. Quienes realizan las pruebas usan estas herramientas para planear actividades de prueba y controlar el flujo de información conforme avanza el proceso de pruebas.

Herramientas representativas:[4]

QaTraq Test Case Management Tool, desarrollada por Traq Software (**www.testmanagement.com**), "alienta un enfoque estructurado de la gestión de pruebas".

QADirector, desarrollada por Compuware Corp. (**www.compuware.com/qacenter**), proporciona un solo punto de control para gestionar todas las fases del proceso de pruebas.

TestWorks, desarrollada por Software Research, Inc. (**www.soft.com/Products/index.html**), contiene una suite completamente integrada de herramientas de prueba, incluidas herramientas para administración y reporte de pruebas.

OpensourceTesting.org (**www.opensourcetesting.org/testmgt.php**), cita varias herramientas de gestión y planificación de pruebas en fuente abierta.

NI TestStand, desarrollada por National Instruments Corp. (**www.ni.com**), le permite "desarrollar, gestionar y ejecutar secuencias de pruebas escritas en cualquier lenguaje de programación".

4 Las herramientas que se mencionan aquí no representan un respaldo, sino una muestra de las herramientas en esta categoría. En la mayoría de los casos, los nombres de las herramientas son marcas registradas por sus respectivos desarrolladores.

dimientos de instalación y el software de instalación especializado (por ejemplo, "instaladores") que usarán los clientes, así como toda la documentación que se usará para introducir el software a los usuarios finales.

Como ejemplo, piense en la versión accesible a internet del software *CasaSegura* que permitiría a un cliente monitorear el sistema de seguridad desde ubicaciones remotas. La *webapp* de *CasaSegura* debe probarse usando todos los navegadores web que es probable que se encuentren. Una prueba de despliegue más profunda puede abarcar combinaciones de navegadores web con varios sistemas operativos (por ejemplo, Linux, Mac OS, Windows). Puesto que la seguridad es un tema principal, un juego completo de pruebas de seguridad se integraría con la prueba de despliegue.

17.8 EL ARTE DE LA DEPURACIÓN

Cita:

"Para nuestra sorpresa, descubrimos que no fue tan fácil obtener programas justo como los habíamos pensado. Recuerdo el instante exacto en el que me di cuenta de que una gran parte de mi vida, a partir de entonces, la iba a pasar descubriendo los errores en mis propios programas".

Maurice Wilkes, descubre la depuración, 1949

La prueba del software es un proceso que puede planearse y especificarse de manera sistemática. Puede realizarse el diseño de casos de prueba, definir una estrategia y evaluar los resultados comparándolos con las expectativas prescritas.

La *depuración* ocurre como consecuencia de las pruebas exitosas. Es decir, cuando un caso de prueba descubre un error, la depuración es el proceso que da como resultado la remoción del error. Aunque la depuración puede y debe ser un proceso ordenado, todavía en mucho es un arte. Los ingenieros de software con frecuencia se enfrentan con indicios "sintomáticos" de un problema de software mientras evalúan los resultados de una prueba, es decir, la manifestación externa del error y su causa interna pueden no tener relación obvia una con otra. El proceso mental, pobremente comprendido, que conecta un síntoma con una causa se conoce como depuración.

17.8.1 El proceso de depuración

La depuración no es una prueba, pero con frecuencia ocurre como consecuencia de una prueba.[5] De acuerdo con la figura 17.7, el proceso de depuración comienza con la ejecución de un caso de prueba. Los resultados se valoran y se encuentra la falta de correspondencia entre el rendimiento esperado y el real. En muchos casos, la no correspondencia de los datos es un síntoma de una causa subyacente y escondida. El proceso de depuración intenta relacionar síntoma con causa, lo que por tanto conduce a la corrección del error.

CONSEJO

Asegúrese de evitar un tercer resultado: se encuentra la causa, pero la "corrección" no resuelve el problema o incluso introduce otro error.

Por lo general, El proceso de depuración dará como resultado que: 1) la causa se encontrará y corregirá o 2) la causa no se encontrará. En el último caso, la persona que realiza la depuración puede sospechar una causa, diseñar un caso de prueba para auxiliarse en la validación de dicha suposición y trabajar hacia la corrección del error en forma iterativa.

¿Por qué es tan difícil la depuración?

¿Por qué es tan difícil la depuración? Con toda probabilidad, la psicología humana (vea la sección 17.8.2) tiene más que ver con la respuesta que la tecnología del software. Sin embargo, ciertas características de los errores brindan algunas pistas:

1. El síntoma y la causa pueden ser geográficamente remotos. Es decir, el síntoma puede aparecer en una parte de un programa, mientras que la causa en realidad puede ubicarse en un sitio que esté alejado. Los componentes altamente acoplados (capítulo 8) exacerban esta situación.
2. El síntoma puede desaparecer (temporalmente) cuando se corrige otro error.

5 Al hacer esta afirmación, se toma la visión más amplia posible de las pruebas. No sólo el desarrollador prueba el software previo a su liberación, ¡sino que el cliente/usuario prueba el software cada vez que lo usa!

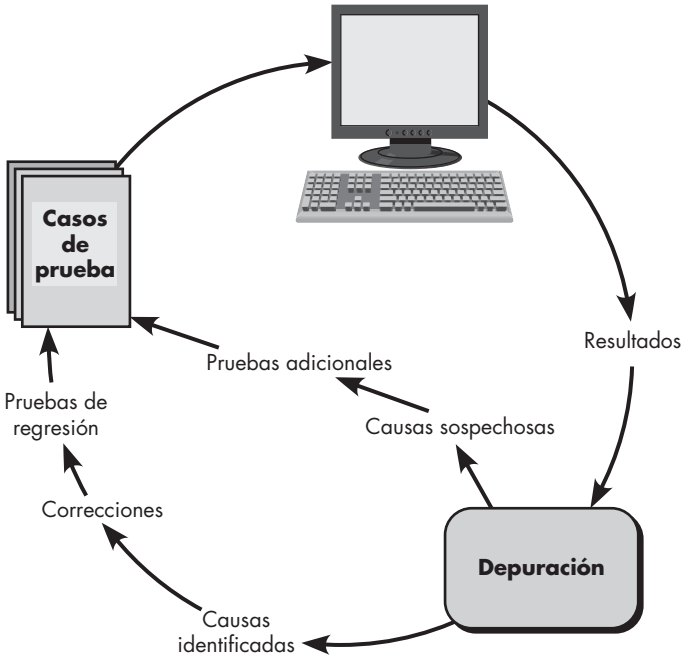


FIGURA 17.7 El proceso de depuración

3. El síntoma en realidad puede no ser causado por errores (por ejemplo, imprecisiones de redondeo).
4. El síntoma puede ser causado por un error humano que no se rastrea con facilidad.
5. El síntoma puede ser resultado de problemas de temporización más que de problemas de procesamiento.
6. Puede ser difícil reproducir con precisión las condiciones de entrada (por ejemplo, una aplicación en tiempo real en la que el orden de la entrada esté indeterminado).
7. El síntoma puede ser intermitente, particularmente común en sistemas embebidos que acoplan hardware y software de manera inextricable.
8. El síntoma puede deberse a causas que se distribuyen a través de algunas tareas que corren en diferentes procesadores.

? "Todo mundo sabe que la depuración es el doble de difícil que escribir un programa por primera vez. De modo que, si se es tan inteligente como se puede ser cuando se escribe el programa, ¿cómo es que se depurará?".

Brian Kernighan

Durante la depuración, encontrará errores que varían desde los ligeramente desconcertantes (por ejemplo, un formato de salida incorrecto) hasta los catastróficos (por ejemplo, la falla del sistema, que provoca serio daño económico o físico). Conforme aumentan las consecuencias de un error, también aumenta la cantidad de presión por encontrar la causa. Con frecuencia, la presión fuerza a algunos de los desarrolladores del software a corregir un error y, al mismo tiempo, introducir dos más.

17.8.2 Consideraciones psicológicas

Por desgracia, parece haber cierta evidencia de que la hazaña de la depuración es un rasgo humano innato. Algunas personas son buenas en ello y otras no lo son. Aunque la evidencia experimental de la depuración está abierta a muchas interpretaciones, se reportan grandes variaciones en la habilidad depuradora para programadores con la misma educación y experiencia. Al comentar acerca de los aspectos humanos de la depuración, Shneiderman [Shn80] afirma:

> La depuración es una de las partes más frustrantes de la programación. Tiene elementos de resolución de problemas o rompecabezas, junto con el desconcertante reconocimiento de que se cometió un error. La elevada ansiedad y la falta de voluntad para aceptar la posibilidad de los errores aumentan la dificultad de la tarea. Por fortuna, hay un gran alivio y la tensión se aligera cuando finalmente el error... se corrige.

Aunque puede ser difícil "aprender" a depurar, es posible proponer algunos enfoques al problema. Revise la sección 17.8.3.

CASASEGURA



Depuración

La escena: Cubículo de Ed mientras se realiza la codificación y la prueba de unidad.

Participantes: Ed y Shakira, miembros del equipo de ingeniería de software *CasaSegura*.

La conversación:

Shakira (observa a través de la entrada del cubículo): Hola... ¿dónde estuviste a la hora del almuerzo?

Ed: Aquí... trabajando.

Shakira: Te ves horrible... ¿cuál es el problema?

Ed (suspira): He estado trabajando en este... error desde que lo descubrí a las 9:30 esta mañana y es, ¿qué?, 2:45... No tengo ni idea.

Shakira: Creí que todos estuvimos de acuerdo en no pasar más de una hora en tareas de depuración por cuenta propia; luego pediríamos ayuda, ¿verdad?

Ed: Sí, pero...

Shakira (entra al cubículo): ¿Así que cuál es el problema?

Ed: Es complicado y, además, lo he visto durante, ¿cuánto?, 5 horas. No lo vas a ver en 5 minutos.

Shakira: Permíteme... ¿cuál es el problema?

[Ed explica el problema a Shakira, quien lo observa durante 30 segundos sin hablar, luego...]

Shakira (se asoma una sonrisa en su cara): Oh, justo ahí, la variable llamada *setAlarmCondition*. ¿No debería ponerse en "falso" antes de comenzar el bucle?

[Ed mira la pantalla con incredulidad, se dobla hacia adelante y comienza a golpear su cabeza suavemente contra el monitor. Shakira, quien ahora sonríe abiertamente, se incorpora y sale del cubículo].

17.8.3 Estrategias de depuración



Establezca un límite, por decir, dos horas, en la cantidad de tiempo que empleará al intentar depurar un problema por cuenta propia. Después de eso, ¡pida ayuda!

Sin importar el enfoque que se tome, la depuración tiene un objetivo dominante: encontrar y corregir la causa de un error o defecto de software. El objetivo se realiza mediante una combinación de evaluación sistemática, intuición y suerte. Bradley [Bra85] describe el enfoque de depuración de la siguiente forma:

> La depuración es una aplicación directa del método científico que se ha desarrollado durante más de 2 500 años. La base de la depuración es localizar la fuente del problema [la causa] mediante una partición binaria, a través del trabajo con hipótesis que predicen nuevos valores por examinar.
>
> Tome un ejemplo simple que no sea de software: una lámpara en mi casa no funciona. Si nada en la casa funciona, la causa debe estar en el interruptor principal o en el exterior; observo mi alrededor para ver si el vecindario está a oscuras. Conecto la lámpara sospechosa en un tomacorriente que funcione y un electrodoméstico operativo en el circuito sospechoso. Y así continúo la alternación entre hipótesis y pruebas.

En general, se han propuesto tres estrategias de depuración [Mye79]: 1) fuerza bruta, 2) vuelta atrás (del inglés *backtracking*) y 3) eliminación de causas. Cada una de estas estrategias puede llevarse a cabo de manera manual, pero modernas herramientas de depuración pueden hacer el proceso mucho más efectivo.

Tácticas de depuración. La categoría *fuerza bruta* de la depuración probablemente es el método más común y menos eficiente para aislar la causa de un error de software. Los métodos

Fuerza Bruta: Consiste en hacer que el programa registre TODO lo que hace mientras se ejecuta, con la esperanza de que en toda esa información se encuentre el origen del error.

de depuración de fuerza bruta se aplican cuando todo lo demás falla. Al usar una filosofía de "deje que la computadora encuentre el error", se toman copias de la memoria (*dumps*), se invocan rastreos en el tiempo de corrida y el programa se carga con enunciados de salida. La esperanza es que, en alguna parte del pantano de información que se produzca, se encontrará una pista que pueda conducir a la causa de un error. Aunque la masa de información producida a final de cuentas puede conducir al éxito, con más frecuencia conduce a desperdicio de esfuerzo y tiempo. ¡Piense que primero debe gastarse!

El *seguimiento hacia atrás* o vuelta atrás es un enfoque de depuración bastante común que puede usarse exitosamente en programas pequeños. Al comenzar en el sitio donde se descubrió un síntoma, el código fuente se rastrea hacia atrás (de manera manual) hasta que se encuentra la causa. Por desgracia, conforme aumenta el número de líneas fuente, el número de rutas potenciales hacia atrás puede volverse inmanejable.

El tercer enfoque de la depuración, la *eliminación de la causa*, se manifiesta mediante inducción o deducción, e introduce el concepto de partición binaria. Los datos relacionados con la ocurrencia del error se organizan para aislar las causas potenciales. Se plantea una "hipótesis de causa" y los datos anteriormente mencionados se usan para probar o refutar la hipótesis. De manera alternativa, se desarrolla una lista de las posibles causas y se realizan pruebas para eliminar cada una. Si las pruebas iniciales indican que una hipótesis de causa particular se muestra prometedora, los datos se refinan con la intención de aislar el error.

Depuración automatizada. Cada uno de estos enfoques de depuración puede complementarse con herramientas de depuración que puedan proporcionar apoyo semiautomático conforme se intenten estrategias de depuración. Hailpern y Santhanam [Hai02] resumen el estado de estas herramientas cuando apuntan: "... se han propuesto muchos nuevos enfoques y están disponibles muchos entornos de depuración comerciales. Los entornos de desarrollo integrados (IDE) brindan una forma de capturar algunos de los errores predeterminados específicos del lenguaje (por ejemplo, falta de caracteres de fin de sentencia , variables indefinidas, etc.) sin requerir compilación". Se dispone de una gran variedad de compiladores de depuración, ayudas dinámicas de depuración ("trazadores"), generadores automáticos de casos de prueba y herramientas de mapeo de referencia cruzada. Sin embargo, las herramientas no son un sustituto

HERRAMIENTAS DE SOFTWARE



Depuración

Objetivo: Estas herramientas proporcionan asistencia automatizada para quienes deben depurar problemas de software. La intención es proporcionar conocimiento que puede ser difícil de obtener si se aborda el proceso de depuración de forma manual.

Mecánica: La mayoría de las herramientas de depuración son específicas del lenguaje de programación y del entorno.

Herramientas representativas:[6]

Borland Gauntlet, distribuido por Borland (**www.borland. com**), auxilia tanto en las pruebas como en la depuración.

Coverty Prevent SQS, desarrollada por Coverty (**www.coverty. com**), proporciona asistencia de depuración tanto para C++ como para Java.

C++Test, desarrollada por Parasoft (**www.parasoft.com**), es una herramienta de prueba de unidad que soporta un rango completo de pruebas en código C y C++. Las características de depuración ayudan en el diagnóstico de errores que se encuentren.

CodeMedic, desarrollada por NewPlanet Software (**www. newplanetsfotware.com/medic/**), proporciona una interfaz gráfica para el depurador estándar UNIX, *gdb*, e implementa sus características más importantes. En la actualidad, *gdb* soporta C/C++, Java, PalmOS, varios sistemas incrustados, lenguaje ensamblador, FORTRAN y Modula-2.

GNATS, una aplicación freeware (**www.gnu.org/software/ gnats**), es un conjunto de herramientas para rastrear reportes de error.

6 Las herramientas que se mencionan aquí no representan un respaldo, sino una muestra de las herramientas en esta categoría. En la mayoría de los casos, los nombres de las herramientas son marcas registradas por sus respectivos desarrolladores.

para la evaluación cuidadosa basada en un modelo completo de diseño y en código fuente claro.

El factor humano. Cualquier discusión de los enfoques y herramientas de depuración está incompleta sin mencionar un poderoso aliado: ¡otras personas! Un punto de vista fresco, no empañado por horas de frustración, puede hacer maravillas.[7] Una máxima final para la depuración puede ser: "Cuando todo lo demás falle, ¡consiga ayuda!"

17.8.4 Corrección del error

Una vez encontrado el error, debe corregirse. Pero, como ya se señaló, la corrección de un error puede introducir otros errores y, por tanto, hacer más daño que bien. Van Vleck [Van89] sugiere tres preguntas simples que deben plantearse antes de hacer la "corrección" que remueva la causa de un error:

Cita:

"El mejor examinador no es aquel que encuentra más errores... el mejor es quien consigue la corrección de más errores."

Cem Kaner *et al.*

1. *¿La causa del error se reproduce en otra parte del programa?* En muchas situaciones, un defecto de programa es causado por un patrón de lógica erróneo que puede reproducirse en alguna otra parte. La consideración explícita del patrón lógico puede resultar en el descubrimiento de otros errores.
2. *¿Qué "siguiente error" puede introducirse con la corrección que está a punto de realizar?* Antes de hacer la corrección, debe evaluarse el código fuente (o, mejor, el diseño) para valorar el acoplamiento de las estructuras lógica y de datos. Si la corrección se realizará en una sección altamente acoplada del programa, debe tenerse especial cuidado cuando se realice algún cambio.
3. *¿Qué debió hacerse para evitar este error desde el principio?* Esta pregunta es el primer paso hacia el establecimiento de un enfoque de aseguramiento de calidad estadística del software (capítulo 16). Si se corrigen tanto el proceso como el producto, el error se removerá del programa actual y podrá eliminarse de todos los programas futuros.

17.9 Resumen

Las pruebas de software representan el porcentaje más grande de esfuerzo técnico en el proceso de software. Sin importar el tipo de software que se construya, una estrategia para planificar, ejecutar y controlar pruebas sistemáticas comienza por considerar pequeños elementos del software y moverse hacia afuera, hacia el programa como un todo.

El objetivo de las pruebas del software es descubrir errores. Para software convencional, este objetivo se logra mediante una serie de pasos de prueba. Las pruebas de unidad e integración se concentran en la verificación funcional de un componente y en la incorporación de componentes en una arquitectura de software. Las pruebas de validación demuestran la conformidad con los requerimientos del software y las pruebas del sistema validan el software una vez que se incorporó en un sistema más grande. Cada paso de la prueba se logra a través de una serie de técnicas de prueba sistemáticas que auxilian en el diseño de casos de prueba. Con cada paso de prueba, se amplía el nivel de abstracción con la que se considera el software.

La estrategia para probar software orientado a objeto comienza con pruebas que ejercitan las operaciones dentro de una clase y luego avanzan hacia la prueba basada en hebra para integración. Las hebras son conjuntos de clases que responden a una entrada o evento. Las pruebas basadas en uso se enfocan en clases que no colaboran demasiado con otras clases.

---

7 El concepto de programación por parejas (recomendado como parte del modelo de programación extrema que se estudió en el capítulo 3), proporciona un mecanismo de "depuración" conforme se diseña y codifica el software.

Las *webapps* se prueban en forma muy parecida a los sistemas OO. Sin embargo, las pruebas se diseñan para ejercitar contenido, funcionalidad, interfaz, navegación y aspectos de rendimiento y seguridad de la *webapp*.

A diferencia de las pruebas (una actividad sistemática planificada), la depuración puede verse como un arte. Al comenzar con una indicación sintomática de un problema, la actividad de depuración debe rastrear la causa de un error. De los muchos recursos disponibles durante la depuración, el más valioso es el consejo de otros miembros del equipo de ingeniería del software.

Problemas y puntos por evaluar

17.1. Con sus palabras, describa la diferencia entre verificación y validación. ¿Ambas usan los métodos de diseño de casos de prueba y estrategias de pruebas?

17.2. Mencione algunos problemas que pueden asociarse con la creación de un grupo de prueba independiente. ¿Los GPI y el SQA se integran con las mismas personas?

17.3. ¿Siempre es posible desarrollar una estrategia para probar software que usa la secuencia de pasos de prueba descritos en la sección 17.1.3? ¿Qué posibles complicaciones pueden surgir para sistemas incrustados?

17.4. ¿Por qué un módulo altamente acoplado es difícil para la prueba de unidad?

17.5. El concepto de "antierrores" (sección 17.3.1) es una forma extremadamente efectiva de brindar asistencia de depuración interna cuando se descubre un error:

a) Desarrolle un conjunto de lineamientos para antierror.
b) Analice las ventajas de usar la técnica.
c) Analice las desventajas.

17.6. ¿Cómo puede la calendarización del proyecto afectar la prueba de integración?

17.7. ¿La prueba de unidad es posible o incluso deseable en todas las circunstancias? Proporcione ejemplos para justificar su respuesta.

17.8. ¿Quién debe realizar la prueba de validación: el desarrollador o el usuario del software? Justifique su respuesta.

17.9. Desarrolle una estrategia de prueba completa para el sistema *CasaSegura* que se estudió anteriormente en este libro. Documéntela en una *Especificación de pruebas*.

17.10. Como proyecto de clase, desarrolle una *Guía de depuración* para su instalación. ¡La guía debe brindar lenguaje y sugerencias orientadas a sistemas aprendidos en la escuela de la vida! Comience por destacar los temas que revisarán la clase y el instructor. Publique la guía para otros en su entorno local.

Lecturas y fuentes de información adicionales

Virtualmente todo libro acerca de las pruebas del software analiza estrategias junto con métodos para diseño de casos de prueba. Everett y Raymond (*Software Testing,* Wiley-IEEE Computer Society Press, 2007), Black (*Pragmatic Software Testing,* Wiley, 2007), Spiller *et al*. (*Software Testing Process: Test Management,* Rocky Nook, 2007), Perry (*Effective Methods for Software Testing,* 3a. ed., Wiley, 2005), Lewis (*Software Testing and Continuous Quality Improvement,* 2a. ed., Auerbach, 2004), Loveland *et al*. (*Software Testing Techniques,* Charles River Media, 2004), Burnstein (*Practical Software Testing,* Springer, 2003), Dustin (*Effective Software Testing,* Addison-Wesley, 2002), Craig y Kaskiel (*Systematic Software Testing,* Artech House, 2002), Tamres (*Introducing Software Testing,* Addison-Wesley, 2002), Whittaker (*How to Break Software,* Addison-Wesley, 2002), y Kaner *et al*. (*Lessons Learned in Software Testing,* Wiley, 2001) son sólo una pequeña muestra de muchos libros que estudian los principios, conceptos, estrategias y métodos de las pruebas.

Para aquellos lectores con interés en los métodos de desarrollo de software ágiles, Crispin y House (*Testing Extreme Programming*, Addison-Wesley, 2002) y Beck (*Test Driven Development: By Example*, Addison-Wesley, 2002) presentan estrategias y tácticas de prueba para programación extrema. Kamer *et al*. (*Lessons Learned*

in Software Testing, Wiley, 2001) presentan una colección de más de 300 "lecciones" pragmáticas (lineamientos) que todo examinador de software debe aprender. Watkins (*Testing IT: An off-the-Shelf Testing Process*, Cambridge University Press, 2001) establece un marco conceptual de prueba efectivo para todo tipo de software desarrollado o adquirido. Manges y O'Brien (*Agile Testing with Ruby and Rails*, Apress, 2008) abordan estrategias y técnicas de prueba para el lenguaje de programación Ruby y el marco conceptual web.

Sykes y McGregor (*Practical Guide to Testing Object-Oriented Software,* Addison-Wesley, 2001), Bashir y Goel (*Testing Object-Oriented Software,* Springer-Verlag, 2000), Binder (*Testing Object-Oriented Systems,* Addison-Wesley, 1999), Kung *et. al*. (*Testing Object-Oriented Software,* IEEE Computer Society Press, 1998) y Marick (*The Craft of Software Testing,* Prentice-Hall, 1997) presentan estrategias y métodos para probar sistemas OO.

Lineamientos para depuración se encuentran en los libros de Grötker *et al*. (*The Developer's Guide to Debugging,* Springer, 2008), Agans (*Debugging,* Amacon, 2006), Zeller (*Why Programs Fail: A Guide to Systematic Debugging,* Morgan Kaufmann, 2005), Tells y Hsieh (*The Science of Debugging,* The Coreolis Group, 2001), y Robbins (*Debugging Applications,* Microsoft Press, 2000). Kaspersky (*Hacker Debugging Uncovered,* A-List Publishing, 2005) addresses the technology of debugging tools. Younessi (*Object-Oriented Defect Management of Software,* Prentice-Hall, 2002) aborda la tecnología de las herramientas para depuración. Younessi (*Object-Oriented Defect Management of Software*, Prentice-Hall, 2002) presenta técnicas para manejar defectos que se encuentran en sistemas orientados a objetos. Beizer [Bei84] presenta una interesante "taxonomía de errores" que puede conducir a métodos efectivos para planificación de pruebas.

Los libros de Madisetti y Akgul (*Debugging Embedded Systems,* Springer, 2007), Robbins (*Debugging Microsoft.NET 2.0 Applications,* Microsoft Press, 2005), Best (*Linux Debugging and Performance Tuning,* Prentice-Hall, 2005), Ford y Teorey (*Practical Debugging in C++,* Prentice-Hall, 2002), Brown (*Debugging Perl,* McGraw-Hill, 2000) y Mitchell (*Debugging Java,* McGraw-Hill, 2000) abordan la naturaleza especial de la depuración para los entornos implicados en sus títulos.

Una gran variedad de fuentes de información acerca de estrategias para pruebas está disponible en internet. Una lista actualizada de referencias en la World Wide Web, que son relevantes para las estrategias de prueba de software, puede encontrarse en el sitio web del libro: **www.mhle.com/engcs/compsci/pressman/professional/olc/ser.htm**.

CAPÍTULO

18

PRUEBA DE APLICACIONES CONVENCIONALES

CONCEPTOS CLAVE

análisis de valor de frontera 425
complejidad ciclomática 417
entornos especializados 429
gráfico de flujo 415
matrices de grafo 420
métodos de prueba basados en gráficos 423
partición de equivalencia . . . 425
patrones 433
prueba basada en modelo . . 429
prueba de arreglo ortogonal 426
prueba de caja blanca 414
prueba de caja negra 423
prueba de estructura de control 420
prueba de ruta básica 414

Las pruebas presentan una interesante anomalía para los ingenieros de software, quienes por naturaleza son personas constructivas. Las pruebas requieren que el desarrollador deseche nociones preconcebidas sobre lo "correcto" del software recién desarrollado y luego trabajen duro para diseñar casos de prueba a fin de "romper" el software. Beizer [Bei90] describe esta situación de manera efectiva cuando afirma:

> Existe el mito de que no habría errores que pescar si fuésemos realmente buenos en programación. Si realmente nos pudiéramos concentrar, si todo mundo usara programación estructurada, diseño descendente... entonces no habría errores. Ése es el mito. Hay errores, dice el mito, porque somos malos en lo que hacemos; y si lo somos, deberíamos sentirnos culpables por ello. Por tanto, la aplicación de pruebas y el diseño de casos de prueba es una admisión del fracaso, que inspira una buena dosis de culpa. Y el tedio de las pruebas es un justo castigo por nuestros errores. ¿El castigo por qué? ¿Por ser humanos? ¿Culpa por qué? ¿Por fracasar en lograr la perfección inhumana? ¿Por no distinguir entre lo que otro programador piensa y lo que dice? ¿Por no poder ser telépatas? ¿Por no resolver problemas de comunicación humana a los que se les ha dado la vuelta... durante siglos?

¿Las pruebas deben inspirar culpa? ¿Las pruebas son realmente destructivas? La respuesta a estas preguntas es: "¡No!"

En este capítulo se estudian técnicas para el diseño de casos de prueba de software para aplicaciones convencionales. Este diseño se enfoca en un conjunto de técnicas para la creación de casos de prueba que satisfacen los objetivos de prueba globales y las estrategias de pruebas que se estudiaron en el capítulo 17.

UNA MIRADA RÁPIDA

¿Qué es? Una vez generado el código fuente, el software debe probarse para descubrir (y corregir) tantos errores como sea posible antes de entregarlo al cliente. La meta es diseñar una serie de casos de prueba que tengan una alta probabilidad de encontrar errores; ¿pero cómo? Ahí es donde entran en escena las técnicas de prueba de software. Dichas técnicas proporcionan lineamientos sistemáticos para diseñar pruebas que: 1) revisen la lógica interna y las interfaces de todo componente de software y 2) revisen los dominios de entrada y salida del programa para descubrir errores en el funcionamiento, comportamiento y rendimiento del programa.

¿Quién lo hace? Durante las primeras etapas del proceso, un ingeniero de software realiza todas las pruebas. Sin embargo, conforme avanza el proceso, pueden involucrarse especialistas en pruebas.

¿Por qué es importante? Las revisiones y otras acciones SQA pueden y deben descubrir errores, pero no son suficientes. Cada vez que el programa se ejecuta, ¡el cliente lo prueba! Por tanto, tiene que ejecutarse el programa antes de que llegue al cliente, con la intención específica de encontrar y remover todos los errores. Para encontrar el mayor número posible de éstos, las pruebas deben realizarse de manera sistemática y deben diseñarse casos de prueba usando técnicas sistematizadas.

¿Cuáles son los pasos? Para aplicaciones convencionales, el software se prueba desde dos perspectivas diferentes: 1) la lógica de programa interno se revisa usando técnicas de diseño de casos de prueba de "caja blanca" y 2) los requerimientos de software se revisan usando técnicas de diseño de casos de prueba de "caja negra". El uso de casos auxilia en el diseño de pruebas para descubrir errores de validación del software. En todo caso, la intención es encontrar el máximo número de errores con la mínima cantidad de esfuerzo y tiempo.

¿Cuál es el producto final? Se diseña y documenta un conjunto de casos de prueba elaborados para revisar la lógica interna, las interfaces, las colaboraciones de componentes y los requerimientos externos; se definen los resultados esperados y se registran los resultados reales.

¿Cómo me aseguro de que lo hice bien? Cuando se realizan pruebas, cambia el punto de vista. ¡Intente con ahínco "romper" el software! Diseñe casos de prueba en forma sistemática y revise minuciosamente los casos de prueba creados. Además, puede evaluar la cobertura de la prueba y rastrear las actividades de detección de errores.

18.1 FUNDAMENTOS DE LAS PRUEBAS DEL SOFTWARE

Cita:

"Todo programa hace algo bien, sólo que puede no ser aquello que queremos que haga."

Anónimo

La meta de probar es encontrar errores, y una buena prueba es aquella que tiene una alta probabilidad de encontrar uno. Por tanto, un sistema basado en computadora o un producto debe diseñarse e implementarse teniendo en mente la "comprobabilidad". Al mismo tiempo, las pruebas en sí mismas deben mostrar un conjunto de características que logren la meta de encontrar la mayor cantidad de errores con el mínimo esfuerzo.

Comprobabilidad. James Bach[1] proporciona la siguiente definición de comprobabilidad: "La *comprobabilidad del software* significa simplemente saber con cuánta facilidad puede probarse [un programa de cómputo]." Las siguientes características conducen a software comprobable.

? ¿Cuáles son las características de la comprobabilidad?

Operatividad. "Mientras mejor funcione, más eficientemente puede probarse." Si un sistema se diseña e implementa teniendo como objetivo la calidad, relativamente pocos errores bloquearán la ejecución de las pruebas, lo que permitirá avanzar en ellas sin interrupciones.

Observabilidad. "Lo que ve es lo que prueba." Las entradas proporcionadas como parte de las pruebas producen distintas salidas. Los estados del sistema y las variables son visibles o consultables durante la ejecución. La salida incorrecta se identifica con facilidad. Los errores internos se detectan y se reportan de manera automática. El código fuente es accesible.

Controlabilidad. "Mientras mejor pueda controlar el software, más podrá automatizar y optimizar las pruebas." Todas las salidas posibles pueden generarse a través de alguna combinación de entradas, y los formatos de entrada/salida (E/S) son consistentes y estructurados. Todo código es ejecutable a través de alguna combinación de entradas. El ingeniero de pruebas puede controlar directamente los estados del software, del hardware y las variables. Las pruebas pueden especificarse, automatizarse y reproducirse convenientemente.

Descomponibilidad. "Al controlar el ámbito de las pruebas, es posible aislar más rápidamente los problemas y realizar pruebas nuevas y más inteligentes." El sistema de software se construye a partir de módulos independientes que pueden probarse de manera independiente.

Cita:

"Los errores son más comunes, más dominantes y más problemáticos en software que en otras tecnologías."

David Parnas

Simplicidad. "Mientras haya menos que probar, más rápidamente se le puede probar." El programa debe mostrar *simplicidad funcional* (por ejemplo, el conjunto característico es el mínimo necesario para satisfacer los requerimientos); *simplicidad estructural* (la arquitectura es modular para limitar la propagación de fallos) y *simplicidad de código* (se adopta un estándar de codificación para facilitar la inspección y el mantenimiento).

Estabilidad. "Mientras menos cambios, menos perturbaciones para probar." Los cambios al software son raros, se controlan cuando ocurren y no invalidan las pruebas existentes. El software se recupera bien de los fallos.

Comprensibilidad. "Mientras más información se tenga, se probará con más inteligencia." El diseño arquitectónico y las dependencias entre componentes internos, externos y compartidos son bien comprendidos. La documentación técnica es accesible al instante, está bien organizada, es específica, detallada y precisa. Los cambios al diseño son comunicados a los examinadores.

Pueden usarse los atributos sugeridos por Bach para desarrollar una configuración de software (es decir, programas, datos y documentos) que sean fáciles de probar.

Características de la prueba. ¿Y qué hay acerca de las pruebas en sí? Kaner, Falk y Nguyen [Kan93] sugieren los siguientes atributos de una "buena" prueba:

1 Los párrafos que siguen se usan con permiso de James Bach (copyright 1994) y se adaptaron de material que originalmente apareció en un comentario en el grupo de noticias comp.software-eng.

¿Qué es una "buena" prueba?

Una buena prueba tiene una alta probabilidad de encontrar un error. Para lograr esta meta, el examinador debe comprender el software e intentar desarrollar una imagen mental de cómo puede fallar. De manera ideal, se prueban las clases de fallas. Por ejemplo, una clase de fallas potenciales en una interfaz gráfica de usuario es la falla para reconocer la posición adecuada del ratón. Se diseña entonces un conjunto de pruebas para revisar el ratón con la intención de demostrar un error en el reconocimiento de la posición del ratón.

Una buena prueba no es redundante. El tiempo y los recursos de la prueba son limitados. No se trata de realizar una prueba que tenga el mismo propósito que otra. Cada una debe tener un propósito diferente (incluso si es sutilmente diferente).

Una buena prueba debe ser "la mejor de la camada" [Kan93]. En un grupo de pruebas que tengan una intención similar, las limitaciones de tiempo y recursos pueden mitigar la ejecución de sólo un subconjunto de dichas pruebas. En tales casos, debe usarse la prueba que tenga la mayor probabilidad de descubrir toda una clase de errores.

Una buena prueba no debe ser demasiado simple o demasiado compleja. Aunque en ocasiones es posible combinar una serie de pruebas en un caso de prueba, los efectos colaterales posibles asociados con este enfoque pueden enmascarar errores. En general, cada prueba debe ejecutarse por separado.

CASASEGURA



Diseño de pruebas únicas

La escena: Cubículo de Vinod.

Participantes: Vinod y Ed, miembros del equipo de ingeniería de software CasaSegura.

La conversación:

Vinod: Así que éstos son los casos de prueba que quieres aplicar para la operación *passwordValidation*.

Ed: Sí, deben cubrir muchas de las posibilidades para los tipos de contraseñas que pueda ingresar un usuario.

Vinod: Veamos... observas que la contraseña correcta será 8080, ¿verdad?

Ed: Ajá.

Vinod: ¿Y especificas las contraseñas 1234 y 6789 para probar el error en el reconocimiento de las contraseñas inválidas?

Ed: Exacto, y también pruebo las contraseñas que están cerca de la contraseña correcta, a ver... 8081 y 8180.

Vinod: Ésos están bien, pero no le veo mucho caso aplicar las entradas 1234 y 6789. Son redundantes... prueban la misma cosa, ¿o no?

Ed: Bueno, son valores diferentes.

Vinod: Es cierto, pero si 1234 no descubre un error... en otras palabras... la operación *passwordValidation* detecta que es una contraseña inválida, no es probable que 6789 nos muestre algo nuevo.

Ed: Ya veo lo que dices.

Vinod: No intento ser quisquilloso... es sólo que tenemos tiempo limitado para hacer las pruebas, así que es buena idea aplicar pruebas que tengan una alta probabilidad de encontrar nuevos errores.

Ed: No hay problema... Pensaré en esto un poco más.

18.2 VISIONES INTERNA Y EXTERNA DE LAS PRUEBAS

Cita:

"Sólo hay una regla en el diseño de casos de prueba: cubrir todas las características, mas no hacer demasiados casos de prueba."

Tsuneo Yamaura

Cualquier producto sometido a ingeniería (y la mayoría de otras cosas) pueden probarse en una de dos formas: 1) al conocer la función específica que se asignó a un producto para su realización, pueden llevarse a cabo pruebas que demuestren que cada función es completamente operativa mientras al mismo tiempo se buscan errores en cada función, 2) al conocer el funcionamiento interno de un producto, pueden realizarse pruebas para garantizar que "todos los engranes embonan"; es decir, que las operaciones internas se realizan de acuerdo con las especificaciones y que todos los componentes internos se revisaron de manera adecuada. El primer

enfoque de pruebas considera una visión externa y se llama prueba de caja negra. El segundo requiere una visión interna y se denomina prueba de caja blanca.[2]

PUNTO CLAVE

Las pruebas de caja blanca pueden diseñarse sólo después de que existe el diseño a nivel de componentes (o código fuente). Debe disponerse de los detalles lógicos del programa.

La *prueba de caja negra* se refiere a las pruebas que se llevan a cabo en la interfaz del software. Una prueba de caja negra examina algunos aspectos fundamentales de un sistema con poca preocupación por la estructura lógica interna del software. La *prueba de caja blanca* del software se basa en el examen cercano de los detalles de procedimiento. Las rutas lógicas a través del software y las colaboraciones entre componentes se ponen a prueba al revisar conjuntos específicos de condiciones y/o bucles.

A primera vista, parecería que las pruebas de caja blanca muy extensas conducirían a "programas 100 por ciento correctos". Lo único que se necesita es definir todas las rutas lógicas, desarrollar casos de prueba para revisarlas y evaluar resultados, es decir, generar casos de prueba para revisar de manera exhaustiva la lógica del programa. Por desgracia, las pruebas exhaustivas presentan ciertos problemas logísticos. Hasta para programas pequeños, el número de posibles rutas lógicas puede ser muy grande. Sin embargo, las pruebas de caja blanca no deben descartarse como imprácticas. Puede seleccionarse y revisarse un número limitado de rutas lógicas importantes. Puede probarse la validez de las estructuras de datos importantes.

INFORMACIÓN

Pruebas exhaustivas

Considere un programa de 100 líneas en el lenguaje C. Después de alguna declaración básica de datos, el programa contiene dos bucles anidados que se ejecutan de 1 a 20 veces cada uno, dependiendo de las condiciones especificadas en la entrada. Dentro del bucle interior, se requieren cuatro constructos if-then-else. ¡Existen aproximadamente 10^{14} rutas posibles que pueden ejecutarse en este programa!

Para poner este número en perspectiva, suponga que se desarrolló un procesador de prueba mágico ("mágico" porque no existe tal procesador) para realizar pruebas exhaustivas. El procesador puede desarrollar un caso de prueba, ejecutarlo y evaluar los resultados en un milisegundo. Si trabajara 24 horas al día los 365 días del año, el procesador trabajaría durante 3 170 años para probar el programa. Esto, sin duda alguna, causaría estragos en la mayoría de los calendarios de desarrollo.

Por tanto, es razonable afirmar que la prueba exhaustiva es imposible para sistemas de software grandes.

18.3 PRUEBA DE CAJA BLANCA

Cita:

"Los errores se esconden en las esquinas y se congregan en las fronteras."

Boris Beizer

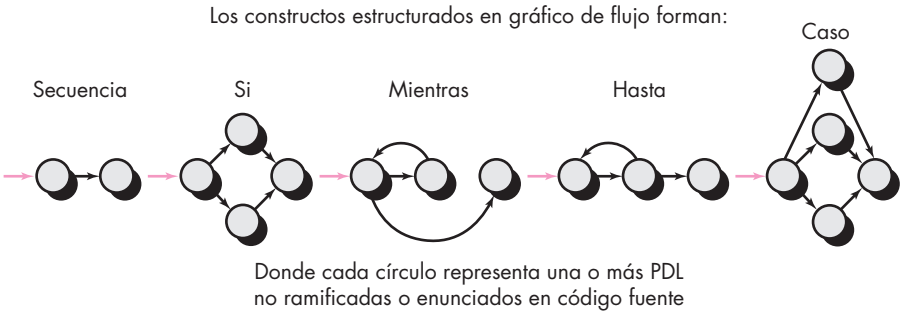
La *prueba de caja blanca*, en ocasiones llamada *prueba de caja de vidrio*, es una filosofía de diseño de casos de prueba que usa la estructura de control descrita como parte del diseño a nivel de componentes para derivar casos de prueba. Al usar los métodos de prueba de caja blanca, puede derivar casos de prueba que: 1) garanticen que todas las rutas independientes dentro de un módulo se revisaron al menos una vez, 2) revisen todas las decisiones lógicas en sus lados verdadero y falso, 3) ejecuten todos los bucles en sus fronteras y dentro de sus fronteras operativas y 4) revisen estructuras de datos internas para garantizar su validez.

18.4 PRUEBA DE RUTA BÁSICA

La *prueba de ruta* o *trayectoria básica* es una técnica de prueba de caja blanca propuesta por primera vez por Tom McCabe [McC76]. El método de ruta básica permite al diseñador de casos de prueba derivar una medida de complejidad lógica de un diseño de procedimiento y usar esta

2 En ocasiones, en lugar de pruebas de caja negra y de caja blanca, se usan, respectivamente, los términos *prueba funcional* y *prueba estructural*.

FIGURA 18.1
Notación de gráfico de flujo



medida como guía para definir un conjunto básico de rutas de ejecución. Los casos de prueba derivados para revisar el conjunto básico tienen garantía para ejecutar todo enunciado en el programa, al menos una vez durante la prueba.

18.4.1 Notación de gráfico o grafo de flujo

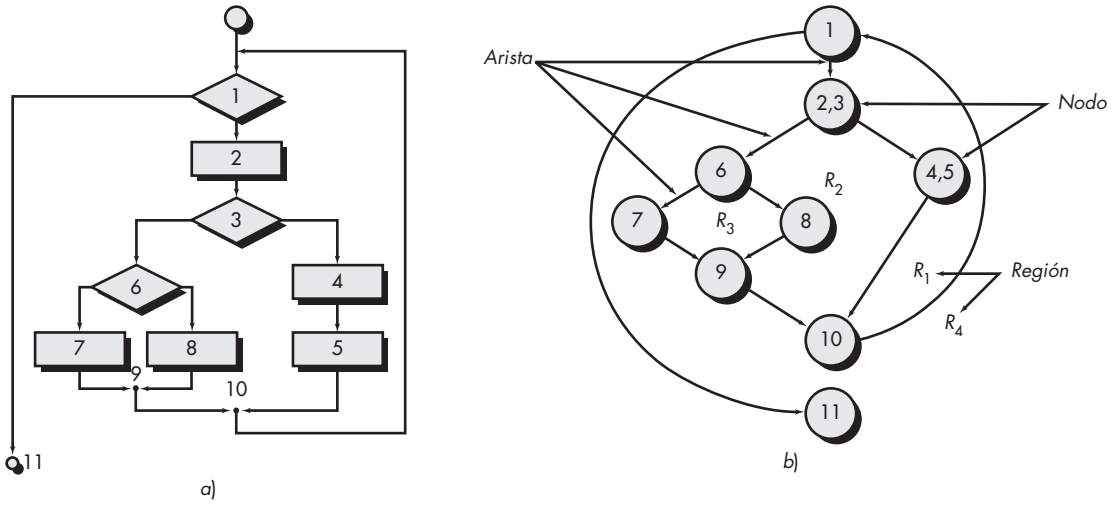
Antes de considerar el método de ruta básica, debe introducirse una notación simple para la representación del flujo de control, llamado *gráfico de flujo* (o *gráfico de programa*).[3] El gráfico de flujo muestra el flujo de control lógico que usa la notación ilustrada en la figura 18.1. Cada constructo estructurado (capítulo 10) tiene un correspondiente símbolo de gráfico de flujo.

CONSEJO

Un gráfico de flujo debe dibujarse sólo cuando la estructura lógica de un componente es compleja. El gráfico de flujo le permite rastrear rutas de programa con más facilidad.

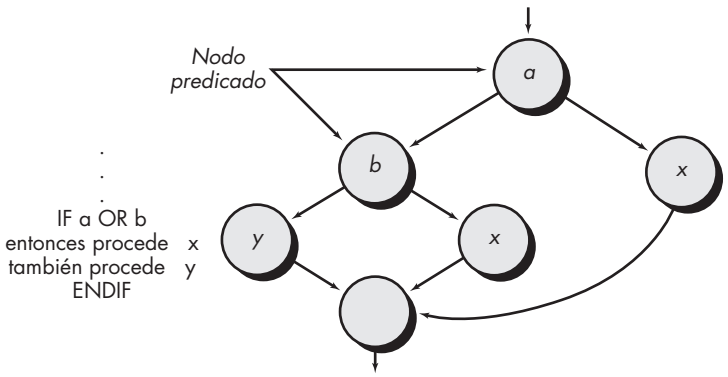
Para ilustrar el uso de un gráfico de flujo, considere la representación del diseño de procedimiento en la figura 18.2*a*). Aquí se usó un diagrama de flujo para mostrar la estructura de control del programa. La figura 18.2*b*) mapea el diagrama de flujo en un gráfico de flujo correspondiente (suponiendo que el diagrama de flujo no contiene condiciones compuestas en los diamantes de decisión). Con referencia a la figura 18.2*b*), cada círculo, llamado *nodo de gráfico de flujo*, representa uno o más enunciados de procedimiento. Una secuencia de cajas de proceso y un diamante de decisión pueden mapearse en un solo nodo. Las flechas en el gráfico de flujo, llamadas *aristas* o *enlaces*, representan flujo de control y son análogas a las flechas en el diagrama de flujo. Una arista debe terminar en un nodo, incluso si el nodo no representa algún enunciado de pro-

FIGURA 18.2
***a*) Diagrama de flujo y *b*) gráfico de flujo**



3 En la actualidad, el método de ruta básica puede realizarse sin el uso de gráficos de flujo. No obstante, sirven como una notación útil para comprender el flujo de control e ilustrar el enfoque.

FIGURA 18.3
Lógica compuesta



cedimiento (por ejemplo, vea el símbolo de gráfico de flujo para el constructo if-then-else). Las áreas acotadas por aristas y nodos se llaman *regiones*. Cuando se cuentan las regiones, el área afuera del gráfico se incluye como región.[4]

Cuando en un diseño de procedimiento se encuentran condiciones compuestas, la generación de un gráfico de flujo se vuelve ligeramente más complicada. Una condición compuesta ocurre cuando uno o más operadores booleanos (OR, AND, NAND, NOR lógicos) se presenta en un enunciado condicional. En la figura 18.3, el segmento en lenguaje de diseño de programa (PDL, por sus siglas en inglés) se traduce en el gráfico de flujo mostrado. Observe que se crea un nodo separado para cada una de las condiciones *a* y *b* en el enunciado IF *a* OR *b*. Cada nodo que contiene una condición se llama *nodo predicado* y se caracteriza por dos o más aristas que emanan de él.

18.4.2 Rutas de programa independientes

Una *ruta independiente* es cualquiera que introduce al menos un nuevo conjunto de enunciados de procesamiento o una nueva condición en el programa. Cuando se establece como un gráfico de flujo, una ruta independiente debe moverse a lo largo de al menos una arista que no se haya recorrido antes de definir la ruta. Por ejemplo, un conjunto de rutas independientes para el gráfico de flujo que se ilustra en la figura 18.2*b*) es

ruta 1: 1-11

ruta 2: 1-2-3-4-5-10-1-11

ruta 3: 1-2-3-6-8-9-10-1-11

ruta 4: 1-2-3-6-7-9-10-1-11

Observe que cada nueva ruta introduce una nueva arista. La ruta

1-2-3-4-5-10-1-2-3-6-8-9-10-1-11

no se considera como independiente porque simplemente es una combinación de rutas ya especificadas y no recorre alguna arista nueva.

Las rutas de la 1 a la 4 constituyen un *conjunto básico* para el gráfico de flujo de la figura 18.2*b*). Es decir, si se pueden diseñar pruebas para forzar la ejecución de estas rutas (un conjunto básico), todo enunciado en el programa tendrá garantizada su ejecución al menos una vez, y cada condición se ejecutará en sus lados verdadero y falso. Debe señalarse que el con-

4 En la sección 18.6.1 se presenta un análisis más detallado de los gráficos y sus usos.

junto básico no es único. De hecho, para un diseño de procedimiento dado, pueden derivarse algunos conjuntos básicos diferentes.

¿Cómo saber cuántas rutas buscar? El cálculo de la complejidad ciclomática proporciona la respuesta. La *complejidad ciclomática* es una medición de software que proporciona una evaluación cuantitativa de la complejidad lógica de un programa. Cuando se usa en el contexto del método de prueba de la ruta básica, el valor calculado por la complejidad ciclomática define el número de rutas independientes del conjunto básico de un programa y le brinda una cota superior para el número de pruebas que debe realizar a fin de asegurar que todos los enunciados se ejecutaron al menos una vez.

CONSEJO

La complejidad ciclomática es una medición útil para predecir aquellos módulos proclives al error. Úsela para planificar las pruebas así como para el diseño de casos de prueba.

? ¿Cómo se calcula la complejidad ciclomática?

La complejidad ciclomática tiene fundamentos en la teoría de gráficos y proporciona una medición de software extremadamente útil. La complejidad se calcula en una de tres formas:

1. El número de regiones del gráfico de flujo corresponde a la complejidad ciclomática.
2. La complejidad ciclomática $V(G)$ para un gráfico de flujo G se define como

   $V(G) = E - N + 2$

   donde E es el número de aristas del gráfico de flujo y N el número de nodos del gráfico de flujo.
3. La complejidad ciclomática $V(G)$ para un gráfico de flujo G también se define como

   $V(G) = P + 1$

   donde P es el número de nodos predicado contenidos en el gráfico de flujo G.

PUNTO CLAVE

La complejidad ciclomática proporciona la cota superior sobre el número de casos de prueba que se requerirán para garantizar que cada enunciado en el programa se ejecuta al menos una vez.

En el gráfico de flujo de la figura 18.2*b*), la complejidad ciclomática puede calcularse usando cada uno de los algoritmos recién indicados:

1. El gráfico de flujo tiene cuatro regiones.
2. $V(G) = 11$ aristas $- 9$ nodos $+ 2 = 4$.
3. $V(G) = 3$ nodos predicado $+ 1 = 4$.

Por tanto, la complejidad ciclomática del gráfico de flujo en la figura 18.2*b*) es 4.

Más importante, el valor para $V(G)$ proporciona una cota superior para el número de rutas independientes que forman el conjunto básico y, por implicación, una cota superior sobre el número de pruebas que deben diseñarse y ejecutarse para garantizar cobertura de todos los enunciados del programa.

CasaSegura



Uso de la complejidad ciclomática

La escena: Cubículo de Shakira.

Participantes: Vinod y Shakira, miembros del equipo de ingeniería del software CasaSegura, quienes trabajan en la planificación de las pruebas para la función de seguridad.

La conversación:

Shakira: Mira... Sé que debemos hacer pruebas de unidad a todos los componentes para la función de seguridad, pero hay muchos de ellos, y si consideras el número de operaciones que se tienen que revisar, no sé... tal vez deberíamos olvidar la prueba de caja blanca, integrar todo y comenzar a aplicar las pruebas de caja negra.

Vinod: ¿Supones que no tenemos tiempo suficiente para hacer pruebas de componentes, revisar las operaciones y luego integrar?

Shakira: La fecha límite para el primer incremento está más cerca de lo que quisiera... sí, estoy preocupada.

Vinod: ¿Por qué al menos no aplicas pruebas de caja blanca sobre las operaciones que tienen probabilidad de ser más proclives a errores?

Shakira (exasperada): ¿Y exactamente cómo sé cuáles son las más proclives a errores?

Vinod: *V* de *G*.

Shakira: ¿Qué?

Vinod: Complejidad ciclomática, *V* de *G*. Sólo calcula *V*(*G*) para cada una de las operaciones dentro de cada uno de los componentes y ve cuáles tienen los valores más altos para *V*(*G*). Ésas son las que tienen más probabilidad de ser proclives a errores.

Shakira: ¿Y cómo calculo *V* de *G*?

Vinod: Realmente es sencillo. Aquí hay un libro que describe cómo hacerlo.

Shakira (hojea el libro): Muy bien, no parece difícil. Lo intentaré. Las operaciones con la *V*(*G*) más altas serán las candidatas para las pruebas de caja blanca.

Vinod: Sólo recuerda que no hay garantías. Un componente con una *V*(*G*) baja puede ser proclive a errores.

Shakira: Bien. Pero al menos esto me ayudará a reducir el número de componentes que tienen que experimentar pruebas de caja blanca.

18.4.3 Derivación de casos de prueba

Cita:

"El cohete Ariane 5 estalló en el despegue debido exclusivamente a un defecto de software (un bug, un error) que involucra la conversión de un valor en punto flotante de 64 bits en un entero de 16 bits. El cohete y sus cuatro satélites no estaban asegurados y valían 500 millones de dólares. [Pruebas de ruta que revisaran la ruta de conversión] habrían descubierto el error, pero se vetaron por razones presupuestarias."

Reporte noticioso

El método de prueba de ruta básica puede aplicarse a un diseño de procedimientos o a un código fuente. En esta sección se presenta la prueba de ruta básica como una serie de pasos. El procedimiento *average* (promedio), que en la figura 18.4 se muestra en PDL, se usará como ejemplo para ilustrar cada paso del método de diseño de caso de prueba. Observe que *average*, aunque es un algoritmo extremadamente simple, contiene condiciones y bucles compuestos. Es posible aplicar los siguientes pasos para derivar el conjunto básico:

1. **Al usar el diseño o el código como cimiento, dibuje el gráfico de flujo correspondiente.** El gráfico de flujo se crea usando los símbolos y reglas de construcción que se presentaron en la sección 18.4.1. En el PDL para *average* de la figura 18.4, el gráfico de flujo se crea al numerar aquellos enunciados PDL que se mapearán en los nodos correspondientes del gráfico de flujo. En la figura 18.5 se muestra el gráfico de flujo correspondiente.
2. **Determine la complejidad ciclomática del gráfico de flujo resultante.** La complejidad ciclomática *V*(*G*) se determina al aplicar los algoritmos descritos en la sección 18.4.2. Debe observarse que *V*(*G*) puede determinarse sin desarrollar un gráfico de flujo,

FIGURA 18.4
PDL con identificación de nodos

```
PROCEDIMIENTO average;

    *  Este procedimiento calcula el promedio de 100 o menos
       números que se encuentran entre valores frontera; también
       calcula la suma y el número total válido.

    INTERFACE RETURNS average, total.input, total.valid;
    INTERFACE ACCEPTS value, minimum, maximum;

    TYPE value[1:100] IS SCALAR ARRAY;
    TYPE average, total.input, total.valid;
       minimum, maximum, sum IS SCALAR;
    TYPE i IS INTEGER;
1   i = 1;
    total.input = total.valid = 0;
    sum = 0;
    DO WHILE value[i] <> –999 AND total.input < 100   3
4      increment total.input by 1;
       IF value[i] > = minimum AND value[i] < = maximum   6
5         THEN increment total.valid by 1;
7                sum = s sum + value[i]
          ELSE skip
8      ENDIF
       increment i by 1;
9   ENDDO
       IF total.valid > 0   10
11        THEN average = sum / total.valid;
12        ELSE average = –999;
13  ENDIF
END average
```

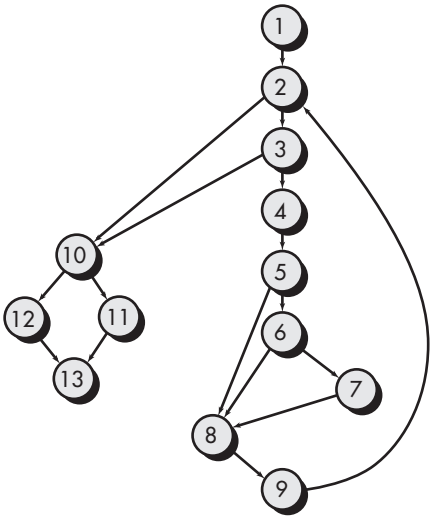


FIGURA 18.5
Gráfico de flujo para el procedimiento *average*

al contar todos los enunciados condicionales en el PDL (para el procedimiento *average*, las condiciones compuestas son dos) y sumar 1. En la figura 18.5,

$V(G)$ = 6 regiones
$V(G)$ = 17 aristas − 13 nodos + 2 = 6
$V(G)$ = 5 nodos predicado + 1 = 6

3. **Determine un conjunto básico de rutas linealmente independientes.** El valor de $V(G)$ proporciona la cota superior sobre el número de rutas linealmente independientes a través de la estructura de control del programa. En el caso del procedimiento *average*, se espera especificar seis rutas:

   ruta 1: 1-2-10-11-13

   ruta 2: 1-2-10-12-13

   ruta 3: 1-2-3-10-11-13

   ruta 4: 1-2-3-4-5-8-9-2-...

   ruta 5: 1-2-3-4-5-6-8-9-2-...

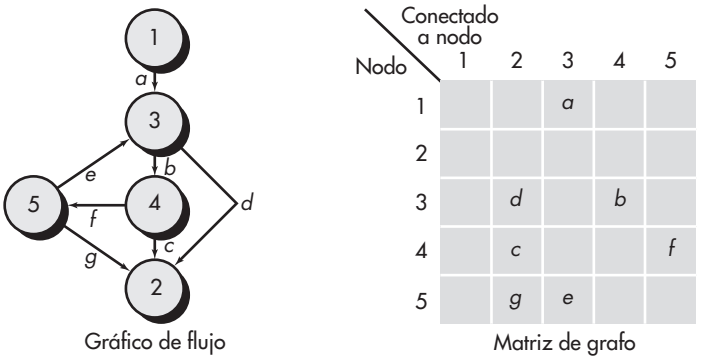
   ruta 6: 1-2-3-4-5-6-7-8-9-2-...

   La elipsis (...) después de las rutas 4, 5 y 6 indica que es aceptable cualquier ruta a través del resto de la estructura de control. Con frecuencia, vale la pena identificar nodos predicado como un auxiliar para la derivación de los casos de prueba. En este caso, los nodos 2, 3, 5, 6 y 10 son nodos predicado.

4. **Prepare casos de prueba que fuercen la ejecución de cada ruta en el conjunto básico.** Los datos deben elegirse de modo que las condiciones en los nodos predicado se establezcan de manera adecuada conforme se prueba cada ruta. Cada caso de prueba se ejecuta y compara con los resultados esperados. Una vez completados todos los casos de prueba, el examinador puede estar seguro de que todos los enunciados del programa se ejecutaron al menos una vez.

Es importante notar que algunas rutas independientes (tomemos por caso la ruta 1 del ejemplo) no pueden probarse en forma individual, es decir, la combinación de datos requerida para recorrer la ruta no puede lograrse en el flujo normal del programa. En tales casos, dichas rutas se prueban como parte de otra prueba de ruta.

FIGURA 18.6
Matriz de grafo



18.4.4 Matrices de grafo

El procedimiento para derivar el gráfico de flujo e incluso determinar un conjunto de rutas básicas es sensible a la mecanización. Una estructura de datos, llamada *matriz de un grafo,* puede ser bastante útil para desarrollar una herramienta de software que auxilie en la prueba de ruta básica.

Una matriz de grafo es una matriz cuadrada cuyo tamaño (es decir, número de filas y columnas) es igual al número de nodos del gráfico de flujo. Cada fila y columna corresponde a un nodo identificado y las entradas de la matriz corresponden a conexiones (una arista) entre nodos. En la figura 18.6 se muestra un ejemplo simple de gráfico de flujo y su correspondiente matriz de grafo [Bei90].

En esa figura, cada nodo en el gráfico de flujo se identifica mediante números, mientras que cada arista se identifica con letras. Una entrada de letra en la matriz corresponde a una conexión entre dos nodos. Por ejemplo, el nodo 3 se conecta con el nodo 4 mediante la arista *b*.

? ¿Qué es una matriz de grafo y cómo se le extiende para su uso en las pruebas?

En este punto, la matriz de grafo no es más que una representación tabular de un gráfico de flujo. Sin embargo, al agregar un enlace ponderado a cada entrada de matriz, la matriz de grafo puede convertirse en una poderosa herramienta para evaluar durante las pruebas la estructura de control del programa. El *enlace ponderado* proporciona información adicional acerca del flujo de control. En su forma más simple, el enlace ponderado es 1 (existe una conexión) o 0 (no existe conexión). Pero a los enlaces ponderados puede asignárseles otras propiedades más interesantes:

- La probabilidad de que un enlace (arista) se ejecutará.
- El tiempo de procesamiento que se emplea durante el recorrido de un enlace.
- La memoria requerida durante el recorrido de un enlace.
- Los recursos requeridos durante el recorrido de una prueba.

Beizer [Bei90] proporciona un tratamiento a fondo de algoritmos matemáticos adicionales que pueden aplicarse a las matrices gráficas. Con estas técnicas, el análisis requerido para diseñar casos de prueba puede ser parcial o completamente automatizado.

18.5 PRUEBA DE LA ESTRUCTURA DE CONTROL

La técnica de prueba de ruta básica descrita en la sección 18.4 es una de varias técnicas para probar la estructura de control. Aunque la prueba de ruta básica es simple y enormemente efectiva, no es suficiente en sí misma. En esta sección se estudian otras variaciones acerca de la prueba de la estructura de control. Esta prueba más amplia cubre y mejora la calidad de la prueba de caja blanca.

Cita:

"Poner más atención a la aplicación de las pruebas que a su diseño es un error clásico."

Brian Marick

18.5.1 Prueba de condición

La *prueba de condición* [Tai89] es un método de diseño de casos de prueba que revisa las condiciones lógicas contenidas en un módulo de programa. Una condición simple es una variable booleana o una expresión relacional, posiblemente precedida de un operador NOT (¬). Una expresión relacional toma la forma

E_1 <operador relacional> E_2

donde E_1 y E_2 son expresiones aritméticas y <operador relacional> es uno de los siguientes: $<$, $\leq$, $=$, $\neq$ (no igualdad), $>$ o $\geq$. Una *condición compuesta* se integra con dos o más condiciones simples, operadores booleanos y paréntesis. Se supone que los operadores booleanos permitidos en una condición compuesta incluyen OR (|), AND (&) y NOT (¬). Una condición sin expresiones relacionales se conoce como expresión booleana.

PUNTO CLAVE

Los errores son mucho más comunes en la cercanía de las condiciones lógicas que en el lugar de los enunciados de procesamiento secuencial.

Si una condición es incorrecta, entonces al menos un componente de la condición es incorrecto. Por tanto, los tipos de errores en una condición incluyen errores de operador booleano (operadores booleanos incorrectos/perdidos/adicionales), de variable booleana, de paréntesis booleanos, de operador relacional y de expresión aritmética. El método de prueba de condición se enfoca en la prueba de cada condición del programa para asegurar que no contiene errores.

Cita:

"Los buenos examinadores son maestros para notar 'algo divertido' y actuar sobre ello."

Brian Marick

18.5.2 Prueba de flujo de datos

El método de prueba de flujo de datos [Fra93] selecciona rutas de prueba de un programa de acuerdo con las ubicaciones de las definiciones y con el uso de variables en el programa. Para ilustrar el enfoque de prueba de flujo de datos, suponga que a cada enunciado en un programa se le asigna un número de enunciado único y que cada función no modifica sus parámetros o variables globales. Para un enunciado con *S* como su número de enunciado,

DEF(*S*) = {*X* | enunciado *S* contiene una definición de *X*}
USE(*S*) = {*X* | enunciado *S* contiene un uso de *X*}

Si el enunciado *S* es un *enunciado if* o *loop*, su conjunto DEF es vacío y su conjunto USE se basa en la condición del enunciado *S*. Se dice que la definición de la variable *X* en el enunciado *S* está *viva* en el enunciado *S*′ si existe una ruta desde el enunciado *S* hasta el enunciado *S*′ que no contiene otra definición de *X*.

CONSEJO

Es irreal suponer que la prueba de flujo de datos se usará de manera extensa cuando se prueba un sistema grande. Sin embargo, puede usarse en forma dirigida para áreas de software que sean sospechosas.

Una *cadena de definición de uso (DU)* de la variable *X* es de la forma [*X, S, S*′], donde *S* y *S*′ son números de enunciado, *X* está en DEF(*S*) y en USE(*S*′), y la definición de *X* en el enunciado *S* está viva en el enunciado *S*′.

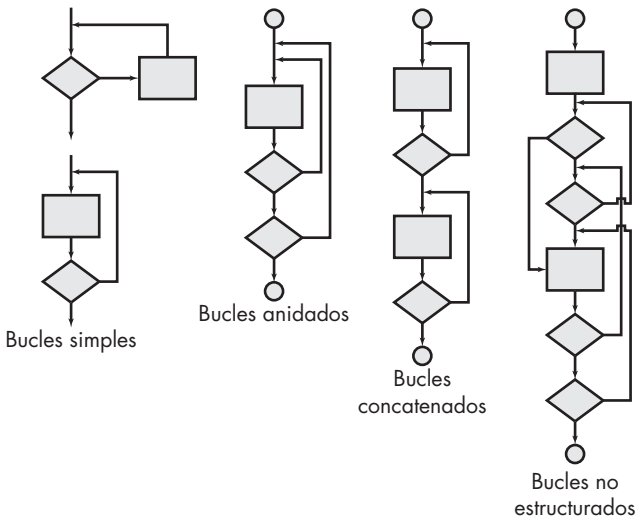
Una estrategia de prueba de flujo de datos simple es requerir que toda cadena DU se cubra al menos una vez. A esta estrategia se le conoce como estrategia de prueba DU. Se ha demostrado que la prueba DU no garantiza la cobertura de todas las ramas de un programa. Sin embargo, la prueba DU no garantiza la cobertura de una rama sólo en raras situaciones, como en los constructos if-then-else en los cuales la *parte then* no tiene definición de alguna variable y la *parte else* no existe. En esta situación, la rama *else* del enunciado *if* no necesariamente se cubre con la prueba DU.

18.5.3 Prueba de bucle

Los bucles son la piedra de toque de la gran mayoría de todos los algoritmos implementados en el software. Y aún así, con frecuencia se les pone poca atención mientras se realizan las pruebas de software.

La *prueba de bucle* es una técnica de prueba de caja blanca que se enfoca exclusivamente en la validez de los constructos bucle. Pueden definirse cuatro clases diferentes de bucles [Bei90]: simples, concatenados, anidados y no estructurados (figura 18.7).

FIGURA 18.7
Clases de bucles



Bucles simples. El siguiente conjunto de pruebas puede aplicarse a los bucles simples, donde *n* es el máximo número de pasadas permisibles a través del bucle.

1. Saltar por completo el bucle.
2. Sólo una pasada a través del bucle.
3. Dos pasadas a través del bucle.
4. *m* pasadas a través del bucle, donde $m < n$.
5. $n - 1$, n, $n + 1$ pasadas a través del bucle.

Bucles anidados. Si tuviera que extender el enfoque de la prueba para bucles simples a los bucles anidados, el número de pruebas posibles crecería geométricamente conforme el nivel de anidado aumenta. Esto daría como resultado un número impráctico de pruebas. Beizer [Bei90] sugiere un acercamiento que ayudará a reducir el número de pruebas:

1. Comience con el bucle más interno. Establezca todos los otros bucles a valores mínimos.
2. Realice pruebas de bucle simple para el bucle más interno mientras mantiene los bucles exteriores en sus valores mínimos de parámetro de iteración (por ejemplo, contador de bucle). Agregue otras pruebas para valores fuera-de-rango o excluidos.
3. Trabaje hacia afuera y realice pruebas para el siguiente bucle, pero mantenga los otros bucles exteriores en valores mínimos y los otros bucles anidados en valores "típicos".
4. Continúe hasta que todos los bucles se hayan probado.



No es posible probar los bucles no estructurados de manera efectiva. Se deben refactorizar.

Bucles concatenados. Los bucles concatenados pueden probarse usando el enfoque definido para bucles simples si cada uno de los bucles es independiente de los otros. No obstante, si dos bucles se concatenan y usa el contador de bucle para el bucle 1 como el valor inicial para el bucle 2, entonces los bucles no son independientes. Cuando los bucles no son independientes, se recomienda el enfoque aplicado a bucles anidados.

Bucles no estructurados. Siempre que sea posible, esta clase de bucles debe rediseñarse para reflejar el uso de los constructos de programación estructurada (capítulo 10).

18.6 PRUEBAS DE CAJA NEGRA

Las *pruebas de caja negra*, también llamadas *pruebas de comportamiento*, se enfocan en los requerimientos funcionales del software; es decir, las técnicas de prueba de caja negra le permiten derivar conjuntos de condiciones de entrada que revisarán por completo todos los requerimientos funcionales para un programa. Las pruebas de caja negra no son una alternativa para las técnicas de caja blanca. En vez de ello, es un enfoque complementario que es probable que descubra una clase de errores diferente que los métodos de caja blanca.

Las pruebas de caja negra intentan encontrar errores en las categorías siguientes: 1) funciones incorrectas o faltantes, 2) errores de interfaz, 3) errores en las estructuras de datos o en el acceso a bases de datos externas, 4) errores de comportamiento o rendimiento y 5) errores de inicialización y terminación.

Cita:

"Errar es humano, pero encontrar un error es divino."

Robert Dunn

A diferencia de las pruebas de caja blanca, que se realizan tempranamente en el proceso de pruebas, la prueba de caja negra tiende a aplicarse durante las últimas etapas de la prueba (vea el capítulo 17). Puesto que, a propósito, la prueba de caja negra no considera la estructura de control, la atención se enfoca en el dominio de la información. Las pruebas se diseñan para responder a las siguientes preguntas:

¿Qué preguntas responden las pruebas de caja negra?

- ¿Cómo se prueba la validez funcional?
- ¿Cómo se prueban el comportamiento y el rendimiento del sistema?
- ¿Qué clases de entrada harán buenos casos de prueba?
- ¿El sistema es particularmente sensible a ciertos valores de entrada?
- ¿Cómo se aíslan las fronteras de una clase de datos?
- ¿Qué tasas y volumen de datos puede tolerar el sistema?
- ¿Qué efecto tendrán sobre la operación del sistema algunas combinaciones específicas de datos?

Al aplicar las técnicas de caja negra, se deriva un conjunto de casos de prueba que satisfacen los siguientes criterios [Mye79]: 1) casos de prueba que reducen, por una cuenta que es mayor que uno, el número de casos de prueba adicionales que deben diseñarse para lograr pruebas razonables y 2) casos de prueba que dicen algo acerca de la presencia o ausencia de clases de errores, en lugar de un error asociado solamente con la prueba específica a mano.

18.6.1 Métodos de prueba basados en gráficos

PUNTO CLAVE

Una gráfica representa las relaciones entre objetos datos y objetos programa, lo que permite derivar casos de prueba que buscan errores asociados con dichas relaciones.

El primer paso en la prueba de caja negra es entender los objetos[5] que se modelan en software y las relaciones que conectan a dichos objetos. Una vez logrado esto, el siguiente paso es definir una serie de pruebas que verifiquen "que todos los objetos tengan la relación mutua esperada" [Bei95]. Dicho de otra forma, la prueba de software comienza con la creación de un gráfico de objetos importantes y sus relaciones, y luego diseña una serie de pruebas que cubrirán el gráfico, de modo que cada objeto y relación se revise y se descubran errores.

Para lograr estos pasos, comience por crear un *gráfico*: una colección de *nodos* que representen objetos, *enlaces* que representen las relaciones entre objetos, *nodos ponderados* que describan las propiedades de un nodo (por ejemplo, un valor de datos o comportamiento de estado específicos) y *enlaces ponderados* que describan alguna característica de un enlace.

En la figura 18.8*a*) se muestra la representación simbólica de un gráfico. Los nodos se representan como círculos conectados mediante ligas que tienen algunas formas diferentes. Un en-

5 En este contexto, debe considerar el término *objetos* en el contexto más amplio posible. Abarca objetos de datos, componentes tradicionales (módulos) y elementos orientados a objeto del software de cómputo.

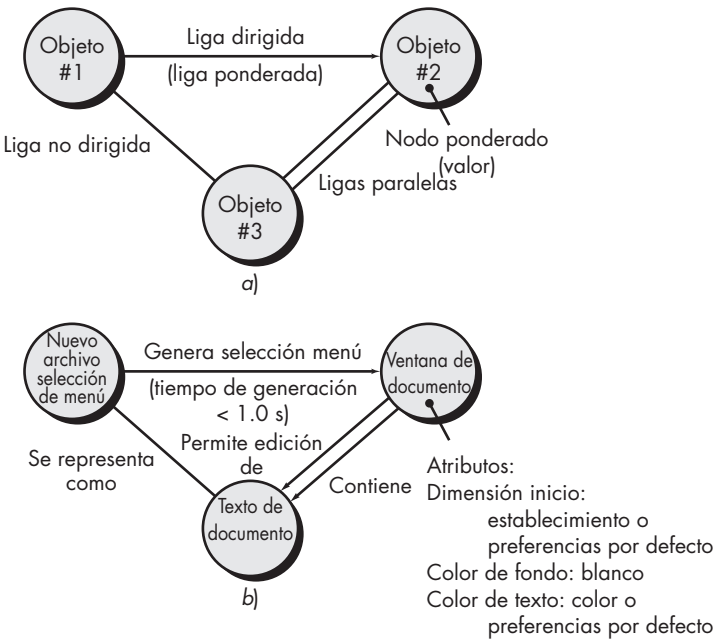


FIGURA 18.8
***a*) Notación de gráfico; *b*) ejemplo simple**

lace dirigido (representado mediante una flecha) indica que una relación sólo se mueve en una dirección. Un *enlace bidireccional*, también llamado *enlace simétrico*, implica que la relación se aplica en ambas direcciones. Los *enlaces paralelos* se usan cuando entre los nodos gráficos se establecen algunas relaciones diferentes.

Como ejemplo simple, considere una porción de un gráfico para una aplicación de un procesador de palabras (figura 18.8*b*) donde

Objeto #1 = **newFile** (selección de menú)

Objeto #2 = **documentWindow**

Objeto #3 = **documentText**

En la figura, una selección de menú en **newFile** genera una ventana de documento. El nodo ponderado de **documentWindow** proporciona una lista de los atributos de ventana que se esperan cuando se genere la ventana. El enlace ponderado indica que la ventana debe generarse en menos de 1.0 segundo. Un enlace no dirigido establece una relación simétrica entre la selección de menú **newFile** y **documentText**, y los enlaces paralelos indican relaciones entre **documentWindow** y **documentText**. En realidad, tendría que generarse un gráfico más detallado como precursor para el diseño de casos de prueba. Entonces podrían derivarse casos de prueba al recorrer el gráfico y cubrir cada una de las relaciones mostradas. Dichos casos de prueba se designan con la intención de encontrar errores en alguna de las relaciones. Beizer [Bei95] describe algunos métodos de prueba de comportamiento que pueden usar gráficos:

Modelado de flujo de transacción. Los nodos representan pasos en alguna transacción (por ejemplo, los pasos requeridos para hacer una reservación en una aerolínea con el uso de un servicio en línea) y los enlaces representan la conexión lógica entre los pasos (por ejemplo, **ingresarInformaciónVuelo** se sigue de *validaciónProcesamientoDisponibilidad*). El diagrama de flujo de datos (capítulo 7) puede usarse para auxiliar en la creación de gráficos de este tipo.

Modelado de estado finito. Los nodos representan diferentes estados del software observables por el usuario (por ejemplo, cada una de las "pantallas" que aparecen cuando un

empleado ingresa información conforme toma una orden telefónica) y los enlaces representan las transiciones que ocurren para moverse de estado a estado (por ejemplo, **pedidoInformación** se verifica durante *inventarioBusquedaDisponibilidad*, y es seguido de la entrada **clienteFacturaInformación**). El diagrama de estado (capítulo 7) puede usarse para auxiliar en la creación de gráficos de este tipo.

Modelado de flujo de datos. Los nodos son objetos datos y los enlaces son las transformaciones que ocurren para traducir un objeto datos en otro. Por ejemplo, el nodo retención de impuesto FICA (**FTW**) se calcula a partir de los ingresos brutos (**IB**), usando la relación **FTW = 0.62 × IB**.

Modelado de temporización. Los nodos son objetos programa y los enlaces son las conexiones secuenciales entre dichos objetos. Los enlaces ponderados se usan para especificar los tiempos de ejecución requeridos conforme se ejecuta el programa.

Un análisis detallado de cada uno de estos métodos de prueba basados en gráfico está más allá del ámbito de este libro. Si se tiene mayor interés, consulte [Bei95] para conocer una cobertura más amplia.

18.6.2 Partición de equivalencia



Las clases de entrada se conocen relativamente pronto en el proceso del software. Por esta razón, se debe pensar acerca de la partición de equivalencia conforme se crea el diseño.

La *partición de equivalencia* es un método de prueba de caja negra que divide el dominio de entrada de un programa en clases de datos de los que pueden derivarse casos de prueba. Un caso de prueba ideal descubre de primera mano una clase de errores (por ejemplo, procesamiento incorrecto de todos los datos carácter) que de otro modo podrían requerir la ejecución de muchos casos de prueba antes de observar el error general.

El diseño de casos de prueba para la partición de equivalencia se basa en una evaluación de las *clases de equivalencia* para una condición de entrada. Con los conceptos introducidos en la sección precedente, si un conjunto de objetos puede vincularse mediante relaciones que son simétricas, transitivas y reflexivas, se presenta una clase de equivalencia [Bei95]. Una clase de equivalencia representa un conjunto de estados válidos o inválidos para condiciones de entrada. Por lo general, una condición de entrada es un valor numérico específico, un rango de valores, un conjunto de valores relacionados o una condición booleana. Las clases de equivalencia pueden definirse de acuerdo con los siguientes lineamientos:

? ¿Cómo se definen las clases de equivalencia para pruebas?

1. Si una condición de entrada especifica un rango, se define una clase de equivalencia válida y dos inválidas.
2. Si una condición de entrada requiere un valor específico, se define una clase de equivalencia válida y dos inválidas.
3. Si una condición de entrada especifica un miembro de un conjunto, se define una clase de equivalencia válida y una inválida.
4. Si una condición de entrada es booleana, se define una clase válida y una inválida.

Al aplicar los lineamientos para la derivación de clases de equivalencia, pueden desarrollarse y ejecutarse los casos de prueba para cada ítem de datos del dominio de entrada. Los casos de prueba se seleccionan de modo que se revise a la vez el número más grande de atributos de una clase de equivalencia.

Cita:

"Una forma efectiva para probar código es revisarlo en sus fronteras naturales."

Brian Kernighan

18.6.3 Análisis de valor de frontera

Un mayor número de errores ocurre en las fronteras del dominio de entrada y no en el "centro". Por esta razón es que el *análisis de valor de frontera* (BVA, del inglés *boundary value analysis*) se desarrolló como una técnica de prueba. El análisis de valor de frontera conduce a una selección de casos de prueba que revisan los valores de frontera.

El análisis de valor de frontera es una técnica de diseño de casos de prueba que complementan la partición de equivalencia. En lugar de seleccionar algún elemento de una clase de equivalencia, el BVA conduce a la selección de casos de prueba en los "bordes" de la clase. En lugar de enfocarse exclusivamente en las condiciones de entrada, el BVA también deriva casos de prueba a partir del dominio de salida [Mye79].

Los lineamientos para el BVA son similares en muchos aspectos a los proporcionados para la partición de equivalencia:

PUNTO CLAVE

El BVA extiende la partición de equivalencia al enfocarse en datos en los "bordes" de una clase de equivalencia.

1. Si una condición de entrada especifica un rango acotado por valores *a* y *b*, los casos de prueba deben designarse con valores *a* y *b*, justo arriba y justo abajo de *a* y *b*.
2. Si una condición de entrada especifica un número de valores, deben desarrollarse casos de prueba que revisen los números mínimo y máximo. También se prueban los valores justo arriba y abajo, mínimo y máximo.
3. Aplicar lineamientos 1 y 2 a condiciones de salida. Por ejemplo, suponga que como salida de un programa de análisis de ingeniería se requiere una tabla de temperatura contra presión. Deben diseñarse casos de prueba para crear un reporte de salida que produzca el número máximo (y mínimo) permisible de entradas de tabla.
4. Si las estructuras de datos de programa internos tienen fronteras prescritas (por ejemplo, una tabla que tenga un límite definido de 100 entradas), asegúrese de diseñar un caso de prueba para revisar la estructura de datos en su frontera.

La mayoría de los ingenieros de software realizan intuitivamente BVA en cierta medida. Al aplicar dichos lineamientos, la prueba de fronteras será más completa y, por tanto, tendrá una mayor probabilidad de detectar errores.

18.6.4 Prueba de arreglo ortogonal

Existen muchas aplicaciones en las cuales el dominio de entrada es relativamente limitado, es decir, el número de parámetros de entrada es pequeño y los valores que cada uno de los parámetros puede tomar están claramente acotados. Cuando dichos números son muy pequeños (por ejemplo, tres parámetros de entrada que toman tres valores discretos cada uno), es posible considerar cada permutación de entrada y probar de manera exhaustiva el dominio de entrada. Sin embargo, conforme crece el número de valores de entrada y el número de valores discretos para cada ítem de datos, la prueba exhaustiva se vuelve impráctica o imposible.

La *prueba de arreglo ortogonal* puede aplicarse a problemas en los que el dominio de entrada es relativamente pequeño pero demasiado grande para alojar la prueba exhaustiva. El método de prueba de arreglo ortogonal es particularmente útil para encontrar los *fallos de región*, una categoría de error asociada con lógica defectuosa dentro de un componente de software.

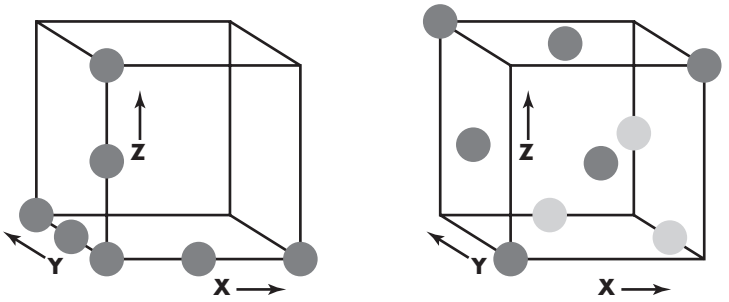
PUNTO CLAVE

La prueba de arreglo ortogonal permite diseñar casos de prueba que proporcionan cobertura máxima de prueba con un número razonable de casos de prueba.

Para ilustrar la diferencia entre prueba de arreglo ortogonal y enfoques más convencionales del tipo "un ítem de entrada a la vez", piense en un sistema que tiene tres ítems de entrada, *X*, *Y* y *Z*. Cada uno tiene tres valores discretos asociados consigo. Existen $3^3 = 27$ posibles casos de prueba. Phadke [Pha97] sugiere una visión geométrica de los posibles casos de prueba asociados con X, Y y Z, que se ilustra en la figura 18.9. En la figura, un ítem de entrada a la vez puede variar en secuencia a lo largo de cada eje de entrada. Esto da como resultado cobertura relativamente limitada del dominio de entrada (representado por el cubo de la izquierda en la figura).

Cuando ocurre la prueba de arreglo ortogonal, se crea un *arreglo ortogonal* L9 de casos de prueba. El arreglo ortogonal L9 tiene una "propiedad de equilibrio" [Pha97]. Es decir, los casos de prueba (representados con puntos oscuros en la figura) se "dispersan de manera uniforme a lo largo de todo el dominio de prueba", como se ilustra en el cubo de la derecha en la figura 18.9. La cobertura de prueba a través del dominio de entrada es más completa.

FIGURA 18.9
Visión geométrica de casos de prueba
Fuente: [Pha97]



Un ítem de entrada a la vez **Arreglo ortogonal L9**

Para ilustrar el uso del arreglo ortogonal L9, considere la función *send* para una aplicación de fax. A la función *send* pasan cuatro parámetros: P1, P2, P3 y P4. Cada uno toma tres valores discretos. Por ejemplo, P1 toma los valores:

P1 = 1, enviar ahora
P1 = 2, enviar una hora más tarde
P1 = 3, enviar después de medianoche

P2, P3 y P4 también tomarían los valores de 1, 2 y 3, que significan otras funciones de envío.

Si se eligiera la estrategia de prueba "un ítem de entrada a la vez", la siguiente secuencia de pruebas (P1, P2, P3, P4) se especificaría: (1, 1, 1, 1), (2, 1, 1, 1), (3, 1, 1, 1), (1, 2, 1, 1), (1, 3, 1, 1), (1, 1, 2, 1), (1, 1, 3, 1), (1, 1, 1, 2) y (1, 1, 1, 3). Phadke [Pha97] valora estos casos de prueba al afirmar:

> Tales casos de prueba son útiles sólo cuando uno está seguro de que estos parámetros de prueba no interactúan. Pueden detectar fallas lógicas donde un solo valor de parámetro genere mal funcionamiento del software. Estas fallas se llaman *fallos de modo individual*. Este método no puede detectar fallos lógicos que causen mal funcionamiento cuando dos o más parámetros toman simultáneamente ciertos valores; es decir, no pueden detectar todas las interacciones. Por tanto, su habilidad para detectar fallas es limitada.

Dado el número relativamente pequeño de los parámetros de entrada y de los valores discretos, es posible la prueba exhaustiva. El número de pruebas requeridas es $3^4 = 81$, grande pero manejable. Se encontrarían todos los fallos asociados con la permutación de ítems de datos, pero el esfuerzo requerido es relativamente elevado.

El enfoque de prueba de arreglo ortogonal permite proporcionar una buena cobertura de pruebas con muchos menos casos de prueba que la estrategia exhaustiva. En la figura 18.10 se ilustra un arreglo ortogonal L9 para la función *send* de fax.

Phadke [Pha97] valora el resultado de las pruebas usando el arreglo ortogonal L9 en la siguiente forma:

> **Detectar y aislar todos los fallos de modo individual.** Un fallo de modo individual es un problema congruente con cualquier nivel de cualquier parámetro individual. Por ejemplo, si todos los casos de prueba del factor P1 = 1 causan una condición de error, se trata de una falla de modo individual. En este ejemplo, las pruebas 1, 2 y 3 [figura 18.10] mostrarán errores. Al analizar la información acerca de qué pruebas muestran errores, uno puede identificar cuáles valores de parámetro causan el fallo. En este ejemplo, al notar que las pruebas 1, 2 y 3 causan un error, uno puede aislar [procesamiento lógico asociado con "enviar ahora" (P1 = 1)] la fuente del error. Tal aislamiento del fallo es importante para corregirlo.
>
> **Detectar todos los fallos de modo doble.** Si existe un problema consistente cuando ocurren en conjunto niveles específicos de dos parámetros, se le llama *fallo de modo doble*. De hecho, un fallo

FIGURA 18.10
Un arreglo ortogonal L9

Caso de prueba	Parámetros de prueba			
	P1	P2	P3	P4
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1

de modo doble es indicio de incompatibilidad pareada o de interacciones dañinas entre dos parámetros de prueba.

Fallos multimodo. Los arreglos ortogonales [del tipo mostrado] sólo pueden garantizar la detección de fallos de modo individual y doble. No obstante, muchos fallos multimodo también son detectables por estas pruebas.

En [Pha89] se puede encontrar un análisis detallado de la prueba de arreglo ortogonal.

HERRAMIENTAS DE SOFTWARE



Diseño de casos de prueba

Objetivo: Auxiliar al equipo de software en el desarrollo de un conjunto completo de casos de prueba tanto para prueba de caja negra como de caja blanca.

Mecánica: Estas herramientas se clasifican en dos categorías amplias: las herramientas de prueba estáticas y las herramientas de prueba dinámicas. En la industria se utilizan tres diferentes tipos de herramientas de prueba estáticas: herramientas de prueba basadas en código, lenguajes de prueba especializados y herramientas de prueba basadas en requerimientos. Las primeras aceptan código fuente como entrada y realizan algunos análisis que dan como resultado la generación de casos de prueba. Los lenguajes de prueba especializados (por ejemplo, ATLAS) permiten al ingeniero de software escribir especificaciones de prueba detalladas que describen cada caso de prueba y la logística para su ejecución. Las herramientas de prueba basadas en requerimientos aíslan requerimientos de usuario específicos y sugieren casos de prueba (o clases de pruebas) que revisarán los requerimientos. Las herramientas de prueba dinámicas interactúan con un programa en ejecución, comprueban la cobertura de ruta, prueban las afirmaciones acerca del valor de variables específicas e instrumentan el flujo de ejecución del programa.

Herramientas representativas:[6]

McCabeTest, desarrollada por McCabe & Associates (**www.mccabe.com**), implementa una variedad de técnicas de prueba de trayectoria derivadas de una valoración de complejidad ciclomática y de otras mediciones de software.

TestWorks, desarrollada por Software Research, Inc. (**www.soft.com/Products**), es un conjunto completo de herramientas de prueba automatizadas que auxilia en el diseño de casos de prueba para software desarrollado en C/C++ y Java, y que proporciona apoyo para pruebas de regresión.

T-VEC Test Generation System, desarrollada por T-VEC Technologies (**www.t-vec.com**), es un conjunto de herramientas que soportan pruebas de unidad, integración y validación al asistir en el diseño de casos de prueba, usando la información contenida en una especificación de requerimientos OO.

e-TEST Suite, desarrollada por Empirix, Inc. (**www.empirix.com**), abarca un conjunto completo de herramientas para probar *webapps*, incluidas herramientas que auxilian en el diseño de casos de prueba y planificación de pruebas.

6 Las herramientas que se mencionan aquí no representan un respaldo, sino una muestra de las herramientas que existen en esta categoría. En la mayoría de los casos, los nombres de las herramientas son marcas registradas por sus respectivos desarrolladores.

18.7 PRUEBA BASADA EN MODELO

Cita:

"Es suficientemente difícil encontrar un error en el código cuando se le busca; pero es todavía más difícil cuando se supone que el código está libre de errores."

Steve McConnell

La *prueba basada en modelo* (PBM) es una técnica de prueba de caja negra que usa la información contenida en el modelo de requerimientos como la base para la generación de casos de prueba. En muchos casos, la técnica de prueba basada en modelo usa diagramas de estado UML, un elemento del modelo de comportamiento (capítulo 7), como la base para el diseño de los casos de prueba.[7] La técnica PBM requiere cinco pasos:

1. **Analizar un modelo de comportamiento existente para el software o crear uno.** Recuerde que un *modelo de comportamiento* indica cómo responderá el software a los eventos o estímulos externos. Para crear el modelo, debe realizar los pasos expuestos en el capítulo 7: 1) evaluar todos los casos de uso para comprender por completo la secuencia de interacción dentro del sistema, 2) identificar los eventos que impulsen la secuencia de interacción y entender cómo dichos eventos se relacionan con objetos específicos, 3) crear una secuencia para cada caso de uso, 4) construir un diagrama de estado UML para el sistema (por ejemplo, véase la figura 7.6), y 5) revisar el modelo de comportamiento para verificar precisión y congruencia.
2. **Recorrer el modelo de comportamiento y especificar las entradas que forzarán al software a realizar la transición de estado a estado.** Las entradas dispararán eventos que harán que ocurra la transición.
3. **Revisar el modelo de comportamiento y observar las salidas esperadas, conforme el software realiza la transición de estado a estado.** Recuerde que cada transición de estado se dispara mediante un evento y que, como consecuencia de la transición, se invoca alguna función y se crean salidas. Para cada conjunto de entradas (casos de prueba) especificado en el paso 2, las salidas esperadas se especifican como se caracterizan en el modelo de comportamiento. "Una suposición fundamental de esta prueba es que existe cierto mecanismo, un *oráculo de prueba*, que determinará si los resultados de una prueba de ejecución son o no correctos" [DAC03]. En esencia, un oráculo de prueba establece la base para cualquier determinación de lo correcto de la salida. En la mayoría de los casos, el oráculo es el modelo de requerimientos, pero también podría ser otro documento o aplicación, datos registrados en cualquier otro lado o, incluso, un experto humano.
4. **Ejecutar los casos de prueba.** Las pruebas pueden ejecutarse manualmente o crearse y ejecutarse un guión de prueba usando una herramienta de prueba.
5. **Comparar los resultados reales y esperados y adoptar una acción correctiva según se requiera.**

La PBM ayuda a descubrir errores en el comportamiento del software y, como consecuencia, es extremadamente útil cuando se prueban aplicaciones impulsadas por un evento.

18.8 PRUEBA PARA ENTORNOS, ARQUITECTURAS Y APLICACIONES ESPECIALIZADOS

En ocasiones, los lineamientos y enfoques únicos para pruebas se garantizan cuando se consideran entornos, arquitecturas y aplicaciones especializados. Aunque las técnicas de prueba estudiadas anteriormente en este capítulo, y en los capítulos 19 y 20, con frecuencia pueden

7 La prueba basada en modelo también puede usarse cuando los requerimientos del software se representan con tablas de decisión, gramáticas o cadenas de Markov [DAC03].

adaptarse a situaciones especializadas, vale la pena considerar individualmente sus necesidades únicas.

18.8.1 Pruebas de interfaces gráficas de usuario

Las interfaces gráficas para usuario (GUI, por sus siglas en inglés) presentan interesantes retos de prueba. Puesto que los componentes reutilizables ahora son parte común en los entornos de desarrollo GUI, la creación de la interfaz para el usuario se ha vuelto menos consumidora de tiempo y más precisa (capítulo 11). Pero, al mismo tiempo, la complejidad de las GUI ha crecido, lo que conduce a más dificultad en el diseño y ejecución de los casos de prueba.

Debido a que muchas GUI modernas tienen la misma apariencia y ambiente, puede derivarse una serie de pruebas estándar. Es posible usar las gráficas de modelado de estado finito para derivar una serie de pruebas que aborden objetos de datos y programa específicos que sean relevantes para la GUI. Esta técnica de prueba basada en modelo se estudió en la sección 18.7.

Como producto del gran número de permutaciones asociadas con las operaciones de la GUI, la prueba de GUI debe abordarse usando herramientas automatizadas. Durante los últimos años apareció una amplia gama de herramientas de prueba GUI.[8]

18.8.2 Prueba de arquitecturas cliente-servidor

Cita:

"El tema de las pruebas es un área en la que existe una buena cantidad de comunión entre los sistemas tradicionales y los sistemas cliente-servidor."

Kelley Bourne

La naturaleza distribuida de los entornos cliente-servidor, los conflictos de rendimiento asociados con el procesamiento de transacciones, la potencial presencia de algunas plataformas de hardware diferentes, las complejidades de la comunicación en red, la necesidad de atender a múltiples clientes desde una base de datos centralizada (o en algunos casos, distribuida) y los requerimientos de coordinación impuestos al servidor se combinan para realizar las pruebas de las arquitecturas cliente-servidor y el software que reside dentro de ellas es considerablemente más difícil que las aplicaciones independientes. De hecho, estudios industriales recientes indican un aumento significativo en el tiempo y costo de las pruebas cuando se desarrollan los entornos cliente-servidor.

WebRef

En **www.csst-technologies.com** pueden encontrarse información y recursos útiles para pruebas cliente-servidor.

En general, la prueba del software cliente-servidor ocurre en tres niveles diferentes: 1) las aplicaciones cliente individuales se prueban en un modo "desconectado"; no se considera la operación del servidor ni la red subyacente. 2) El software cliente y las aplicaciones servidor asociadas se prueban en concierto, pero las operaciones de red no se revisan de manera explícita. 3) Se prueba la arquitectura cliente-servidor completa, incluidos la operación de red y el rendimiento.

Aunque en cada uno de estos niveles de detalle se realizan muchos tipos de pruebas diferentes, para las aplicaciones cliente-servidor se encuentran comúnmente los siguientes abordajes de prueba:

? ¿Qué tipos de pruebas se llevan a cabo para los sistemas cliente-servidor?

- **Pruebas de función de aplicación.** La funcionalidad de las aplicaciones cliente se prueba usando los métodos analizados anteriormente en este capítulo y en los capítulos 19 y 20. En esencia, la aplicación se prueba en forma independiente con la intención de descubrir errores en su operación.
- **Pruebas de servidor.** Se prueban las funciones de coordinación y gestión de datos del servidor. También se considera el rendimiento del servidor (tiempo de respuesta global y cantidad de datos transmitidos).
- **Pruebas de base de datos.** Se prueban la precisión y la integridad de los datos almacenados por el servidor. Se examinan las transacciones colocadas por las aplicaciones

8 Cientos, si no miles, de recursos acerca de herramientas de prueba GUI pueden evaluarse en la web. Un buen punto de partida para herramientas de fuente abierta es **www.opensourcetesting.org/functional.php**.

cliente para asegurar que los datos se almacenen, actualicen y recuperen de manera adecuada. También se prueba la forma de archivar.

- **Pruebas de transacción.** Se crea una serie de pruebas para garantizar que cada clase de transacciones se procese de acuerdo con los requerimientos. Las pruebas se enfocan en comprobar lo correcto del procesamiento y también en los conflictos de rendimiento (por ejemplo, tiempos de procesamiento de transacción y volumen de transacción).
- **Pruebas de comunicación de red.** Estas pruebas verifican que la comunicación entre los nodos de la red ocurre de manera correcta y que el mensaje que pasa, las transacciones y el tráfico de red relacionado ocurren sin errores. Como parte de estas pruebas, también pueden realizarse pruebas de seguridad de red.

Para lograr estos abordajes de prueba, Musa [Mus93] recomienda el desarrollo de *perfiles operativos* derivados de escenarios de uso cliente-servidor.[9] Un perfil operativo indica cómo interactúan con el sistema cliente-servidor diferentes tipos de usuarios. Es decir, los perfiles proporcionan un "patrón de uso" que puede aplicarse cuando las pruebas se diseñan y ejecutan. Por ejemplo, para un tipo particular de usuario, ¿qué porcentaje de transacciones serán consultas?, ¿cuántas serán actualizaciones?, ¿cuántos serán pedidos?

Para desarrollar el perfil operativo, es necesario derivar un conjunto de escenarios que sean similares a los casos de uso (capítulos 5 y 6). Cada escenario aborda quién, dónde, qué y por qué. Es decir: quién es el usuario, dónde (en la arquitectura cliente-servidor física) ocurre la interacción del sistema, cuál es la transacción y por qué ocurre. Los escenarios pueden derivarse usando técnicas de respuesta a requerimientos (capítulo 5) o a través de análisis menos formales con los usuarios finales. Sin embargo, el resultado debe ser el mismo. Cada escenario debe proporcionar un indicio de las funciones del sistema que se requerirán para atender a un usuario particular, el orden en el que se requieren dichas funciones, la temporización y la respuesta que se espera, así como la frecuencia con la que se usa cada función. Luego, estos datos se combinan (para todos los usuarios) a fin de crear el perfil operativo. En general, el esfuerzo de prueba y el número de casos de prueba por ejecutar se asignan a cada escenario de uso con base en la frecuencia de uso y en lo crítico de las funciones realizadas.

18.8.3 Documentación de prueba y centros de ayuda

El término *prueba de software* invoca imágenes de gran número de casos de prueba preparados para revisar los programas de cómputo y los datos que manipulan. Al recordar la definición de software que se presentó en el capítulo 1, es importante notar que las pruebas también deben extenderse al tercer elemento de la configuración del software: la documentación.

Los errores en la documentación pueden ser tan devastadores para la aceptación del programa como los errores en los datos o en el código fuente. Nada es más frustrante que seguir con exactitud una guía de usuario o un centro de ayuda en línea y obtener resultados o comportamientos que no coinciden con los predichos por la documentación. Por esta razón, las pruebas de documentación deben ser parte significativa de todo plan de prueba de software.

La prueba de documentación puede abordarse en dos fases. La primera, la revisión técnica (capítulo 15), examina el documento en su claridad editorial. La segunda, prueba en vivo, usa la documentación en conjunto con el programa real.

Sorprendentemente, una prueba en vivo para la documentación puede abordarse usando técnicas que son análogas a muchos de los métodos de prueba de caja negra estudiados anteriormente. La prueba basada en gráfico puede usarse para describir el uso del programa; la partición de equivalencia y el análisis del valor de frontera pueden usarse para definir varias

9 Debe señalarse que los perfiles operativos pueden usarse para probar todo tipo de arquitecturas de sistema, no sólo arquitectura cliente-servidor.

clases de entrada e interacciones asociadas. La PBM puede usarse para garantizar que el comportamiento documentado y el comportamiento real coinciden. Entonces, el uso del programa puede rastrearse a través de la documentación.

INFORMACIÓN



Pruebas de documentación

Las siguientes preguntas deben responderse durante las pruebas de documentación y/o en el centro de ayuda:

- ¿La documentación describe con precisión cómo lograr cada modo de uso?
- ¿La descripción de cada secuencia de interacción es precisa?
- ¿Los ejemplos son precisos?
- ¿La terminología, descripciones de menú y respuestas del sistema son consistentes con el programa real?
- ¿Es relativamente fácil localizar guías dentro de la documentación?
- ¿La solución de problemas puede lograrse con facilidad usando la documentación?
- ¿La tabla de contenido y el índice del documento son consistentes, precisos y completos?
- ¿El diseño del documento (plantilla, fuentes, sangrías, gráficos) contribuye a comprender y asimilar rápidamente la información?
- ¿Todos los mensajes de error del software que se muestran al usuario se describen con más detalle en el documento? ¿Las acciones por tomar como consecuencia de un mensaje de error se delinean con claridad?
- Si se usan enlaces de hipertexto, ¿son precisos y completos?
- Si se usa hipertexto, ¿el diseño de navegación es apropiado para la información requerida?

La única forma viable para responder estas preguntas es hacer que una tercera parte independiente (por ejemplo, usuarios seleccionados) pruebe la documentación en el contexto del uso del programa. Todas las discrepancias se anotan y las áreas de ambigüedad o debilidad en el documento se definen para su potencial reescritura.

18.8.4 Prueba para sistemas de tiempo real

La naturaleza asíncrona, dependiente del tiempo de muchas aplicaciones de tiempo real, agrega un nuevo y potencialmente difícil elemento a la mezcla de pruebas: el tiempo. El diseñador de casos de prueba no sólo debe considerar los casos de prueba convencionales, sino también la manipulación de eventos (es decir, el procesamiento de interrupciones), la temporización de los datos y el paralelismo de las tareas (procesos) que manejan los datos. En muchas situaciones, probar los datos proporcionados cuando un sistema de tiempo real está en un estado dará como resultado un procesamiento adecuado, mientras que los mismos datos proporcionados cuando el sistema está en un estado diferente pueden conducir a error.

Por ejemplo, el software de tiempo real que controla una nueva fotocopiadora acepta interrupciones del operador (es decir, el operador de la máquina presiona teclas de control como RESET o DARKEN) sin error cuando la máquina saca copias (en el estado "copying"). Estas mismas interrupciones del operador, si se ingresan cuando la máquina está en el estado "jammed", generan una pantalla del código de diagnóstico que indica la ubicación del atasco que se tiene que resolver (un error).

Además, la íntima relación que existe entre el software de tiempo real y su entorno de hardware también puede causar problemas en las pruebas. Las pruebas del software deben considerar el impacto de los fallos de hardware en el procesamiento del software. Tales fallos pueden ser extremadamente difíciles de simular de manera realista.

Los métodos amplios de diseño de casos de prueba para sistemas en tiempo real continúan evolucionando. Sin embargo, puede proponerse una estrategia global de cuatro pasos:

? ¿Cuál es una estrategia efectiva para probar un sistema en tiempo real?

- **Prueba de tareas.** El primer paso en la prueba del software en tiempo real es probar cada tarea de manera independiente. Es decir, las pruebas convencionales se diseñan para cada tarea y se ejecutan independientemente durante dichas pruebas. La prueba de tareas descubre errores en lógica y función, mas no en temporización y comportamiento.

- **Prueba de comportamiento.** Con modelos de sistema creados con herramientas automatizadas, es posible simular el comportamiento de un sistema en tiempo real y examinar su comportamiento como consecuencia de eventos externos. Estas actividades de análisis pueden servir de base para el diseño de los casos de prueba que se realizan cuando se construye el software en tiempo real. Al usar una técnica similar a la partición de equivalencia (sección 18.6.2), los eventos (por ejemplo, interrupciones, señales de control) se categorizan para las pruebas. Por ejemplo, los eventos para la fotocopiadora pueden ser interrupciones del usuario (contador de restablecimiento), interrupciones mecánicas (atasco de papel), interrupciones del sistema (baja de tóner) y modos de fallo (sobrecalentamiento del rodillo). Cada uno se prueba de manera individual y el comportamiento del sistema ejecutable se examina para detectar los errores que ocurren como consecuencia del procesamiento asociado con dichos eventos. El comportamiento del modelo del sistema (desarrollado durante la actividad de análisis) y el software ejecutable pueden compararse para asegurar que actúan en conformidad. Una vez que se prueba cada clase de eventos, éstos se presentan al sistema en orden aleatorio y con frecuencia aleatoria. El comportamiento del software se examina para detectar errores de comportamiento.
- **Prueba intertarea.** Una vez aislados los errores en las tareas individuales y en el comportamiento del sistema, las pruebas se cambian a los errores relacionados con el tiempo. Las tareas asíncronas que se sabe que se comunican mutuamente se prueban con diferentes tasas de datos y carga de procesamiento para determinar si ocurrirán errores de sincronización intertarea. Además, las tareas que se comunican vía cola de mensaje o almacenamiento de datos se prueban para descubrir errores en el tamaño de estas áreas de almacenamiento de datos.
- **Prueba de sistema.** Al integrar software y hardware, se lleva a cabo un amplio rango de pruebas del sistema con la intención de descubrir errores en la interfaz software-hardware. La mayoría de los sistemas en tiempo real procesan las interrupciones. Por tanto, probar la manipulación de estos eventos booleanos es esencial. Al usar el diagrama de estado (capítulo 7), el examinador desarrolla una lista de las posibles interrupciones y del procesamiento que ocurre como consecuencia de las interrupciones. Entonces se diseñan pruebas para valorar las siguientes características del sistema:
  - ¿Las prioridades de interrupción se asignan y manejan de manera adecuada?
  - ¿El procesamiento para cada interrupción se maneja de manera correcta?
  - ¿El rendimiento (por ejemplo, tiempo de procesamiento) de cada procedimiento de manejo de interrupción se apega a los requerimientos?
  - ¿Un alto volumen de interrupciones que llegan en momentos críticos crea problemas en el funcionamiento o en el rendimiento?

Además, las áreas de datos globales que se usan para transferir como parte del procesamiento de interrupción deben probarse a fin de valorar el potencial para la generación de efectos colaterales.

18.9 PATRONES PARA PRUEBAS DE SOFTWARE

WebRef

Un catálogo de patrones de prueba de software puede encontrarse en **www.rbsc.com/pages/TestPatternList.htm**

El uso de patrones como un mecanismo para describir soluciones a problemas de diseño específicos se estudió en el capítulo 12. Pero los patrones también pueden usarse para proponer soluciones a otras situaciones de ingeniería de software; en este caso, prueba del software. Los *patrones de prueba* describen problemas y soluciones de prueba comunes que pueden auxiliar en su tratamiento.

Los patrones de prueba no sólo proporcionan lineamientos útiles conforme comienzan las actividades de prueba; también proporcionan tres beneficios adicionales descritos por Marick [Mar02]:



Los patrones de prueba pueden ayudar al equipo de software a comunicarse de manera más efectiva acerca de las pruebas y comprender mejor las fuerzas que conducen a un enfoque de prueba específico.

1. Proporcionan un vocabulario para quienes solucionan problemas. "Oiga, usted sabe, debemos usar un objeto nulo".
2. Enfocan la atención en las fuerzas que hay detrás de un problema. Esto permite que los diseñadores [de caso de prueba] entiendan mejor cuándo y por qué se aplica una solución.
3. Alientan el pensamiento iterativo. Cada solución crea un nuevo contexto en el que pueden resolverse nuevos problemas.

Aunque estos beneficios son "leves", no deben pasarse por alto. Gran parte de las pruebas del software, incluso durante la década pasada, han sido actividades *ad hoc*. Si los patrones de prueba pueden ayudar a un equipo de software a comunicarse de manera más efectiva acerca de las pruebas, a comprender las fuerzas de motivación que conducen a un enfoque específico para las pruebas y a abordar el diseño de las pruebas como una actividad evolutiva en la que cada iteración resulta en una suite más completa de casos de prueba, entonces los patrones lograron mucho.

Los patrones de prueba se describen en forma muy similar a los patrones de diseño (capítulo 12). En la literatura se han propuesto decenas de patrones de prueba (por ejemplo, [Mar02]). Los siguientes tres (presentados sólo en forma resumida) proporcionan ejemplos representativos:

WebRef

Patrones que describen la organización, eficiencia, estrategia y resolución de problemas de las pruebas pueden encontrarse en **www.testing.com/test-patterns/patterns**.

Nombre del patrón: **PairTesting**

Resumen: Patrón orientado a proceso, **PairTesting** describe una técnica que es análoga a la programación por parejas (capítulo 3) en la que dos examinadores trabajan en conjunto para diseñar y ejecutar una serie de pruebas que pueden aplicarse a actividades de prueba de unidad, integración o validación.

Nombre del patrón: **SeparateTestInterface**

Resumen: Hay necesidad de probar cada clase en un sistema orientado a objetos, incluidas "clases internas" (es decir, clases que no exponen alguna interfaz afuera del componente que los usa). El patrón **SeparateTestInterface** describe cómo crear "una interfaz de prueba que puede usarse para describir pruebas específicas sobre clases que son visibles solamente de manera interna en un componente" [Lan01].

Nombre del patrón: **ScenarioTesting**

Resumen: Una vez realizadas las pruebas de unidad e integración, hay necesidad de determinar si el software se desempeñará en forma que satisfaga a los usuarios. El patrón **ScenarioTesting** describe una técnica para revisar el software desde el punto de vista del usuario. Un fallo en este nivel indica que el software fracasó para satisfacer un requisito visible del usuario [Kan01].

Un análisis amplio de los patrones de prueba está más allá del ámbito de este libro. Si tiene más interés, vea [Bin99] y [Mar02] para información adicional acerca de este importante tema.

18.10 Resumen

El objetivo principal para el diseño de casos de prueba es derivar un conjunto de pruebas que tienen la mayor probabilidad de descubrir errores en el software. Para lograr este objetivo, se usan dos categorías diferentes de técnicas de diseño de caso de prueba: pruebas de caja blanca y pruebas de caja negra.

Las pruebas de caja blanca se enfocan en la estructura de control del programa. Los casos de prueba se derivan para asegurar que todos los enunciados en el programa se ejecutaron al me-

nos una vez durante las pruebas y que todas las condiciones lógicas se revisaron. La prueba de ruta o trayectoria básica, una técnica de caja blanca, usa gráficos de programa (o matrices gráficas) para derivar el conjunto de pruebas linealmente independientes que garantizarán la cobertura del enunciado. Las pruebas de condición y de flujo de datos revisan aún más la lógica del programa, y la prueba de bucles complementa otras técnicas de caja blanca al proporcionar un procedimiento para revisar los bucles de varios grados de complejidad.

Hetzel [Het84] describe las pruebas de caja blanca como "pruebas en lo pequeño". Su implicación es que las pruebas de caja blanca que se consideraron en este capítulo por lo general se aplican a pequeños componentes del programa (por ejemplo, módulos o pequeños grupos de módulos). Las pruebas de caja blanca, por otra parte, amplían el foco y pueden llamarse "pruebas en lo grande".

Las pruebas de caja negra se diseñan para validar los requerimientos funcionales sin considerar el funcionamiento interno de un programa. Las técnicas de prueba de caja negra se enfocan en el dominio de información del software, y derivan casos de prueba mediante la partición de los dominios de entrada y salida de un programa en forma que proporciona cobertura de prueba profunda. La partición de equivalencia divide el dominio de entrada en clases de datos que es probable que revisen una función de software específica. El análisis del valor de frontera sondea la habilidad del programa para manejar datos en los límites de lo aceptable. La prueba de arreglo ortogonal proporciona un método sistemático eficiente para probar sistemas con pequeño número de parámetros de entrada. La prueba basada en modelo usa elementos del modelo de requerimientos para probar el comportamiento de una aplicación.

Los métodos de prueba especializados abarcan un amplio arreglo de capacidades de software y áreas de aplicación. La prueba para interfaces gráficas de usuario, arquitecturas cliente-servidor, documentación y centros de ayuda, y los sistemas en tiempo real requieren cada uno lineamientos y técnicas especializadas.

Con frecuencia, los desarrolladores de software experimentados dicen: "las pruebas nunca terminan, sólo se transfieren de uno [el ingeniero de software] al cliente. Cada vez que el cliente usa el programa, se realiza una prueba". Al aplicar el diseño de casos de prueba, pueden lograrse pruebas más completas y, en consecuencia, descubrir y corregir el mayor número de errores antes de comenzar "las pruebas del cliente".

Problemas y puntos para reflexionar

18.1. Myers [Mye79] usa el siguiente programa como una autovaloración de su habilidad para especificar pruebas adecuadas: un programa lee tres valores enteros. Los tres se interpretan como representación de las longitudes de los lados de un triángulo. El programa imprime un mensaje que indica si el triángulo es escaleno, isósceles o equilátero. Desarrolle un conjunto de casos de prueba que crea que probarán este programa de manera adecuada.

18.2. Diseñe e implemente el programa (con manipulación de error donde sea adecuado) que se especifica en el problema 18.1. Derive un gráfico de flujo para el programa y aplique prueba de ruta básica para desarrollar casos de prueba que garanticen la prueba de todos los enunciados en el programa. Ejecute los casos y muestre sus resultados.

18.3. ¿Puede pensar en algunos objetivos de prueba adicionales que no se estudiaron en la sección 18.1.1?

18.4. Seleccione un componente de software que haya diseñado e implementado recientemente. Diseñe un conjunto de casos de prueba que garantice que todos los enunciados se ejecutan, usando prueba de ruta o trayectoria básica.

18.5. Especifique, diseñe e implemente una herramienta de software que calcule la complejidad ciclomática para el lenguaje de programación de su elección. Use la matriz de grafo como la estructura de datos operativa en su diseño.

18.6. Lea Beizer [Bei95] o una fuente en web relacionada (por ejemplo, **www.laynetworks.com/Discret%20Mathematics_1g.htm**) y determine cómo puede extenderse el programa que desarrolló en el problema 18.5 a fin de alojar varias enlaces ponderados. Extienda su herramienta para procesar probabilidades de ejecución o tiempos de procesamiento de liga.

18.7. Diseñe una herramienta automatizada que reconozca bucles y que los clasifique como se indica en la sección 18.5.3.

18.8. Extienda la herramienta descrita en el problema 18.7 a fin de generar casos de prueba para cada categoría de bucle, una vez encontrada. Será necesario realizar esta función de manera interactiva con el examinador.

18.9. Proporcione al menos tres ejemplos en los que la prueba de caja negra puede dar la impresión de que "todo está bien", mientras que las pruebas de caja blanca pueden descubrir un error. Proporcione al menos tres ejemplos en los que las pruebas de caja blanca pueden dar la impresión de que "todo está bien", mientras que las pruebas de caja negra pueden descubrir un error.

18.10. ¿Las pruebas exhaustivas (incluso si es posible para programas muy pequeños) garantizarán que el programa es 100 por ciento correcto?

18.11. Pruebe un manual de usuario (o centro de ayuda) para una aplicación que use con frecuencia. Encuentre al menos un error en la documentación.

Lecturas adicionales y fuentes de información

Virtualmente, todos los libros dedicados a las pruebas de software consideran tanto estrategia como tácticas. Por tanto, las lecturas adicionales anotadas para el capítulo 17 son igualmente aplicables para este capítulo. Everett y Raymond (*Software Testing,* Wiley-IEEE Computer Society Press, 2007), Black (*Pragmatic Software Testing,* Wiley, 2007), Spiller *et al*. (*Software Testing Process: Test Management,* Rocky Nook, 2007), Perry (*Effective Methods for Software Testing,* 3d. ed., Wiley, 2005), Lewis (*Software Testing and Continuous Quality Improvement,* 2a. ed., Auerbach, 2004), Loveland *et al*. (*Software Testing Techniques,* Charles River Media, 2004), Burnstein (*Practical Software Testing,* Springer, 2003), Dustin (*Effective Software Testing,* Addison-Wesley, 2002), Craig y Kaskiel (*Systematic Software Testing,* Artech House, 2002), Tamres (*Introducing Software Testing,* Addison-Wesley, 2002) y Whittaker (*How to Break Software,* Addison-Wesley, 2002) son sólo una pequeña muestra de muchos libros que analizan los principios, conceptos, estrategias y métodos de las pruebas.

Una segunda edición del texto clásico de Myers [Mye79], producido por Myers *et al.* (*The Art of Software Testing*, 2a. ed., Wiley, 2004), cubre con mucho detalle las técnicas de diseño de casos de prueba. Pezze y Young (*Software Testing and Analysis,* Wiley, 2007), Perry (*Effective Methods for Software Testing,* 3a. ed., Wiley, 2006), Copeland (*A Practitioner's Guide to Software Test Design,* Artech, 2003), Hutcheson (*Software Testing Fundamentals,* Wiley, 2003), Jorgensen (*Software Testing: A Craftsman's Approach,* 2a. ed., CRC Press, 2002) proporcionan cada uno presentaciones útiles de los métodos y técnicas del diseño de casos de prueba. El texto clásico de Beizer [Bei90] proporciona una amplia cobertura de las técnicas de caja blanca e introduce un nivel de rigor matemático que con frecuencia falta en otros tratamientos de las pruebas. Su último libro [Bei95] presenta un tratamiento conciso de métodos importantes.

La prueba del software es una actividad que consume muchos recursos. Es por esto que muchas organizaciones automatizan partes del proceso de prueba. Los libros de Li y Wu (*Effective Software Test Automation,* Sybex, 2004); Mosely y Posey (*Just Enough Software Test Automation,* Prentice-Hall, 2002); Dustin, Rashka, y Poston (*Automated Software Testing: Introduction, Management, and Performance,* Addison-Wesley, 1999); Graham *et al*. (*Software Test Automation*, Addison-Wesley, 1999) y Poston (*Automating Specification-Based Software Testing,* IEEE Computer Society, 1996) exponen herramientas, estrategias y métodos para pruebas automatizadas. Nquyen *et al.* (*Global Software Test Automation*, Happy About Press, 2006) presentan un panorama ejecutivo de la automatización de las pruebas.

Thomas *et al.* (*Java Testing Patterns*, Wiley, 2004) y Binder [Bin99] describen patrones de prueba que abarcan pruebas de métodos, clases/grupos, subsistemas, reutilización de componentes, marcos conceptuales y sistemas, así como la automatización de las pruebas y la prueba de bases de datos especializadas.

En internet está disponible una amplia variedad de recursos de información acerca de los métodos de diseño de casos de pruebas. Una lista actualizada de referencias en la World Wide Web que son relevantes para las técnicas de prueba puede encontrarse en el sitio del libro: **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**